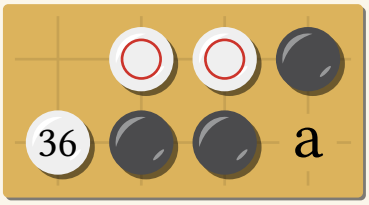
gobanist



Go boards, games and problems for Typst

Manual for version 0.1.0

Jonatán Perren Hug
jonatan.perren@gmail.com

Contents

1 Overview 4
- 1.1 What gobanist is 4
- 1.2 First steps 4
- 1.3 How to read this manual 4
- 1.4 For pgf-go users 5

2 Kāru's boards 7
- 2.1 The first game 7
- 2.2 A position by hand 8
- 2.3 Moves 10
- 2.4 Partial board and marks 11
- 2.5 Explaining a fight: arrows, letters and views 12
- 2.6 A ko: captures and notes 14
- 2.7 A whole game 16
- 2.8 Variations: going on from a board 17
- 2.9 The end of the game: territory 19
- 2.10 A problem book 20
- 2.11 A wooden board: a profile with images 22
- 2.12 Slides 25
- 2.13 A joseki dictionary 26
- 2.14 Other languages 27
- 2.15 Writing less: the coordinate syntax 28

3 Settings 30
- 3.1 The options 30
- 3.2 The kernel 31
- 3.3 Languages 31

4 Coordinate syntax 33
- 4.1 Single forms 33
- 4.2 Paths 35
- 4.3 The order of the intersections 37
- 4.4 Removing intersections 38

5 The board 40
- 5.1 The grid 42
- 5.2 Star points 44
- 5.3 The background 46
- 5.4 Partial board 50
- 5.5 The border coordinates 51

6 Stones, moves and prisoners 54
- 6.1 Stones 54
- 6.2 Moves 60
- 6.3 Prisoners 62

7 Marks, arrows and lines 64
- 7.1 The shapes 64

7.2 The options of the marks · · · 66
7.3 Shapes of your own · · · 68
7.4 Arrows and lines · · · 69
8 The game: captures, notes and territory · · · 72
8.1 Captures · · · 72
8.2 Moves on points already played · · · 73
8.3 Territory · · · 73
8.4 Score · · · 78
9 Boards as values and views · · · 80
9.1 Going on from a board · · · 80
9.2 Views in the text · · · 81
10 Games in SGF · · · 84
10.1 Reading a game · · · 84
10.2 A game on a board · · · 85
10.3 Data and comments · · · 88
10.4 Diagrams as in books · · · 90
10.5 Problems · · · 96
11 Layout of the diagrams · · · 101
12 Players and profiles · · · 104
12.1 Players · · · 104
12.2 Profiles · · · 105
13 Catalog of profiles · · · 106
13.1 default · · · 106
13.2 pgfgo · · · 106
13.3 pgfgosmooth · · · 107
13.4 wain · · · 107
13.5 simple · · · 108
13.6 draft · · · 108
13.7 shell · · · 109
13.8 yunzi · · · 110
13.9 baduktv · · · 110
14 News · · · 111
Index of functions and options · · · 112

1 Overview

1 What gobanist is

gobanist is a Typst package to draw Go boards, positions, games and problems. It is a port of pgf-go, a LaTeX package published two years ago by the same author, with the same ideas and almost all of its options, written with Typst values:

- Games in SGF: a game on one board, series of diagrams as in books, and problems with their solutions.
- The rules of the game, in a WebAssembly kernel: captures, "N at M" notes for kos, territory and score.
- Profiles for the board and the players, which can use images: wood in the background, slate and shell stones.
- Boards as values, to follow a variation from a position or to show part of a board in the middle of a paragraph.
- Marks, arrows and lines, and mark shapes of your own.
- A grid layout to arrange diagrams of the same size.
- The texts of the package in ten languages, and border coordinates in the Western, Japanese or Chinese style.
- A syntax to write many coordinates at once: rows, columns, rectangles, paths and ladders.

The drawing, the coordinates and the options are written in Typst. What playing the game needs (captures, territory, regions, score and reading SGF files) is done by a WebAssembly plugin compiled from Rust code (p. 31), which comes with the package.

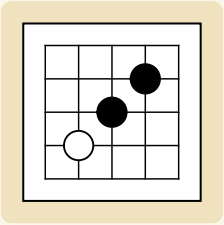
2 First steps

gobanist is imported like any other package. It needs Typst 0.15 or later.

```
#import "@preview/gobanist:0.1.0": *

#goban(stones(black: "C3, D4", white: "E5"))
```

Everything that is drawn goes in a `goban`: the stones, the moves, the marks, the arrows, the territory or a game read from a file. The options of the board go in the same `goban`, named (`size: 13`), and those of the whole document, in `gobanist-set` (p. 30).



```
#goban(size: 5, stones(black: "C3, D4", white: "B2"))
```

3 How to read this manual

The manual has three parts: this overview; the tutorials (p. 7), in which Kāru prepares an article, a problem book and some lessons, and learns the package along the way; and the reference, with every function and every option, ordered by object. At the end there are the catalog of profiles and an index of the functions and the options.

It is best to go through the tutorials first, in one go, to get an overall picture of everything, and then the reference, to see everything that can be configured. For something specific:

- **Drawing a game saved in SGF**: the tutorial on p. 7 and, for the whole game in diagrams, the one on p. 16. The reference is on p. 84.
- **Drawing a position by hand**: the tutorial on p. 8, and the stones on p. 54. To write fewer coordinates, the tutorial on p. 28 and the complete syntax on p. 33.
- **Numbering moves**: the tutorial on p. 10, and `moves` on p. 60.
- **Showing only part of the board or marking points**: the tutorial on p. 11 `partial` on p. 50, and the marks on p. 64.
- **Arrows, lines and letters**: the tutorial on p. 12, and the reference on p. 69.
- **Showing a stone or part of the board inside the text**: the end of the tutorial on p. 12, and `goban-view` on p. 81.
- **Captures, kos and "N at M" notes**: the tutorial on p. 14, and the reference on p. 72.
- **Marking the territory and counting the points**: the tutorial on p. 19, and `territory` and `goban-score` on p. 73.
- **Going on from a position, for example to show variations**: the tutorial on p. 17, and `board` and `resume` on p. 80.
- **Josekis from a dictionary in SGF**: the tutorial on p. 26, and `variation` and `variations` on p. 84.
- **Problems (tsumego) with their solutions**: the tutorial on p. 20, and `sgf-problem` on p. 96.
- **Laying out several diagrams in a grid**: the tutorials on pp. 16 and 20, and `layout` and `goban-layout` on p. 101.
- **Changing the style of every board**: the end of the tutorial on p. 8 and the catalog (p. 106). To change a detail, `gobanist-set` (p. 30) and the options of each object; to save a style of your own, the profiles (p. 105).
- **Using images on the board and on the stones**: the tutorial on p. 22 the background and the grid on p. 40, and the stones on p. 58.
- **Slides**: the tutorial on p. 25.
- **Writing in another language**: the tutorial on p. 27, and the reference on p. 31.

A general rule: the branches that turn something on or off (the grid, the background, the shadows, …) have the key `on`; `true` and `false` turn them on and off, and a dictionary changes their options without turning them on (p. 30).

4 For pgf-go users

The options of gobanist are those of pgf-go, with Typst syntax: a tree of dictionaries instead of a tree of *keys*, and Typst values instead of TikZ ones. The most visible changes:

pgf-go	gobanist
`\begin{goban}[13] ... \end{goban}`	`#goban(size: 13, ...)`
`\pgfgoset{goban = {...}}`	`#gobanist-set(...)`
`\stones[b]{C3, D4}`	`stones(black: "C3, D4")`
`\stone{C3}`, `\stones{C3}`	`stones("C3")`
`\moves[w]{D2, B3}`	`moves(white: "D2, B3")`
`\mark[triangle]{C3}`	`marks(triangle: "C3")`
`\mark[custom = {!}]{D3}`	`marks(custom: "D3", content: [!])`
`\arrow{B3}{F3}`, `\line{B3}{F3}`	`arrows(("B3", "F3"))`, `lines(("B3", "F3"))`
`\territory[b]{auto}`	`territory(black: auto)`
`shine = {true, angle = 45}`	`shine: (on: true, angle: 45deg)`
`format = Alph`	`format: "A"` (a `numbering` pattern)
`label = {at = {north, west}}`	`label: (at: (top, left))`
`name = base`, `resume = base`	`#let base = board(...)`, `goban(base)`, `resume: base`
`\pgfgoview{base}{C3}`	`goban-view(base, "C3")`
`\pgfgodefinesgf{game}{game.sgf}`	`#let game = sgf-read(path("game.sgf"))`

pgf-go	gobanist
`\sgf[to = 7]{game}`	`sgf(game, to: 7)`
`\sgfdiagrams`, `\sgfproblem`	`sgf-diagrams`, `sgf-problem`
`\pgfgomovenotes`, `\pgfgoscore{b}`	`goban-notes()`, `goban-score("black")`
`pgfgolayout`	`goban-layout(...)`
`\pgfgodefinemark`	the `shapes` option
`\pgfgodefineprofile{wood}{...}`	`#let wood = (...)`, `profile: wood`

Some things behave differently:

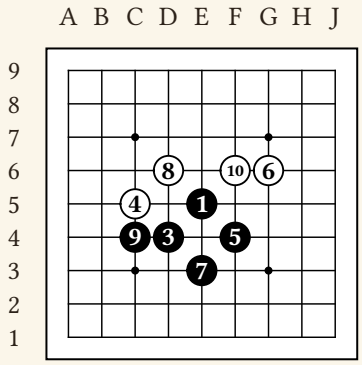
- A mark without a player on a stone takes the player of the stone (in pgf-go it was neutral).
- Typst has no warnings: what pgf-go warned about (a coordinate that cannot be understood or that is outside the board, a game of another size) is an error. A move on an occupied point, with captures, is also an error, except in `sgf-diagrams` and `sgf-problem`.
- The grays are those of xcolor: the gray of gobanist is `luma(50%)`, not Typst's `gray`.
- The board has at least 2×2 lines and at most 25 columns.
- Images are `path` or `bytes`, not file names.

And some are not ported: `\sgfanimation` (Typst does not make animated PDFs; for slides there are Touying and Polylux, p. 25), `export` and `\pgfgoimport` (a board is a value: it is saved in a `.typ` module and imported), drawing with TikZ inside the board, the `go cs` coordinate system, `\pgfgodefinegoban`, `\pgfgodefineplayer` and the partial profiles (`goban profile`, `players profile`): a profile is a dictionary, and its branches are taken.

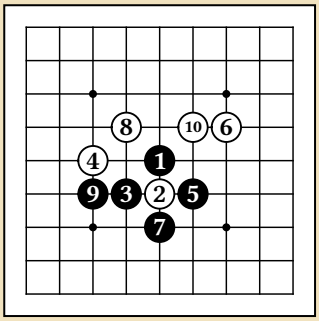
2 Kāru's boards

1 The first game

Kāru is writing an article for their students and decides to try gobanist. The first thing they want to show is a short game played in class. It is saved in an SGF file, `game.sgf`: the format in which almost every Go program and server saves games.



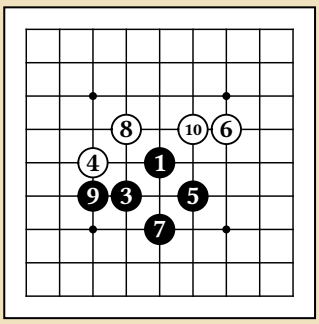
`sgf-read` reads the file and returns the game as a value, which Kāru saves with `let` to use it many times. Everything that is drawn goes in a `goban`, and so does the game: inside it, its main line is drawn. The board takes the size of the file.



```
#let game = sgf-read(path("game.sgf"))

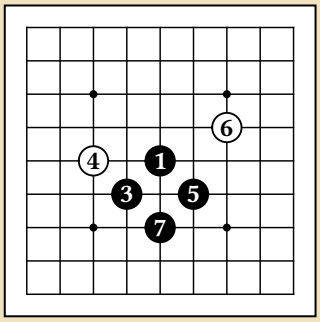
#goban(game)
```

The moves are numbered, as in books. But with move 7 Black captured the white stone 2, which is still on the board: gobanist draws what it is asked to, without applying the rules. With the option `captures`, the `goban` removes the captured stones (p. 72). The options of the board are named, next to what is drawn.



```
#goban(captures: "remove", game)
```

To comment on the game in the text, Kāru stops it at move 7 with `to`: `sgf(game, ...)` is the same game with other options. The file has a comment for that move, and `sgf-comment` writes it.



Move 7: Tesuji. Black captures.

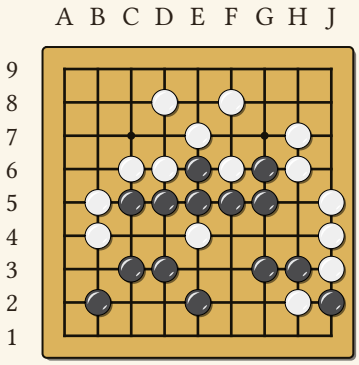
```
#goban(captures: "remove", sgf(game, to: 7))

Move 7: #sgf-comment(game, 7)
```

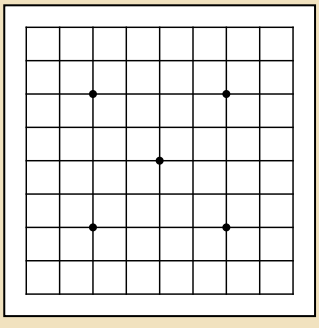
This game is 9×9; a 19×19 one is read the same way, and the board takes its size. Kāru will do so with the game on p. 16, where they will also see how to draw a long game in a series of diagrams.

2 A position by hand

For the introduction of the article, Kāru wants a position of their own, which is not in any file. It has to be drawn stone by stone.

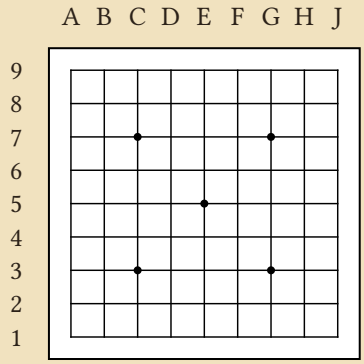


Kāru starts with the empty board. `goban` takes care of the size of the board, its color, its coordinates and everything drawn on it.



```
#goban()
```

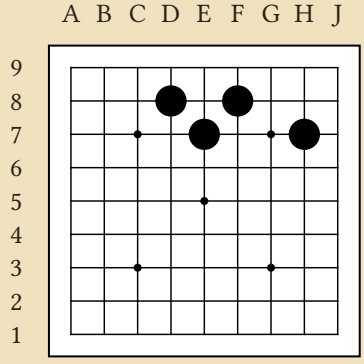
The board is 9×9, just what is needed, but the coordinates on the borders are missing. The options of each part of the board are grouped in branches: `grid`, the grid; `background`, the border and the fill, and `label`, the coordinates (p. 40). Inside `label`, `at` says on which sides they go: `top`, `bottom`, `left` and `right`, Typst's alignments. Kāru wants them at the top and on the left.




```
#goban(label: (at: (top, left)))
```

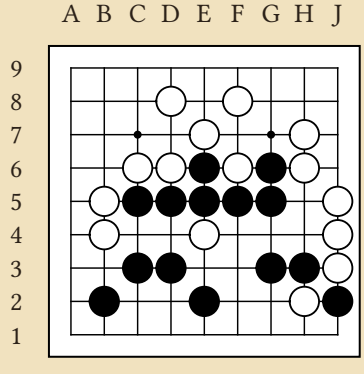
The columns go from H to J: as is customary in Go, gobanist skips the I, both in the coordinates and in the labels. If it is ever needed, the option `use-i` (p. 41) brings it back.

Now, the stones. `stones` takes the intersections, such as `"D8"`, and the player is the key: `black` or `white`. Without a player, the stone is neutral.



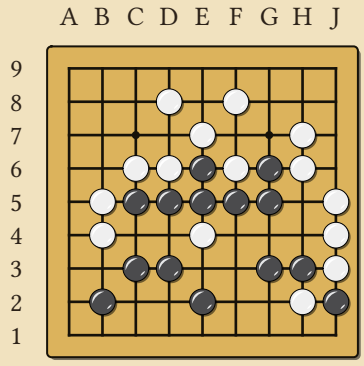

```
#goban(label: (at: (top, left)),
  stones(black: "D8, F8, E7, H7"))
```

A single string holds all the stones of a player, separated by commas, and the same `stones` can hold those of both. Strictly speaking, `black` and `white` are not colors but players: each stone takes the style of its own (p. 104).




```
#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

The style is missing. In the catalog (p. 106), Kāru finds one they like: the `pgfgo` profile. They apply it with `gobanist-set(profile: "pgfgo")` (p. 105), and from there on it holds for every board that follows.



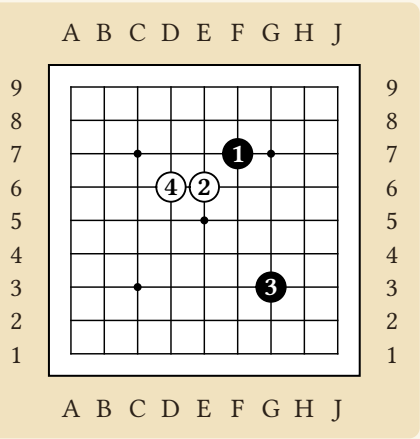
```
#gobanist-set(profile: "pgfgo")

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

The lists in this tutorial are long, and Kāru's patience is short. In the last tutorial (p. 28) they learn to write them shorter.

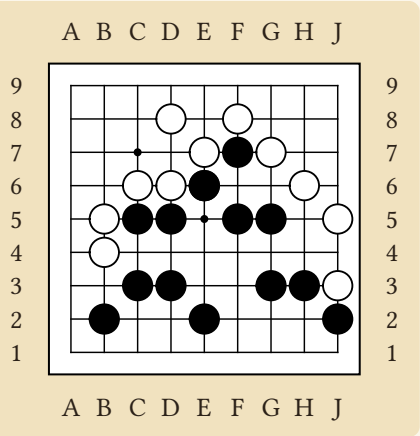
3 Moves

To explain how a fight began, Kāru wants to show the stones in the order in which they were played. With `stones` they would have to separate those of each player and would lose the order. That is what `moves` is for (p. 60): it plays the intersections in the order in which they are written, alternates the colors and numbers the moves.



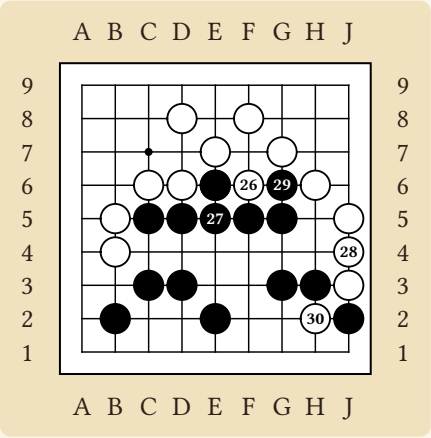
```
#goban(label: (at: (top, bottom, left, right)),
  moves("F7, E6, G3, D6"))
```

The fight began with a black stone at F7, which Kāru places with `stones`. They will not comment on the moves that follow: with `numbers: false`, they are drawn without numbers. With `white` as the key, the sequence starts with White: in `moves`, the player only says who plays first.



```
#goban(label: (at: (top, bottom, left, right)),
  stones(black: "F7"),
  moves(numbers: false, white: "E6, G3, D6, F5,
    E7, E2, F8, E6, D8, D3, G7, C5, C6, D5,
    B5, G5, B4, B2, H6, C3, J3, J2, J5, H3"))
```

They do comment on the last moves, so these go with numbers. The numbering and the turn carry on from one `moves` to the next, even if the previous moves have no numbers, and `from` chooses the number to carry on from.

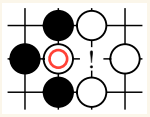



```
#goban(label: (at: (top, bottom, left, right)),
  moves(numbers: false, white: "E6, G3, D6, F5,
    E7, E2, F8, E6, D8, D3, G7, C5, C6, D5,
    B5, G5, B4, B2, H6, C3, J3, J2, J5, H3"),
  moves("F6, E5, J4, G6", from: 26),
  moves("H2"))
```

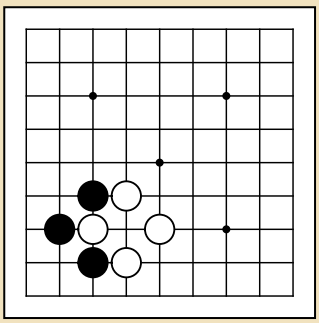
`H2` goes in another `moves` and, without saying so, is white and numbered 30: the previous move, `G6`, was Black's 29.

4 Partial board and marks

For an exercise in the article, Kāru wants to show only part of the board, with two signs: a circle on the stone in atari and a "!" on the key point.

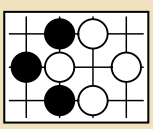


They start, as always, with the stones.



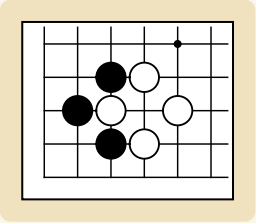
```
#goban(stones(black: "C2, B3, C4",
  white: "D4, E3, D2, C3"))
```

The option `partial` says which part of the board to show: two opposite corners of the rectangle, separated by a colon, such as `"B2:E4"` (p. 50).



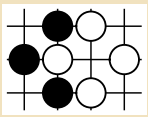
```
#goban(partial: "B2:E4",
  stones(black: "C2, B3, C4",
    white: "D4, E3, D2, C3"))
```

If Kāru does not want to look for the corners, with `partial: auto` the board chooses the part: the one used by what is drawn, with one more line around it. A side two lines or less from the border reaches the border, like the bottom and the left ones here.



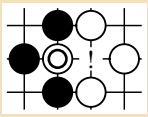
```
#goban(partial: auto,
  stones(black: "C2, B3, C4",
    white: "D4, E3, D2, C3"))
```

Kāru prefers the tight crop, so they go on with `"B2:E4"`. On a partial board, the border of the board is too much: they remove it with `line: false` in the `background`.



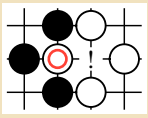
```
#goban(partial: "B2:E4",
  background: (line: false),
  stones(black: "C2, B3, C4",
    white: "D4, E3, D2, C3"))
```

The marks are placed with `marks`, with the shape as the key; there are many (p. 64). Kāru needs only two: `circle`, the circle, and `custom`, which places any content (`content`), such as "!". A mark can also go in the options of the stone, with `mark`.



```
#goban(partial: "B2:E4",
  background: (line: false),
  stones(black: "C2, B3, C4",
    white: "D4, E3, D2"),
  stones(white: "C3", mark: "circle"),
  marks(custom: "D3", content: [!]))
```

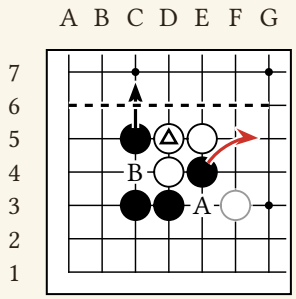
So that the circle stands out, Kāru paints it red, with the mark's `line.color`. With options, the mark is a dictionary, and the shape goes in `shape`.



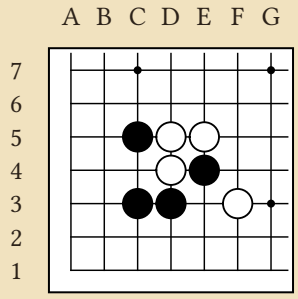
```
#goban(partial: "B2:E4",
  background: (line: false),
  stones(black: "C2, B3, C4",
    white: "D4, E3, D2"),
  stones(white: "C3", mark: (shape: "circle",
    line: (color: red))),
  marks(custom: "D3", content: [!]))
```

5 Explaining a fight: arrows, letters and views

In the second part of the article, Kāru wants to explain a fight. They need to point out the possible moves with letters, show where each group runs and, in the text, name the stones without making the reader look for them in the diagram.

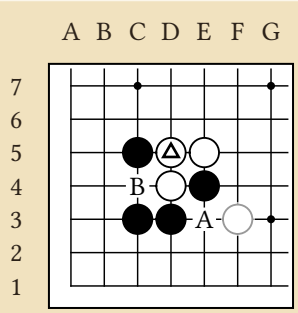


They start, as always, with the stones. To save space, they show only the corner with `partial`.



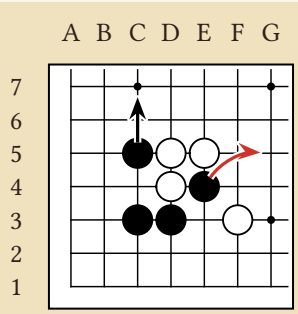
```
#goban(partial: "A1:G7",
  label: (at: (top, left)),
  stones(black: "C3, D3, E4, C5",
    white: "D4, D5, E5, F3"))
```

The candidate moves are marked with letters, with `custom`, which takes any content. The `triangle` mark highlights the stone the text talks about, and `dim` dims the white stone at F3, which no longer matters. The same `marks` can hold several shapes; two `custom`s with different content go in two `marks`.



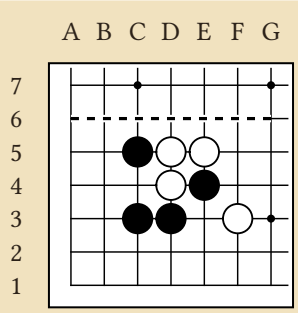
```
#goban(partial: "A1:G7",
  label: (at: (top, left)),
  stones(black: "C3, D3, E4, C5",
    white: "D4, D5, E5, F3"),
  marks(dim: "F3", triangle: "D5"),
  marks(custom: "E3", content: [A]),
  marks(custom: "C4", content: [B]))
```

To show where the groups run, Kāru uses `arrows`, which joins two intersections (p. 69). The arrow is drawn with a halo the color of the board, so that it is not confused with the grid. With `bend`, it is an arc; with `shorten`, it does not cover the stones at its ends.



```
#goban(partial: "A1:G7",
  label: (at: (top, left)),
  stones(black: "C3, D3, E4, C5",
    white: "D4, D5, E5, F3"),
  arrows(("C5", "C7"), shorten: 4pt),
  arrows(("E4", "G5"), color: red.darken(20%),
    bend: 30deg, shorten: 4pt))
```

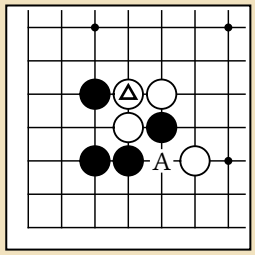
`lines` is like `arrows`, but without a tip, and it takes the same stroke options, such as `dash`.



```
#goban(partial: "A1:G7",
  label: (at: (top, left)),
  stones(black: "C3, D3, E4, C5",
    white: "D4, D5, E5, F3"),
  lines(("A6", "G6"), dash: "dashed"))
```

Now, the text. Kāru wants to write "the black stone at E4" and show it right there. For that they save the board as a value, with `board`, which takes the same as `goban` but draws nothing. To draw it, the board goes first

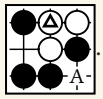
in a `goban`, which adds its options to it, such as the visible part. And `goban-view` shows part of that board in the middle of the paragraph, as it was at the end (p. 81): a point or a rectangle.

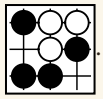



```
#let fight = board(
  stones(black: "C3, D3, E4, C5",
    white: "D4, D5, E5, F3"),
  marks(triangle: "D5"),
  marks(custom: "E3", content: [A]))

#goban(fight, partial: "A1:G7")
```

```
The black stone #goban-view(fight, "E4") is cut off from its friends.
If it plays at A, it connects: #goban-view(fight, "C3:E5").
Without marks: #goban-view(fight, "C3:E5", marks: false).
```

The black stone  is cut off from its friends. If it plays at A, it connects: 

. Without marks:  .

The view has the style of the board, but `view.goban` can change it: Kāru prefers to remove the background in every view of the article.

```
#gobanist-set(view: (goban: (background: false)))

The black stone #goban-view(fight, "E4") is cut off from its friends.
If it plays at A, it connects: #goban-view(fight, "C3:E5").
```

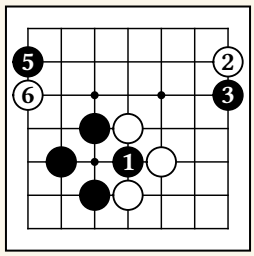
The black stone  is cut off from its friends. If it plays at A, it connects: 

.

Since the board is a value, the view can go before the diagram that draws it: `fight` is known from the `let`, not from the `goban`.

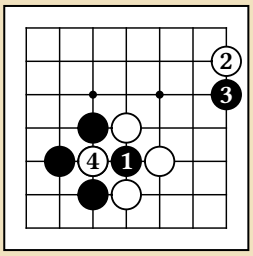
6 A ko: captures and notes

In the first tutorial, `captures` removed a captured stone from a game in SGF. To explain a ko, Kāru needs the same with moves written by hand: to place them in order and have gobanist remove the captured stones.



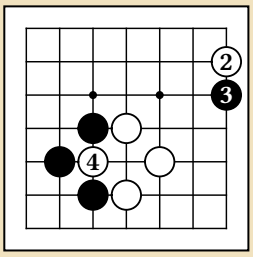
7 at 1

Black takes the ko at D3 (1), White threatens elsewhere (2), Black answers (3) and White takes back (4). As it is, the moves are drawn on top of one another and no stone leaves the board.



```
#goban(size: 7,
  stones(black: "B3, C2, C4",
    white: "C3, D2, D4, E3"),
  moves("D3, G6, G5, C3"))
```

With `captures: "remove"` (p. 72), the `goban` keeps count of the liberties and removes each group left without them. The white stone at C3 disappears with move 1, and black 1, with move 4. `goban-prisoners` says how many stones each player captured.

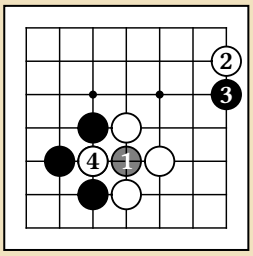


Black captured 1, White 1.

```
#goban(size: 7, captures: "remove",
  stones(black: "B3, C2, C4",
    white: "C3, D2, D4, E3"),
  moves("D3, G6, G5, C3"))

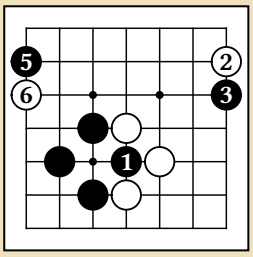
Black captured
#goban-prisoners("black"), White
#goban-prisoners("white").
```

If the captured stones must still be seen, `captures: "fade"` leaves them as prisoners.



```
#goban(size: 7, captures: "fade",
  stones(black: "B3, C2, C4",
    white: "C3, D2, D4, E3"),
  moves("D3, G6, G5, C3"))
```

The ko goes on: Black threatens (5), White answers (6) and Black takes back at D3 (7), where 1 had already been played. In books, that move is not drawn: below the diagram one writes "7 at 1". That is what the option `notes` does (p. 73), and `goban-notes` writes the notes.



7 at 1.

```
#goban(size: 7, captures: "remove",
  notes: true,
  stones(black: "B3, C2, C4",
    white: "C3, D2, D4, E3"),
  moves("D3, G6, G5, C3, A6, A5, D3"))

#goban-notes().
```

7 A whole game

The article also comments on a long game: the first one between Lee Sedol and AlphaGo, from 2016, saved in `lee.sgf`. As in the first tutorial (p. 7), there is no need to write its 186 moves: gobanist reads them from the file (p. 84).

The first thing is the data of the game, which `sgf-info` takes from the root of the file.

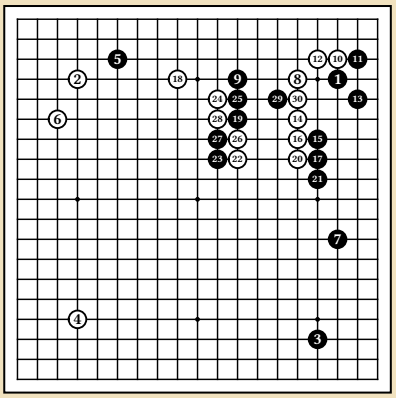
```
#let lee = sgf-read(path("lee.sgf"))

#sgf-info(lee, "PB") (Black) against #sgf-info(lee, "PW") (White),
#sgf-info(lee, "EV"), #sgf-info(lee, "RO"), #sgf-info(lee, "DT").
Result: #sgf-info(lee, "RE").
```

Lee Sedol (Black) against AlphaGo (White), Google DeepMind Challenge Match, Game 1, 2016-03-09. Result: W+Resign.

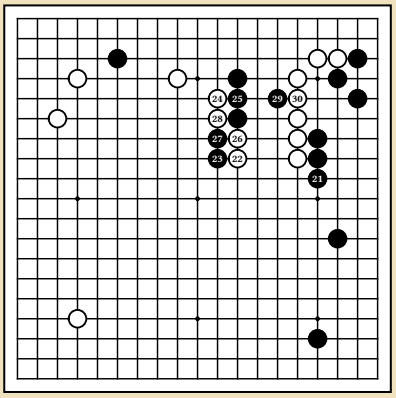
`sgf-info` writes the result as it is in the file; later on, `sgf-diagrams` writes it in words.

The game is 19×19, and the board takes that size. With `to`, Kāru shows it up to move 30, and with `scale`, they shrink the board.



```
#goban(captures: "remove", scale: 0.6,
  sgf(lee, to: 30))
```

To comment on a stretch, Kāru numbers only the moves from 21 on with `from`: the previous ones are drawn without numbers.



```
#goban(captures: "remove", scale: 0.6,
  sgf(lee, from: 21, to: 30))
```

Then Kāru wants the whole game, as in books: a diagram every 50 moves, each one with its caption and its "N at M" notes, and the result at the end. That is what `sgf-diagrams` does, which goes outside `goban`. With `layout`, the diagrams go in a two-column grid, and with `fit` they are scaled to fill them (p. 101). Here only the last two are shown, with `from`.

```
#sgf-diagrams(lee, step: 50, from: 101,
  layout: (columns: 2, fit: true))
```

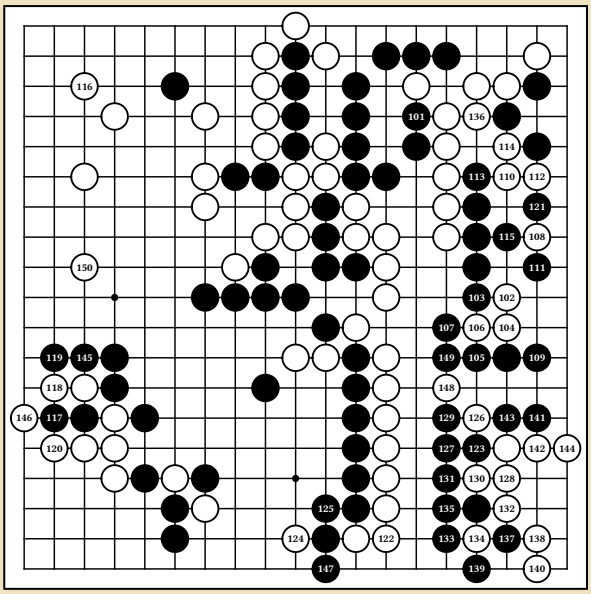


Diagram 3 (101–150)

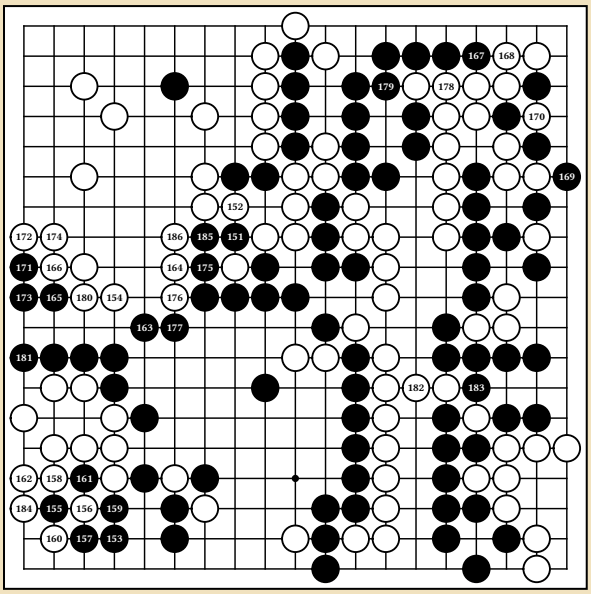


Diagram 4 (151–186)
Result: White wins by resignation

The caption and the result are in English because the document is in English (`text.lang`, p. 31). Some books restart the numbering in each diagram; with `restart`, each diagram, from the second on, numbers from 1.

```
#sgf-diagrams(lee, step: 50, from: 51, to: 150,
  restart: true, goban: (label: (at: (top, left))),
  layout: (columns: 2, fit: true))
```

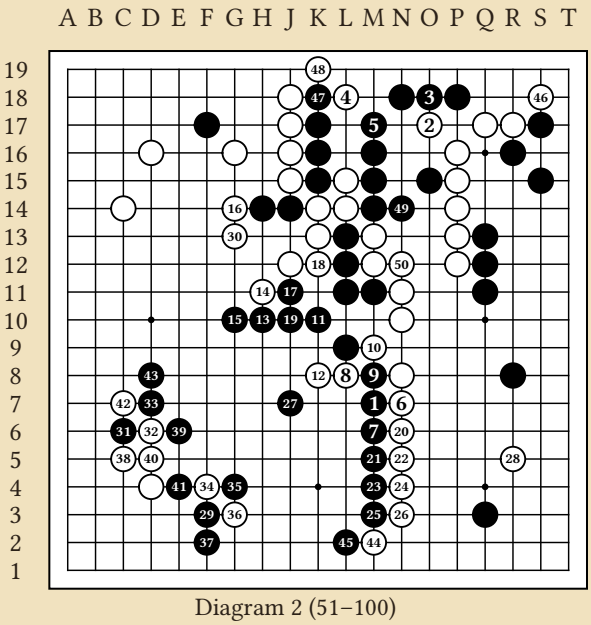


Diagram 2 (51–100)

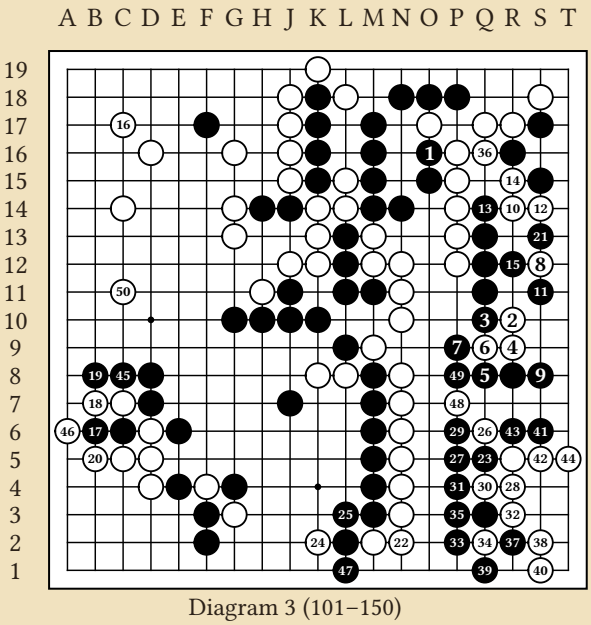
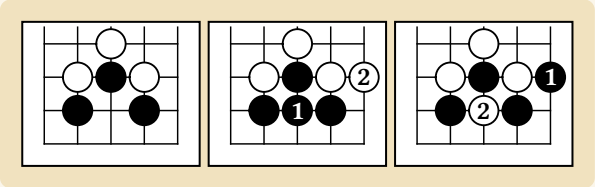


Diagram 3 (101–150)

8 Variations: going on from a board

In the comments, Kāru often shows what would have happened with another move. Copying the position in each diagram is tedious and error-prone: if a stone changes, it has to be changed in all of them. In the fight,

Kāru saved a board as a value, with `board`. With `resume`, another board starts with that one, as it was left: the numbering and the turn carry on from there (p. 80).

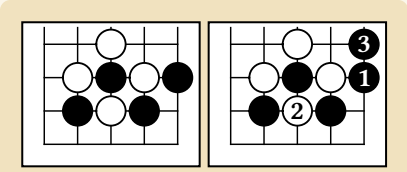


```
#gobanist-set(size: 5, partial: "A1:E4")

#let base = board(
  stones(black: "B2, C3, D2",
    white: "B3, C4, D3"))

#goban(base)
#goban(resume: base, moves(black: "C2, E3"))
#goban(resume: base, moves(black: "E3, C2"))
```

A variation can also be a value, and one can go on from it: a `board` with `resume` has what it brought plus its own. By default, `resume` brings only the stones; `numbers: true` also brings the numbers.



```
#gobanist-set(size: 5, partial: "A1:E4")

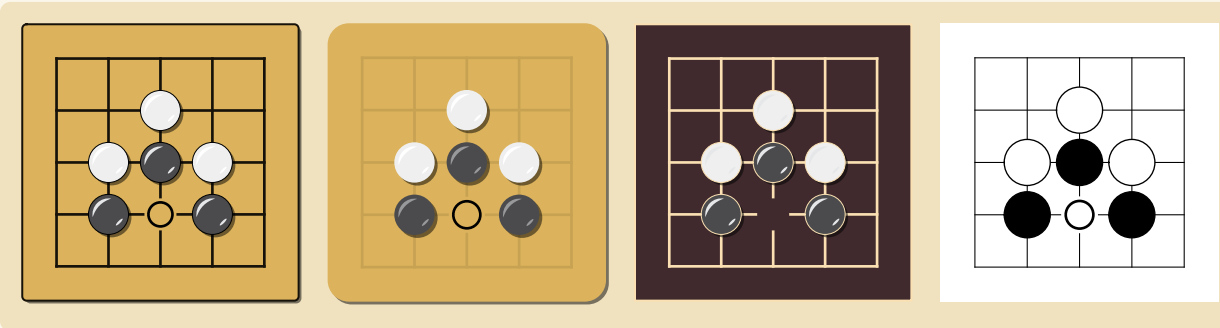
#let base = board(
  stones(black: "B2, C3, D2",
    white: "B3, C4, D3"))
#let variation = board(resume: base,
  moves(black: "E3, C2"))

#goban(resume: variation)
#goban(resume: (board: variation,
  numbers: true), moves("E4"))
```

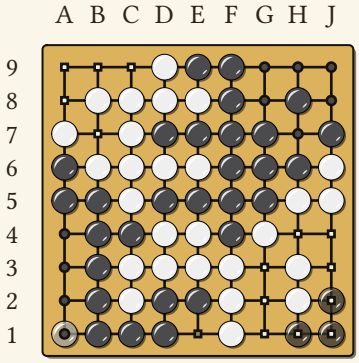
A saved board also serves to compare styles. Before choosing the profile of their article, Kāru draws the same board with several profiles from the catalog (p. 106). To put them side by side, with the same width, they use `goban-layout` (p. 101), which lays out boards made with `board`. Each one is `styles` with another profile: a board that goes first in `board` is that same board, with what is added to it.

```
#let styles = board(size: 5,
  stones(black: "B2, C3, D2", white: "B3, C4, D3"),
  marks(circle: "C2"))
#let styled(p) = board(styles, profile: p)

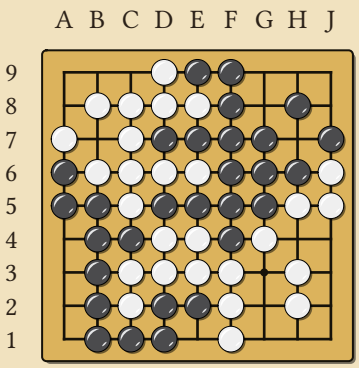
#goban-layout(columns: 4, fit: true,
  styled("pgfgo"), styled("pgfgosmooth"),
  styled("wain"), styled("simple"))
```



9 The end of the game: territory



To finish the article, Kāru wants to show how a game is counted. They start from the final position. Since it has many stones and they will draw it several times, they save it with `board`, like the variations of the previous tutorial, and the boards that follow take it first, without repeating the coordinates.

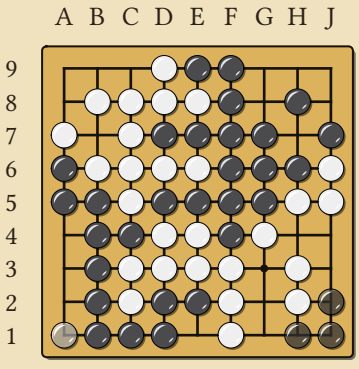



```
#gobanist-set(profile: "pgfgo")

#let final = board(stones(
  black: "A5, A6, B1, B2, B3, B4, B5, C1, C4,
    D1, D2, D5, D7, E2, E5, E7, E9, F4, F5,
    F6, F7, F8, F9, G5, G6, G7, H6, H8, J7",
  white: "A7, B6, B8, C2, C3, C5, C6, C7, C8,
    D3, D4, D6, D8, D9, E3, E4, E6, E8, F1,
    F2, F3, G4, H2, H3, H5, J5, J6"))

#goban(final, label: (at: (top, left)))
```

Next come the prisoners: the dead stones, which are removed when counting. They are drawn with `prisoners`, which takes the same as `stones` (p. 62). Kāru also saves this board, with another name.

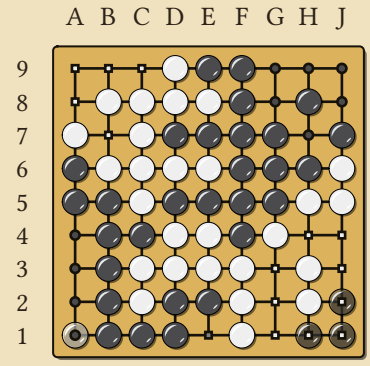



```
#gobanist-set(profile: "pgfgo")

#let final2 = board(final,
  prisoners(black: "H1, J1, J2",
    white: "A1"))

#goban(final2, label: (at: (top, left)))
```

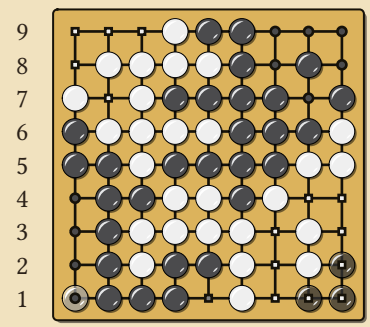
Now, the territory. One way is to mark each point with `t-square` or `t-circle`. These marks take as their fill that of the stones of their player, and as their border, that of the grid. On an empty point, a mark is neutral unless it says its player, with `player`.



```
#gobanist-set(profile: "pgfgo")

#goban(final2, label: (at: (top, left)),
  marks(player: "black",
    t-circle: "A1, A2, A3, A4, G8, G9, H9,
      J9, J8, H7"),
  marks(player: "white",
    t-square: "A9, B9, C9, A8, B7, H4, J4,
      J3, J2, J1, H1, G1, G2, G3"),
  marks(t-square: "E1"))
```

Another way is `territory` (p. 73). With a polygon, it marks the points of that region that have no living stone; with `auto`, it finds the territory of each player by itself. `goban-score` writes the score.



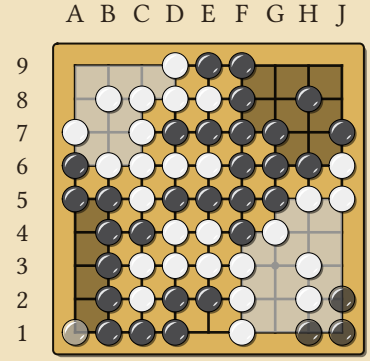
Black: 11. White: 23.5.

```
#gobanist-set(profile: "pgfgo")

#goban(final2, label: (at: (top, left)),
  territory(auto, dame: true))

Black: #goban-score("black").
White: #goban-score("white", komi: 6.5).
```

For the article, Kāru prefers the territory to look like a patch of color and not point by point. With `draw: "region"`, the territory of each color is a single polygon, whose sides run along the lines of the board.



```
#gobanist-set(profile: "pgfgo")

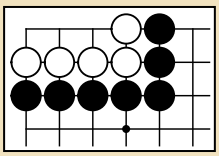
#goban(final2, label: (at: (top, left)),
  territory(auto, draw: "region"))
```

10 A problem book

For their students, Kāru prepares a book of life-and-death problems. Problems are usually saved in SGF too: the initial position, who plays and a tree with each variation of the solution, right or wrong, with its comments. `sgf-problem` draws all that: first the statement and then one diagram per variation. If the file does not say which part of the board to show, the part the problem uses is shown, as with `partial: auto`.

```
#let problem = sgf-read(path("problem.sgf"))

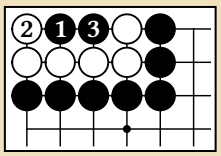
#sgf-problem(problem, title: "Problem 1")
```



Problem 1:

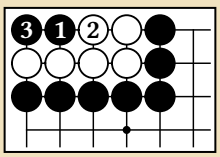
Black to play

Black to kill.



Variation 1

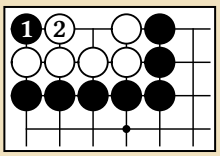
1: Tesuji. Correct: the vital point.



Variation 2

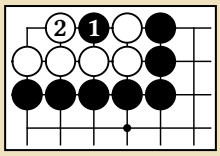
1: Tesuji. Correct: the vital point.

3: White has only one eye.



Variation 3

1: Bad move. Wrong: White lives.

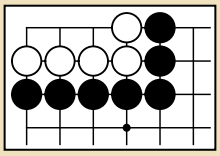


Variation 4

1: Wrong.

In a book, the solutions do not go next to the statement. Kāru puts the statements in the first part, with `solutions: false`, and the solutions at the end, with `statement: false`.

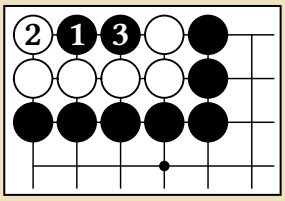
```
#sgf-problem(problem, title: "Problem 1",
  solutions: false)
// ... at the end of the book:
#sgf-problem(problem, statement: false,
  layout: (columns: 4, fit: true))
```



Problem 1:

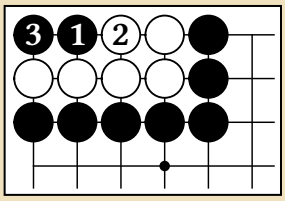
Black to play

Black to kill.



Variation 1

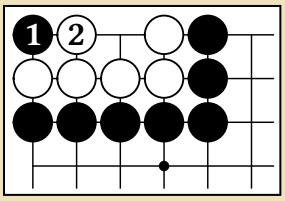
1: Tesuji. Correct: the vital point.



Variation 2

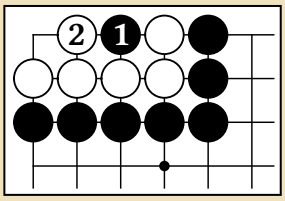
1: Tesuji. Correct: the vital point.

3: White has only one eye.



Variation 3

1: Bad move. Wrong: White lives.

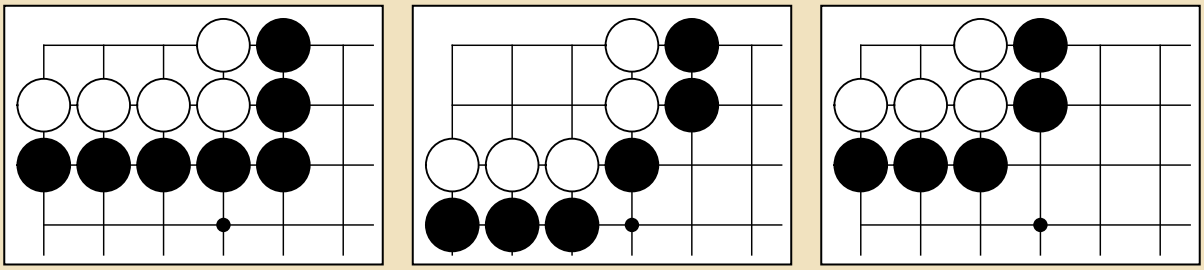


Variation 4

1: Wrong.

Some problems are not in any file, and Kāru draws them by hand, with the same part of the board that `sgf-problem` shows. Here they write the part, `"A16:F19"`: with `partial: auto`, each board would show only its own, and they would not match. So that they have the same size as the others, Kāru puts them together with those of `sgf-problem` in the same `goban-layout`, which arranges in a grid all the boards it receives.

```
#goban-layout(columns: 3, fit: true,
  sgf-problem(problem, title: "Problem 1",
    solutions: false),
  board(size: 19, partial: "A16:F19",
    stones(black: "A16, B16, C16, D17, E18, E19",
      white: "A17, B17, C17, D18, D19")),
  board(size: 19, partial: "A16:F19",
    stones(black: "A17, B17, C17, D18, D19",
      white: "A18, B18, C18, C19")))
```

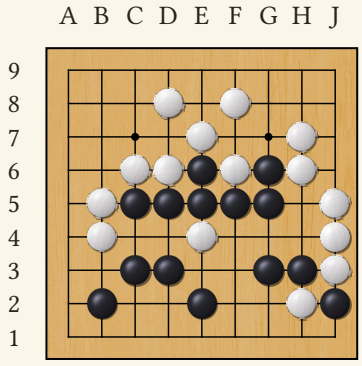


Problem 1: Black to play

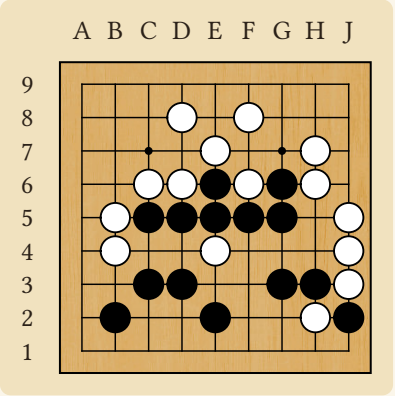
Black to kill.

11 A wooden board: a profile with images

For the cover of the book, Kāru wants a board that looks like a real one, since the budget does not stretch to a kaya one: wood, stones with volume and white stones with grain. The profiles of the catalog draw the stones as circles; for this, images are needed (PNG, JPEG, GIF, WebP, SVG or PDF). Kāru copies to their project, in the `img` folder, the ones used by the `shell` profile, which are in the `images` folder of the package: a photo of wood, `pgfgo-board.jpg`, a black slate stone, `pgfgo-shell-black.png`, and five white shell stones, `pgfgo-shell-white-1.png` to `pgfgo-shell-white-5.png`.



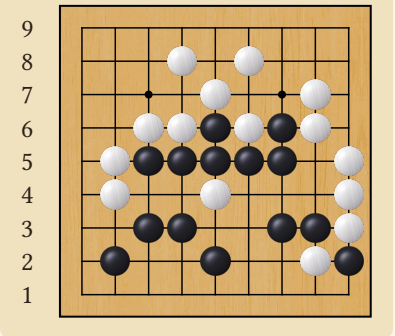
They start from the first board of the article. The wood goes in the background of the board, with `background.fill.image` (p. 40). An image is a file: a `path`, relative to the document that names it (or its `bytes`). By default, the image covers the whole board, cropped if needed; with `fit`, it can be stretched or repeated in tiles.




```
#gobanist-set(background: (fill: (
  image: path("img/pgfgo-board.jpg"))))

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

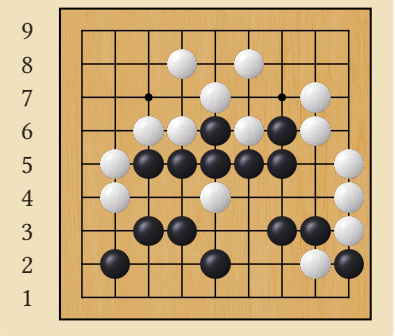
The stones of a player are drawn with an image through `stone.image` (p. 58), in the options of that player. The image takes the place of the circle: neither the fill nor the border are drawn. In an image branch, what is not a dictionary is the file.




```
#gobanist-set(
  background: (fill: (
    image: path("img/pgfgo-board.jpg"))),
  players: (
    black: (stone: (image:
      path("img/pgfgo-shell-black.png"))),
    white: (stone: (image:
      path("img/pgfgo-shell-white-1.png")))))

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

The black stones look good, but the white ones are all the same, and on a real board each shell stone has its own grain. With several files in `file`, each stone uses one, chosen by its place: a stone on a point always shows the same image, in every diagram.

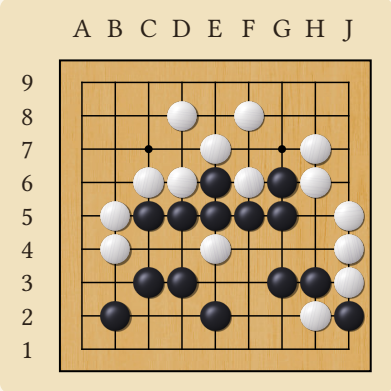



```
#let shells = range(1, 6).map(n => path(
  "img/pgfgo-shell-white-" + str(n) + ".png"))

#gobanist-set(
  background: (fill: (
    image: path("img/pgfgo-board.jpg"))),
  players: (
    black: (stone: (image:
      path("img/pgfgo-shell-black.png"))),
    white: (stone: (image: shells))))

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

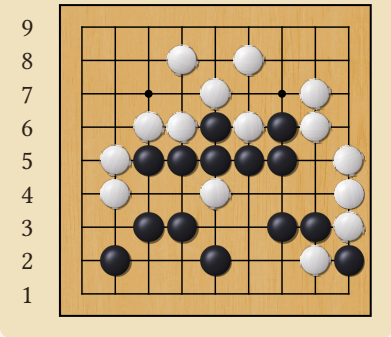
For the stones to look as if they rest on the wood, a shadow is missing. `shadow` draws it with the shape of the image (if the image has transparency, like these PNGs). Since it is the same for the three players, it goes in `stone`, at the first level: it is `players: (all: (stone: ...))`.




```
#gobanist-set(
  background: (fill: (
    image: path("img/pgfgo-board.jpg"))),
  stone: (shadow: (on: true, offset: 0.75pt,
    angle: -45deg)),
  players: (
    black: (stone: (image:
      path("img/pgfgo-shell-black.png"))),
    white: (stone: (image: shells))))

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

With five images, a repeated one can still be seen. `rotate` turns each stone by a different angle, also chosen by its place. These stones have the light at the top left, so Kāru turns them only a little, between −15 and 15 degrees; the shadow turns with the image, but it still falls to the same side.




```
#gobanist-set(
  background: (fill: (
    image: path("img/pgfgo-board.jpg"))),
  stone: (shadow: (on: true, offset: 0.75pt,
    angle: -45deg)),
  players: (
    black: (stone: (image:
      path("img/pgfgo-shell-black.png"))),
    white: (stone: (image: (file: shells,
      rotate: (-15deg, 15deg))))))

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

Kāru is going to use this style in the whole book, and perhaps in other documents. Instead of repeating the `gobanist-set`, they save it as a profile (p. 105): a profile is a dictionary of options, like those of `gobanist-set`, and it is used with the option `profile`. They put it in a file of its own, `wood.typ`, to import it from any document.

```
// wood.typ
#let shells = range(1, 6).map(n => path("img/pgfgo-shell-white-" + str(n) + ".png"))
#let wood = (
  background: (fill: (image: path("img/pgfgo-board.jpg"))),
  stone: (shadow: (on: true, offset: 0.75pt, angle: -45deg)),
  players: (
    black: (stone: (image: path("img/pgfgo-shell-black.png"))),
    white: (stone: (image: (file: shells, rotate: (-15deg, 15deg))))),
  ),
)

// The book:
#import "wood.typ": wood
#gobanist-set(profile: wood)
```

Before setting its own, `profile` brings the style back to the initial one, so the profile looks the same in any document. An almost identical profile, with a soft shadow and without the border of the board, comes with gobanist: `shell` (p. 106); with `profile: "shell"` there is nothing to copy. There is also `yunzi`, with matte stones.

A profile holds for everything gobanist draws, also for the diagrams of an SGF file. This is the game between Lee Sedol and AlphaGo on wood, with a diagram every 50 moves and the numbering restarted in each one.

```
#gobanist-set(profile: wood)

#sgf-diagrams(lee, step: 50, restart: true, layout: (columns: 2, fit: true))
```

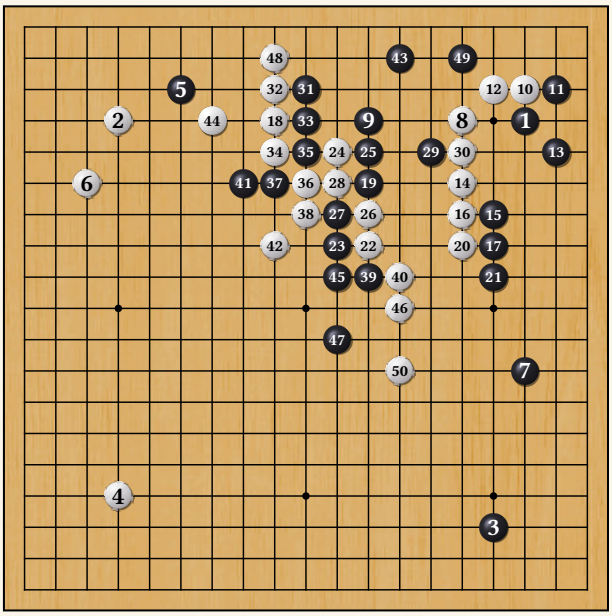


Diagram 1 (1–50)

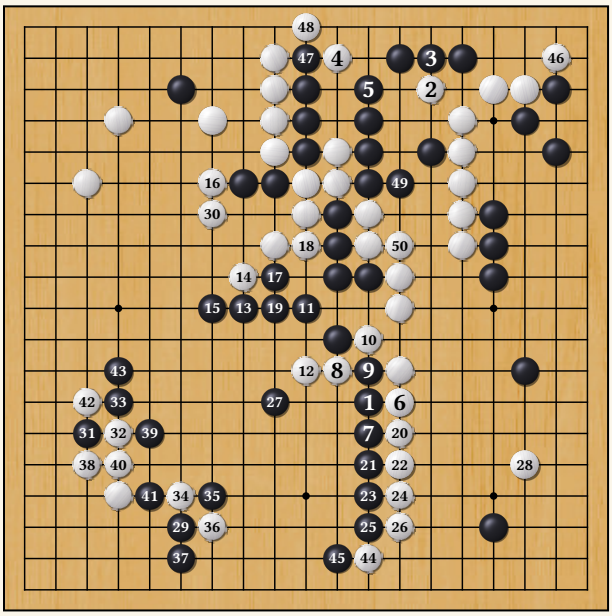


Diagram 2 (51–100)

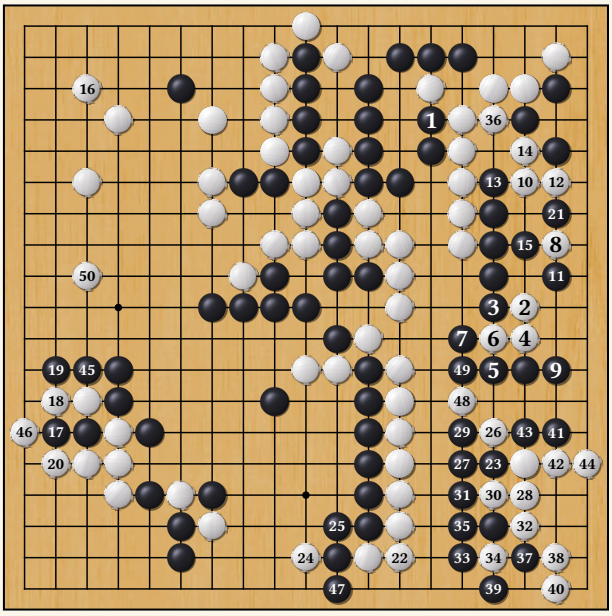


Diagram 3 (101–150)

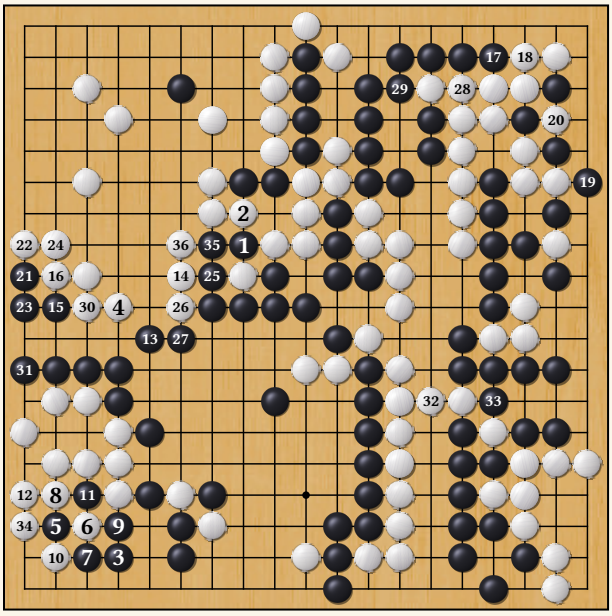


Diagram 4 (151–186)
Result: White wins by resignation

12 Slides

Kāru also teaches with slides. Typst does not make animated PDFs, but a `goban` works on a Touying or Polylux slide as anywhere else, and `sgf` with `to` lets them show the game move by move, one slide per move, with its comment beside it.

```
#import "@preview/touying:0.6.1": *
#import themes.simple: *
#import "@preview/gobanist:0.1.0": *

#show: simple-theme
#let game = sgf-read(path("game.sgf"))

#for k in range(1, 11) [
  == A short game
  #grid(columns: (1fr, 1fr), gutter: 1em,
    goban(captures: "remove", scale: 1.5, sgf(game, to: k)),
    text(size: 0.8em, sgf-comment(game, k)))
]
```

To go over the whole game, on the other hand, the diagrams of Lee Sedol and AlphaGo from the article (p. 16) are enough, one per slide. With `layout: (per-page: 1)`, `sgf-diagrams` puts each diagram on its own page, with its caption (p. 101).

```
#sgf-diagrams(lee, step: 50, goban: (scale: 0.6), layout: (per-page: 1))
```

The slides are another document, but Kāru does not want to copy the boards of the article. Boards are values: they save them in a module, `boards.typ`, and import them in the article and in the slides, where they draw them, show them with `goban-view` or go on from them with `resume`.

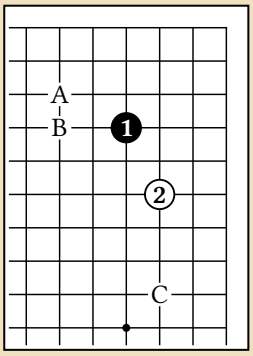
```
// boards.typ
#import "@preview/gobanist:0.1.0": *
#let fight = board(stones(black: "C3, D3, E4, C5", white: "D4, D5, E5, F3"),
  marks(triangle: "D5"), marks(custom: "E3", content: [A]))

// In the slides:
#import "boards.typ": fight
== If Black connects
#goban(fight, partial: auto, moves("E3"))
```

13 A joseki dictionary

For another lesson, Kāru wants to explain the answers to the low approach to the 4-4 point. The known opening sequences in a corner, the *josekis*, are usually saved in an SGF file shaped like a dictionary: a tree with a branch for each possible move, comments on the ones that matter and letters on the moves that can follow. Kāru puts together a small one, `joseki.sgf`.

First, the position: the first two moves, with the letters of the answers. The board is 19×19, but everything happens in a corner; with `partial: auto` (p. 11), the `goban` shows only the part used by what it draws, the letters included.




```
#let josekis = sgf-read(path("joseki.sgf"))

#goban(partial: auto, sgf(josekis, to: 2))
```

For the answers, Kāru chooses the line with `variation`. In a dictionary there are so many branches that it is easier to give the moves of the line, from the first one, than to look for its number: `variation: "Q16, R14"` (p. 84). With `variations`, `sgf-diagrams` also draws, each in its own diagram, the commented variations that depart from that line, and with `from: 3`, only those born at move 3 or later: the answers, and not the 3-3 invasion, which is another move 2. With `partial: auto`, every diagram shows the same part of the board.

```
#sgf-diagrams(josekis, variation: "Q16, R14",
  variations: (on: true, from: 3),
  partial: auto, comments: true,
  layout: (columns: 3, fit: true))
```

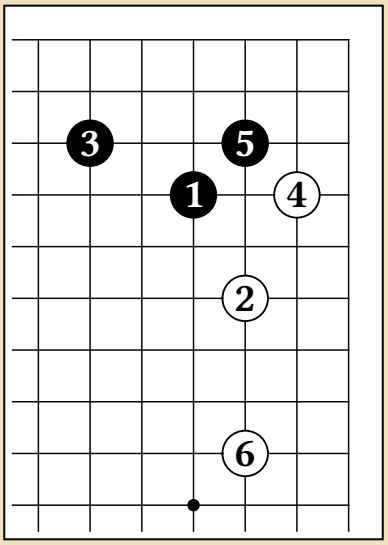


Diagram 1 (1–6)

2: The low approach.
3: A: the knight's move.
4: White slides into the corner.
6: And extends along the side.

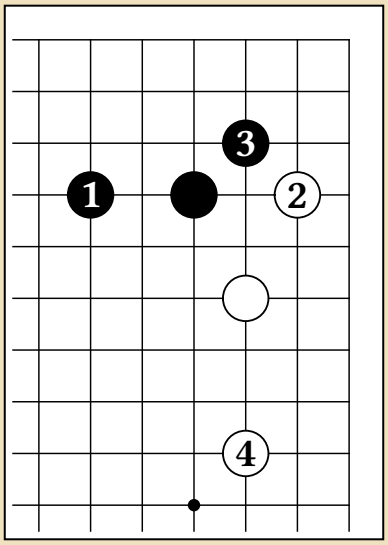


Diagram 1a (variation at 3)

1: B: the one-space jump.

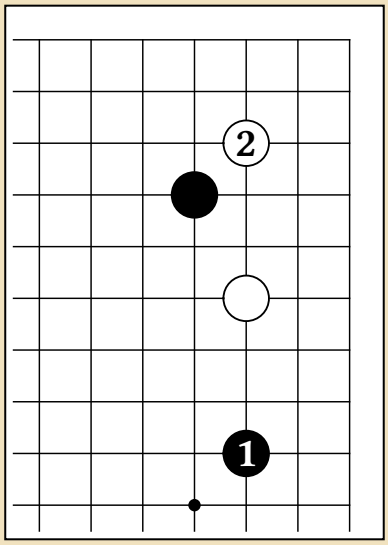


Diagram 1b (variation at 3)

1: C: the low pincer.
2: White invades at the 3-3 point.

The first diagram is the chosen line, to the end. The moves of each variation are numbered from 1, and the caption says at which move it departs. In a large dictionary, each move brings letters on the moves that can follow; with `marks: false`, the diagrams do not draw them.

14 Other languages

A colleague of Kāru's wants to translate the book into French. The texts written by gobanist (the captions, the result, the "N at M" notes and the annotations of the comments) follow the language of the text, `text.lang`, also if it changes inside the document: with `set text(lang: "fr")` they translate themselves (p. 31).

```
#set text(lang: "fr")

#sgf-diagrams(game, step: 5,
  layout: (columns: 2))
#sgf-problem(problem, solutions: false)
```

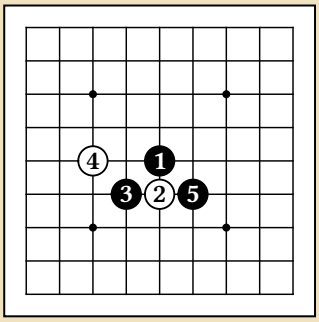


Diagramme 1 (1–5)

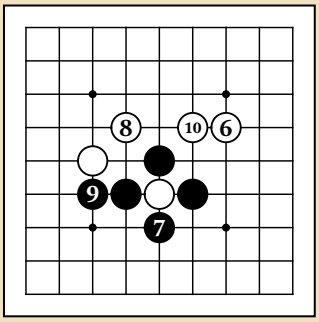
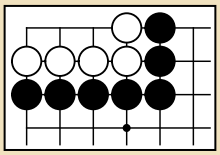


Diagramme 2 (6–10)



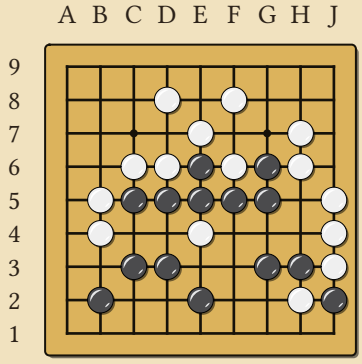
Noir joue
Black to kill.

The comment of the problem, "Black to kill.", is not translated: it is written that way in the file. Each text of the package can also be changed with its own option, in any language: for example, the word in the caption of the diagrams, with `diagrams.title`; or the texts of a language, with `texts`.

```
#gobanist-set(diagrams: (title: "Figure"))
#gobanist-set(texts: (en: (diagram: "Figure")))
```

15 Writing less: the coordinate syntax

In all the tutorials, Kāru wrote the stones one by one. When there are many, gobanist lets you abbreviate the lists of `stones`, `marks` and `prisoners` (p. 33). The most useful forms are the stretches of a row or of a column: `C:G5` is row 5, from C to G, and `J3:5` is column J, from 3 to 5. With them, the first board of the article becomes:

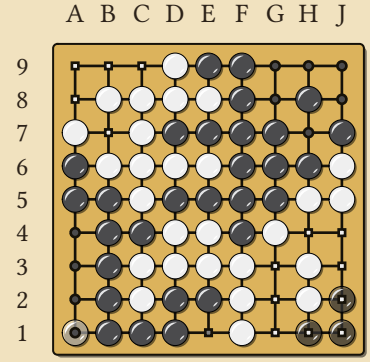



```
#gobanist-set(profile: "pgfgo")

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C:G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, B4, E4, H2, J3:5"))
```

For larger groups, there are paths that turn at right angles. `A6 |- B5` goes from A6 down to row 5 and then right to B5: these are the stones A6, A5 and B5. `E9 -| F4` goes right first and then down. `D5 -- D7` is a

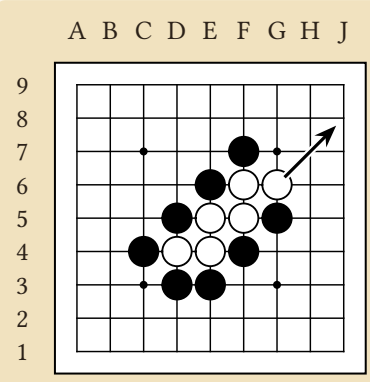
straight line. Paths can be chained, and with them the final position of the territory tutorial (p. 19) fits in two lines:




```
#gobanist-set(profile: "pgfgo")

#goban(label: (at: (top, left)),
  stones(
    black: "A6|-B5|-D1|-E2, C4, E9-|F4,
      D5-|G7--D7, H6, H8, J7",
    white: "E8-|C5, B:E6, A7, B8, D9, J6|-H5,
      H2:3, F1|-C3, C2, D:E4, G4"),
  prisoners(black: "H1-|J2", white: "A1"),
  territory(auto, dame: true))
```

Between two intersections that are neither in the same row nor in the same column, `--` is a slanted line: `D5 -- F7` are D5, E6 and F7. Those stones touch only at the corners; for a connected chain, `-|-` fills each step, first sideways and then up or down: `D4 -|- F6` are D4, E4, E5, F5 and F6 (`|-|` starts with the column). With that, a ladder (*shicho*) is written with its ends:




```
#goban(label: (at: (top, left)),
  stones(white: "D4 -|- F6 -- G6",
    black: "C4, D3, D5 -- F7, E3 -- G5"),
  // White runs. Not for long.
  arrows(("G6", "J8"), shorten: 4pt))
```

There is more: filled rectangles (`A1:C3`) or only their border (`A1 r C3`), whole columns and rows (`D`, `4`), the whole board (`*`) and, after a `;`, stones removed from the list. It is all in the reference, on p. 33.

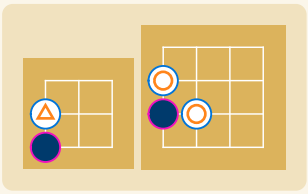
3 Settings

1 The options

The options of gobanist form a tree of dictionaries: each object (the background, the grid, the stones of each player, the diagrams of a game, …) is a branch, and its options are its keys. Those of a board go in the `goban`, named, next to what is drawn; those of the whole document, in `gobanist-set`.

```
gobanist-set(..options)
```

Sets options for everything that follows in the document: the boards, the views, the diagrams and the problems. It takes the same options as `goban`, and also the branches of the games (`sgf`, `diagrams`, `problems`), the layout (`layout`), the views (`view`), the texts (`texts`) and the mark shapes (`shapes`). Each `gobanist-set` is added to the previous ones. Like Typst's `set`, it holds from where it is to the end of the block that contains it.



```
#gobanist-set(
  background: (
    fill: (color: rgb("#dcb35c")),
    line: false),
  grid: (line: (color: white)),
  players: (
    black: (stone: (
      line: (color: fuchsia),
      fill: (color: blue.darken(50%)))),
    white: (
      stone: (line: (color: blue)),
      mark: (line: (color: orange)))))

#goban(size: 3,
  stones(black: "A1"),
  stones(white: "A2", mark: "triangle"))
#goban(size: 4,
  stones(black: "A2"),
  stones(white: "B2, A3", mark: "circle"))
```

The `goban`s that follow inherit these options, and each one can change them with its own: `goban(size: 9, stone: (radius: 4pt), mark: (line: (color: red)), ...)`. The options are resolved when the board is drawn, with the `gobanist-set`s in effect at that place of the document.

The options are merged branch by branch: a dictionary changes only the keys it gives, and the others stay as they were. `gobanist-set(background: (fill: (color: red)))` changes the color of the fill, and the border, the opacity and the image stay the same. A key that does not exist is an error. A length on a branch of lengths (`grid.sep`, `background.rounded-corners`) goes to all of its keys: `grid: (sep: 1em)` is `grid: (sep: (x: 1em, y: 1em))`.

Turning on and off

Many branches turn something on or off: the grid, the background, the border and the fill of the stones, their shadows and their shines, etc. They all have the key `on`, and they all follow the same rule:

- `true` turns the branch on: `shine: true`;
- `false` turns it off: `shine: false`;
- a dictionary changes its options without turning it on: `shine: (angle: 45deg)` changes the angle, but the shine stays as it was. To turn it on and change it at once, `shine: (on: true, angle: 45deg)`.

So `gobanist-set(stone: (shadow: (opacity: 0.3)))` sets the opacity of the shadows of the whole document without turning them on, and each board that wants them turns them on with `stone: (shadow: true)`.

The image branches (`stone.image`, `background.fill.image`, ...) have no `on`: the file turns them on. In them, what is not a dictionary is the file: `stone: (image: path("stone.png"))` is `stone: (image: (file: path("stone.png")))`, and `image: none` removes it.

- **Player shortcuts**

The options of the stones, the prisoners, the moves and the marks belong to each player (p. 104). At the first level, they hold for the three at once:

`stone` stone options

It is `players: (all: (stone: ...))`: the style of the stones of the three players (p. 54).

`prisoner` stone options

It is `players: (all: (prisoner: ...))`: the style of the prisoners of the three players (p. 62).

`move` move options

It is `players: (all: (move: ...))`: the style of the moves of the three players (p. 60).

`mark` mark options

It is `players: (all: (mark: ...))`: the style of the marks of the three players (p. 64).

`players` `neutral`, `black`, `white` and `all`

The options of each player (p. 104).

The other first-level branches are in the chapter of each object: the board (`size`, `partial`, `background`, `grid`, `hoshi`, `label`, ..., p. 40), the arrows and the lines (`arrow`, `line`, p. 69), the game (`captures`, `notes`, `territory`, `score`, p. 72), the games in SGF (`sgf`, `diagrams`, `problems`, p. 84), the layout (`layout`, p. 101), the views (`view`, p. 81), the profiles (`profile`, p. 105), the texts (`texts`, p. 31) and the mark shapes (`shapes`, p. 68).

2 The kernel

gobanist separates the computations from the drawing. The coordinates, the numbering of the moves and the drawing are written in Typst. What playing the game needs is done by a *kernel*: a WebAssembly plugin, `src/kernel.wasm`, compiled from the Rust code in the `kernel/` folder of the repository. The kernel:

- plays the moves and removes the captured stones (`captures`, p. 72);
- finds the territory (`territory(auto)`), builds the regions (`draw: "region"`) and counts the score (p. 73);
- reads SGF files and their encodings (p. 84);
- makes the opacity and the silhouettes of PNG and JPEG images (p. 58).

It plays each board in a single call, when the board has captures or territory. It comes compiled with the package: there is nothing to install.

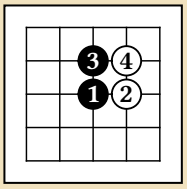
3 Languages

Some texts are written by the package itself: the captions of `sgf-diagrams` and `sgf-problem`, the result of the game, the annotations of the comments and the words of the "N at M" and "N: pass" notes. They are in English (`en`), Spanish (`es`), French (`fr`), German (`de`), Portuguese (`pt`), Italian (`it`), Esperanto (`eo`), Japanese (`ja`), Chinese (`zh`) and Korean (`ko`). Japanese, Chinese and Korean need a font with those characters.

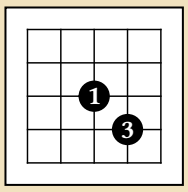
The texts are in the language of the text, `text.lang`, and they follow it if it changes inside the document: `set text(lang: "fr")`. In a language without a translation, they are in English.

`texts` a dictionary of languages

The texts of each language: `texts: (en: (diagram: "Figure"))`. A new language takes from English what it lacks. The texts with variable parts are functions: `n` is the number of a move, `w` the name of the winner and `p` the difference in points.



5 on 1



2: transit

```
#gobanist-set(texts: (
  en: (at: "on"),
  la: (pass: "transit"),
))

#goban(size: 5, notes: true,
  moves("C3, D3, C4, D4"),
  moves("C3", from: 5))
#goban-notes()

#set text(lang: "la")
#goban(notes: true, sgf-read(
  "(;SZ[5];B[cc];W[];B[dd])"))
#goban-notes()
```

The keys of each language, with their value in English and in Spanish, are:

Key	English	Spanish
`tesuji`	Tesuji	Tesuji
`very-good-tesuji`	Very good tesuji	Muy buen tesuji
`bad-move`	Bad move	Mala jugada
`very-bad-move`	Very bad move	Muy mala jugada
`doubtful`	Doubtful move	Jugada dudosa
`interesting`	Interesting move	Jugada interesante
`good-for-black`	Good for Black	Bueno para Negras
`very-good-for-black`	Very good for Black	Muy bueno para Negras
`good-for-white`	Good for White	Bueno para Blancas
`very-good-for-white`	Very good for White	Muy bueno para Blancas
`even`	Even position	Posición pareja
`unclear`	Unclear position	Posición incierta
`diagram`	Diagram	Diagrama
`variation`	Variation	Variante
`problem`	Problem	Problema
`correct`	correct	correcta
`wrong`	wrong	incorrecta
`variation-at`	variation at 26	variante en 26
`black-to-play`	Black to play	Juegan Negras
`white-to-play`	White to play	Juegan Blancas
`at`	at	en
`pass`	pass	pasa
`result`	Result	Resultado
`black`	Black	Negras
`white`	White	Blancas
`win`	*w* wins	Ganan *w*
`win-by-points`	*w* wins by *p* points	Ganan *w* por *p* puntos
`win-by-resignation`	*w* wins by resignation	Ganan *w* por abandono
`win-on-time`	*w* wins on time	Ganan *w* por tiempo
`win-by-forfeit`	*w* wins by forfeit	Ganan *w* por incomparecencia
`draw`	Draw (jigo)	Empate (jigo)
`void`	No result	Sin resultado
`unknown-result`	Unknown result	Resultado desconocido

4 Coordinate syntax

Intersections are written as in Go: a letter for the column, from the left, and a number for the row, from the bottom. `"C3"` is the third column and the third row. The letters are capitals and skip the I, except with `use-i` (p. 41).

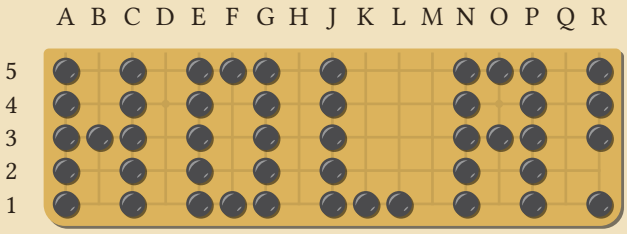
`stones`, `prisoners` and `marks` take, for each player or each shape, a string with one or many intersections, written with a syntax that allows abbreviating:

- several entries separated by commas;
- rows, columns, stretches, rectangles and paths;
- intersections removed from the list, after a `;`.

The same syntax serves for `moves` (without the `;`), for the vertices of `territory` and for `hoshi.at`. The arrows and the lines, and `partial`, take single intersections.

```
#gobanist-set(profile: "pgfgosmooth")

#goban(size: 17, partial: "A1:R5", label: (at: (top, left)),
  stones(black: "*; B, D, H, M, R2, Q, F2:4, K2 r L5, O[1:2, 4]; B3"))
```



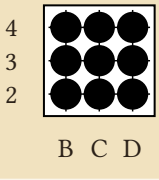
A coordinate that cannot be understood, or that is outside the board, is an error. Spaces separate the elements, but they cannot go inside an intersection: `"B3"` is B3, and `"B 3"` is an error.

1 Single forms

These forms go alone: they are not chained with the operators on p. 35, and they are separated from the others with commas.

The whole board

`*` is every **visible** intersection of the board: with `partial`, those of the visible part.

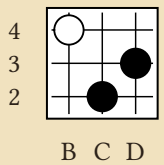



```
#gobanist-set(size: 5, partial: "B2:D4")

#goban(label: (at: (bottom, left)),
  stones(black: "*"))
```

Intersection

A letter and a number, together: `"C2"`.

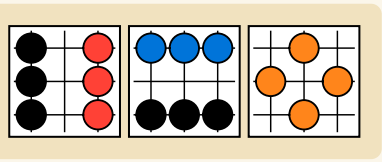


```
#gobanist-set(size: 5, partial: "B2:D4")

#goban(label: (at: (bottom, left)),
  stones(black: "C2, D3", white: "B4"))
```

Whole columns and rows

A letter alone is a whole column: `D` is column D. A number alone is a whole row: `4` is row 4.

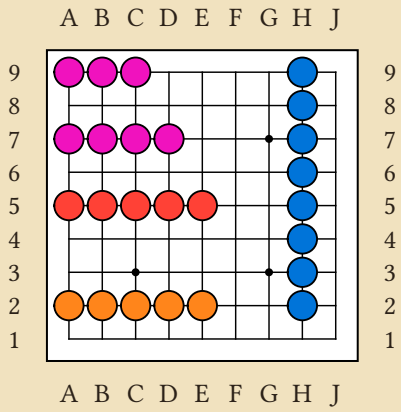


```
#gobanist-set(size: 5, partial: "B2:D4")

#goban(stones(black: "B2, B3, B4"),
  stones("D", fill: (color: red)))
#goban(stones(black: "B2, C2, D2"),
  stones("4", fill: (color: blue)))
// Lines C and 3, without C3.
#goban(stones("C, 3; C3",
  fill: (color: orange)))
```

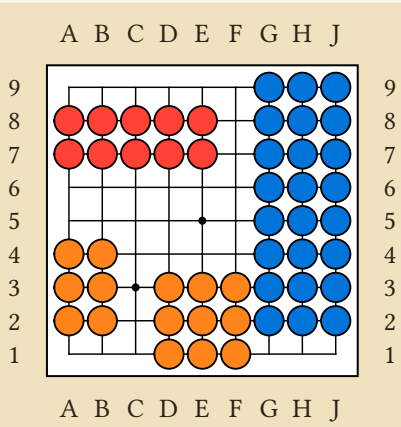
Stretches

A stretch is part of a row or of a column: `C:D2` is row 2, from C to D; `B2:4` is column B, from 2 to 4. There can be a space between the letter and the number: `C:D 2`, `B 2:4`. Stretches go from the first end to the second: `H9:2` goes down from H9 to H2, and `E:A2` goes from E2 to A2.



```
#goban(label: (at: (top, bottom, left, right)),
  stones("A:E5", fill: (color: red)),
  stones("H9:2", fill: (color: blue)),
  stones("E:A2", fill: (color: orange)),
  stones("A:D7, A:C9", fill: (color: fuchsia)))
```

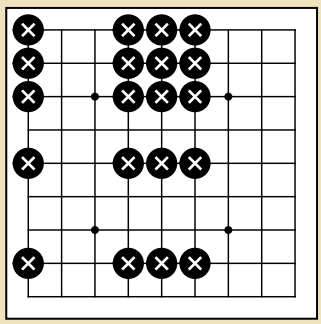
With two ranges, a stretch is a rectangle: `A:E 7:8` are columns A to E of rows 7 and 8. The operators of the next section write it more clearly: `A7:E8`.



```
#goban(label: (at: (top, bottom, left, right)),
  stones("A:E 7:8", fill: (color: red)),
  stones("J:G 9:2", fill: (color: blue)),
  stones("A:B 2:4, F:D 1:3",
    fill: (color: orange)))
```

Products

A letter followed by a list in brackets is distributed over each element of the list: `A[1:3, 5, 7:8]` is `A1:3, A5, A7:8`. The elements need not be numbers: `A[:D 5:7, 1:2]` is `A:D 5:7, A1:2`. In the same way, a list in brackets followed by a number: `[A, C, E:G]12` is `A12, C12, E:G12`. And two lists, one after the other, are combined each with each: `[A, D:F][2, 5, 7:9]` is `A2, A5, A7:9, D:F2, D:F5, D:F 7:9`. Brackets are not nested.



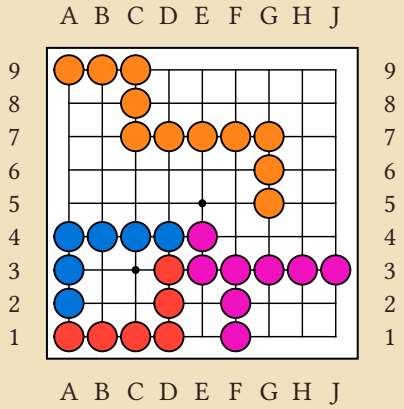
```
#goban(stones(black: "[A, D:F][2, 5, 7:9]"),
  marks(cross: "A2, A5, A7:9, D:F2, D:F5,
    D:F 7:9", player: "black"))
```

2 Paths

The operators join two intersections with a path, and they are chained.

At right angles

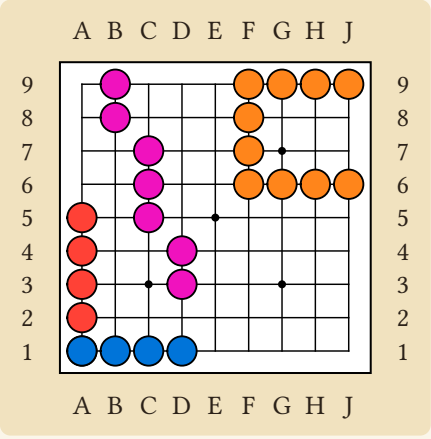
`A1 -| D3` goes from A1 sideways to column D, and then up to D3. `A1 |- D3` goes first along the column and then along the row. Chained, they make stairs or walks with the Manhattan metric.




```
#goban(label: (at: (top, bottom, left, right)),
  stones("A1 -| D3", fill: (color: red)),
  stones("D4 -| A2", fill: (color: blue)),
  stones("A9 -| C7 -| G5", fill: (color: orange)),
  stones("F1 |- J3 -| E4", fill: (color: fuchsia)))
```

In any direction

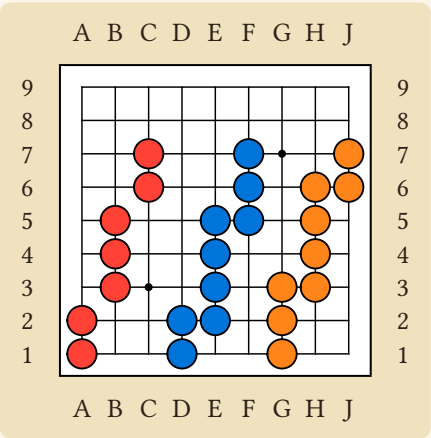
`A1 -- C5` is the line of stones closest to the straight line that joins the two intersections: one stone per row (or per column, if the line advances more than it goes up or down), the one closest to the straight line. If the two intersections are in the same row or column, it is the straight line between them. When the straight line passes exactly between two intersections, the stone goes on the side of the nearer end, and at the exact middle, on the lower column (or row): so `A1 -- C5` and `C5 -- A1` give the same stones.



```
#goban(label: (at: (top, bottom, left, right)),
  stones("A1 -- A5", fill: (color: red)),
  stones("A1 -- D1", fill: (color: blue)),
  stones("B9 -- D3", fill: (color: fuchsia)),
  stones("J9 -| F6 -- J6", fill: (color: orange)))
```

- **Ladders**

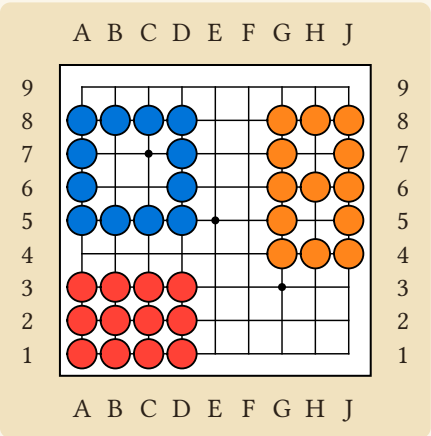
The stones of a slanted line touch only at the corners, so in Go they do not make a group. `D1 -|- F7` and `G1 |-| J7` fill each step with the corner of `-|` (first along the row) or of `|-` (first along the column): the result is a connected ladder. If the two intersections are in the same row or column, they give the same line as `--`. In ladders, as in `-|` and `|-`, the order of the ends matters.



```
#goban(label: (at: (top, bottom, left, right)),
  stones("A1 -- C7", fill: (color: red)),
  stones("D1 -|- F7", fill: (color: blue)),
  stones("G1 |-| J7", fill: (color: orange)))
```

- **Rectangles**

`A1:D3` fills the rectangle with those two opposite corners (it is the way `partial` is written); `A5 r D8` is only its outline. A rectangle of a single column or a single row is a stretch: `A5:A9` is `A5:9`.

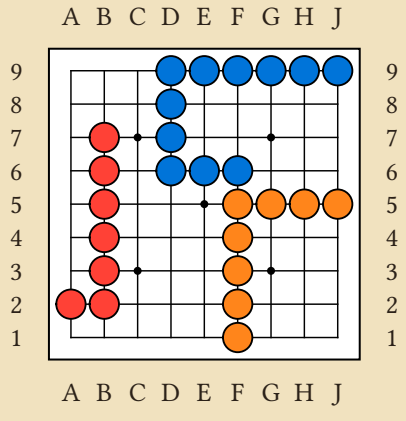


```
#goban(label: (at: (top, bottom, left, right)),
  stones("A1:D3", fill: (color: red)),
  stones("A5 r D8", fill: (color: blue)),
  // It's alive!
  stones("J8 r G4, H6", fill: (color: orange)))
```

- **Chaining**

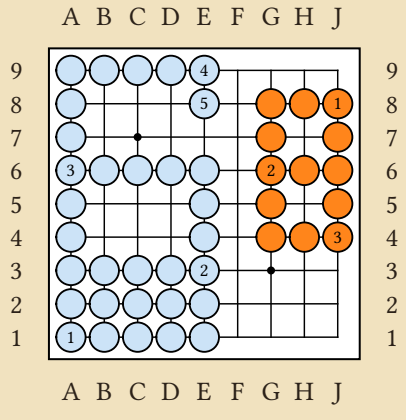
The operators (`-|`, `|-`, `--`, `-|-`, `|-|` and `r`) are chained: `A1 -| C3 -- E3` is `A1 -| C3` followed by `C3 -- E3`, and C3 goes only once. Between two operators there can be an intersection, a stretch (`B5:7`,

`A:B3`) or a rectangle (`A1:C3`). Each operator goes from the last point of what comes before to the first of what comes after: `A2 -| B5:7` is `A2 -| B5` followed by `B5:7`, and `B7 -| A:B3` is `B7 -| A3 -- B3`.



```
#goban(label: (at: (top, bottom, left, right)),
  stones("A2 -| B5:7", fill: (color: red)),
  stones("J9 -| D:F6", fill: (color: blue)),
  stones("F1:3 |- J5", fill: (color: orange)))
```

Rectangles are chained too.

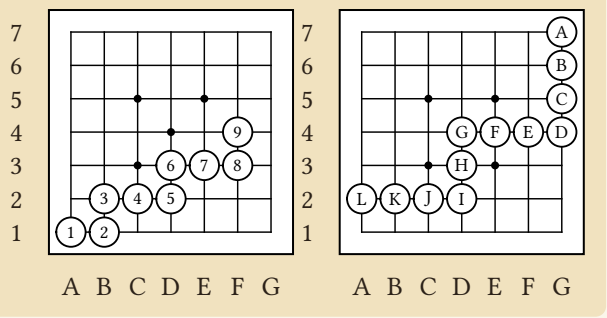


```
#goban(label: (at: (top, bottom, left, right)),
  stones("J8 r G6 r J4", fill: (color: orange)),
  marks(sequence: "J8, G6, J4"),
  stones("A1:E3 r A6 |- E9 -- E8",
    fill: (color: blue.lighten(80%))),
  marks(sequence: "A1, E3, A6, E9, E8"))
```

The single forms (`*`, a whole column or row, a stretch with two ranges) are not chained.

3 The order of the intersections

The intersections come out in the order in which they are written: stretches go from the first end to the second (`N10:7` goes down from N10 to N7), operators follow their path and brackets are distributed in place (`D4, B[3:1]` is `D4, B3:1`). That is the order numbered by `moves` and by the `sequence` mark.



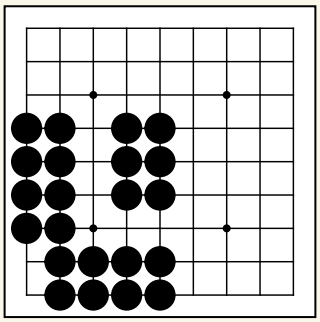
```
#goban(size: 7, label: (at: (bottom, left)),
  stones(white: "A1 -|- F4"),
  marks(sequence: "A1 -|- F4"))
#goban(size: 7, label: (at: (bottom, left)),
  stones(white: "G7:4 -| D2 -- A2"),
  marks(sequence: "G7:4 -| D2 -- A2",
    label: (format: "A")))
```

Some forms have no fixed path: rectangles (`A1:C3`, `A:C 1:3` and `A1 r C3`), whole columns and rows, and `*`. `moves` and `sequence` accept them, and go through them in an order not chosen by the writer.

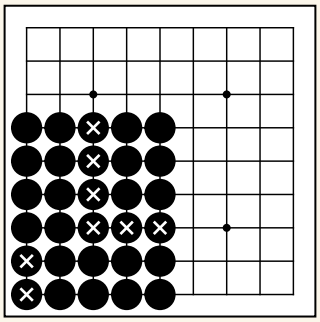
4 Removing intersections

After a `;`, intersections are removed from the list instead of added. Each `;` switches from adding to removing, or from removing to adding, as many times as wanted. `moves` does not accept the `;`: it only adds moves.

Suppose we want to draw this:

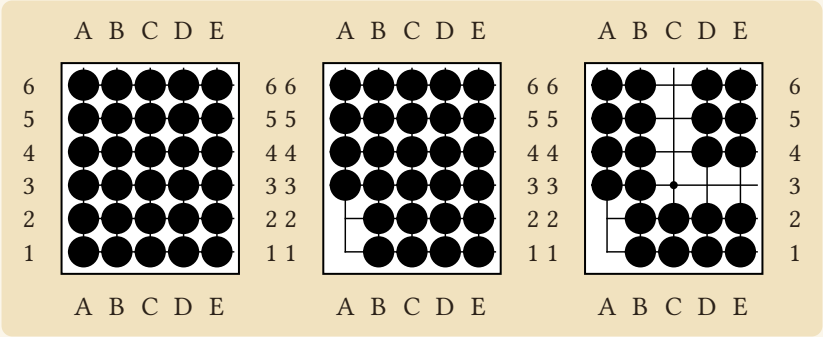


It can be split into parts, or thought of as a full rectangle from which the marked stones are removed:



```
#gobanist-set(label: (at: (top, bottom, left, right)),
  partial: "A1:E6")

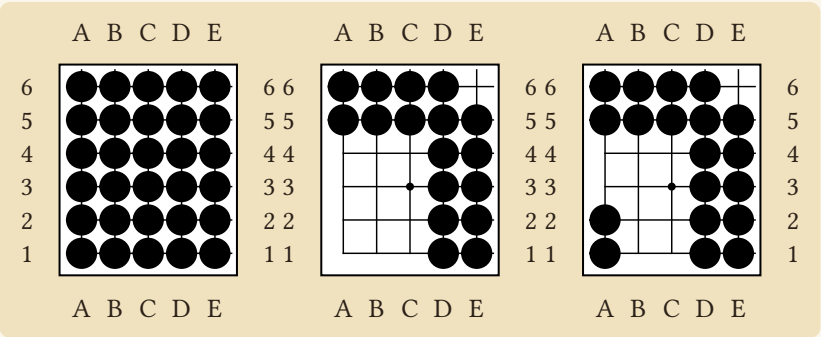
#goban(stones(black: "A1:E6"))
#goban(stones(black: "A1:E6; A1:2"))
#goban(stones(black: "A1:E6; A1:2, C6 |- E3"))
```



First, the rectangle `A1:E6`; then the stretch `A1:2` is removed, and finally the path `C6 |- E3`. The groups alternate with each `;`:

```
#gobanist-set(label: (at: (top, bottom, left, right)),
  partial: "A1:E6")

#goban(stones(black: "A1:E6"))
#goban(stones(black: "A1:E6; A1:C4, E6"))
#goban(stones(black: "A1:E6; A1:C4, E6; A1:2"))
```



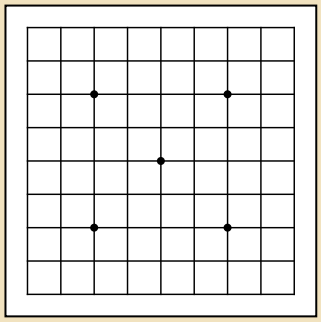
In the last one, the rectangle is added, `A1:C4` and `E6` are removed, and `A1:2` is added again.

5 The board

The board is what `goban` draws: the grid, the background, the coordinates and everything on top.

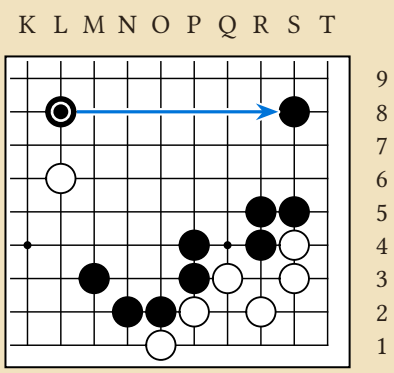
```
goban(..options, ..elements)
```

Draws a board. The elements are what goes on it, in order: stones (`stones`, `prisoners`), moves (`moves`), marks (`marks`), arrows and lines (`arrows`, `lines`), territory (`territory`) and games (`sgf-read`, `sgf`). A board made with `board` can go first: it is that board, with its options and what it has, and the options and the elements of the `goban` are added to it. The options are named, and they are those of `gobanist-set` (p. 30): they hold only for this board. By default, the board is 9×9, with a visible border, a white background and no coordinates.



```
#goban()
```

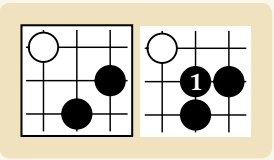
The options of the board (`size`, `use-i`, `partial`, `scale`, ...) go at the first level; those of its parts, in their branches: `grid`, `hoshi`, `background` and `label`.




```
#goban(size: 19, partial: "K1:T9",
  label: (at: (top, right)),
  stones(black: "L8, S8, R5, S5, P4, R4, M3,
    P3, N2, O2",
    white: "L6, S4, S3, Q3, O1, P2, R2"),
  marks(circle: "L8"),
  arrows(("L8", "S8"), color: blue,
    shorten: 6pt))
```

```
board(..options, ..elements)
```

A board as a value: it takes the same as `goban`, but draws nothing. It is resolved when it is drawn, with the global options of the place where it is drawn. Another board made with `board` can go first, as in `goban`: `board(base, moves("C3"))` is `base` with one more move. It is drawn with `goban(board)`, in `goban-layout` (p. 101) and with `goban-view` (p. 81); with `resume`, another board goes on from it (p. 80).

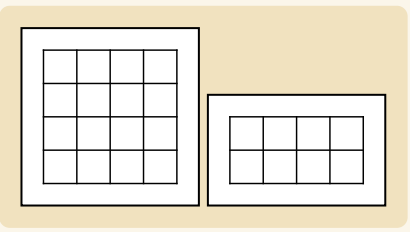


```
#let base = board(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"))

#goban(base)
#goban(base, moves("C3"),
  background: (line: false))
```

`size` a number or `(columns, rows)` initially `9`

The size of the board: the vertical and horizontal lines. A number is a square board; `(5, 3)`, one with 5 columns and 3 rows. Columns go from 2 to 25, and rows, from 2 to 99. With a game (`sgf-read`), the board takes the size of the file (`SZ`), and another `size` is an error.



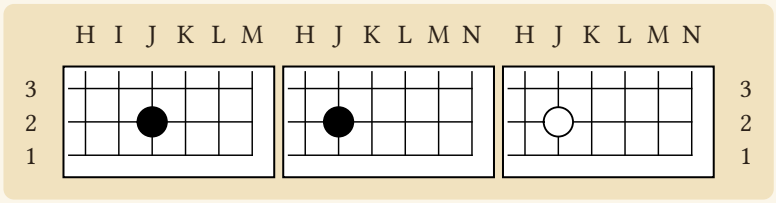
```
#goban(size: 5)
#goban(size: (5, 3))
```

`use-i` `true` or `false` initially `false`

Uses the letter I in the coordinates and in the labels. Without it, as is customary in Go, the columns go from H to J, and an I in the coordinates is the J.

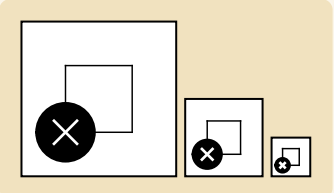
```
#gobanist-set(size: 13, label: (at: top))

#goban(use-i: true, partial: "H1:M3",
  label: (at: (top, left)),
  stones(black: "J2"))
#goban(partial: "H1:N3",
  stones(black: "J2"))
// Without use-i, I is J.
#goban(partial: "H1:N3",
  label: (at: (top, right)),
  stones(white: "I2"))
```



`scale` a number initially `1`

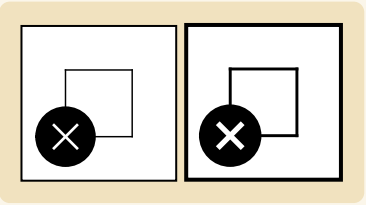
The scale of the board: its measures. The width of the lines does not change, except with `scale-line-width`.



```
#goban(size: 2, scale: 2,
  stones(black: "A1", mark: "cross"))
#goban(size: 2,
  stones(black: "A1", mark: "cross"))
#goban(size: 2, scale: 0.5,
  stones(black: "A1", mark: "cross"))
```

`scale-line-width` `true` or `false` initially `false`

With `true`, `scale` also multiplies the width of the lines: the grid, the border, the stones, the marks, the arrows, the lines and the territory. With `layout.fit`, which chooses the scale to fill the column, the width does not change (p. 101).

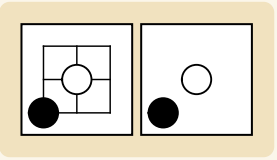


```
#goban(size: 2, scale: 2,
  stones(black: "A1", mark: "cross"))
#goban(size: 2, scale: 2,
  scale-line-width: true,
  stones(black: "A1", mark: "cross"))
```

1 The grid

`grid` `true`, `false` or a dictionary

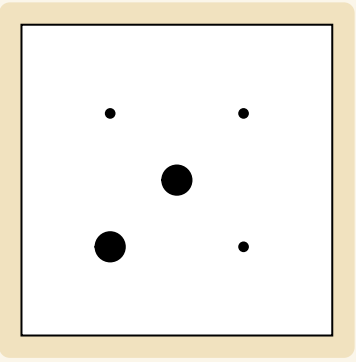
The grid: the lines of the board. With `false`, no grid and no star points.



```
#goban(size: 3,
  stones(black: "A1", white: "B2"))
#goban(size: 3, grid: false,
  stones(black: "A1", white: "B2"))
```

`grid.line` `true`, `false` or a dictionary

The lines. With `false`, no lines, but with the star points (unlike `grid: false`).



```
#goban(grid: (line: false),
  hoshi: (radius: 2pt),
  stones(black: "C3, E5"))
```

`grid.line.color` a color — initially `black`

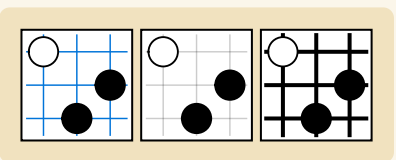
The color of the lines.

`grid.line.opacity` a number from 0 to 1 — initially `1`

The opacity of the lines.

`grid.line.width` a length — initially `0.55pt`

The width of the lines.

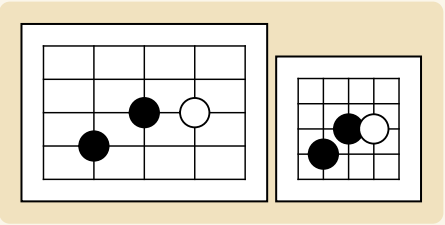


```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, grid: (line: (color: blue)))
#goban(b, grid: (line: (opacity: 0.2)))
#goban(b, grid: (line: (width: 1.5pt)))
```

`grid.sep` a length or `(x: ..., y: ...)` — initially `(x: 12.5pt, y: 12.5pt)`

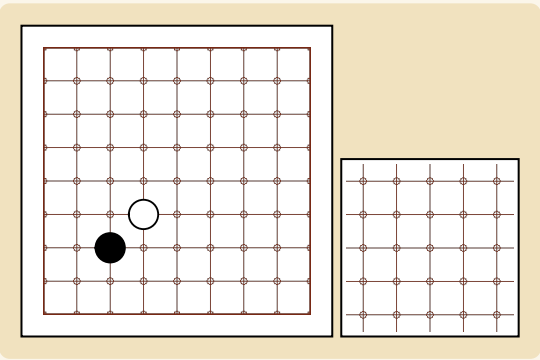
The distance between the lines: `x`, between the vertical ones, and `y`, between the horizontal ones. A single length goes to both.



```
#goban(size: 5, grid: (sep: (x: 1.8em)),
  stones(black: "B2, C3", white: "D3"))
#goban(size: 5, grid: (sep: 0.9em),
  stones(black: "B2, C3", white: "D3"))
```

`grid.image` a file or a dictionary

An image instead of the lines, with the opacity of `grid.line.opacity`. With `partial`, the corresponding part is seen. The star points are still drawn (p. 44). What is not a dictionary is the file; `none` removes it.

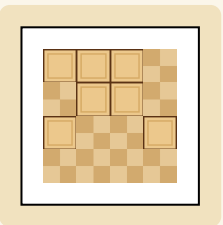


```
#let grid-image = path("img/pgfgo-grid.png")

#goban(grid: (image: grid-image),
  hoshi: false,
  stones(black: "C3", white: "D4"))
#goban(grid: (image: grid-image),
  hoshi: false, partial: "C3:G7")
```

`grid.image.file` a file or an array (initially `none`)

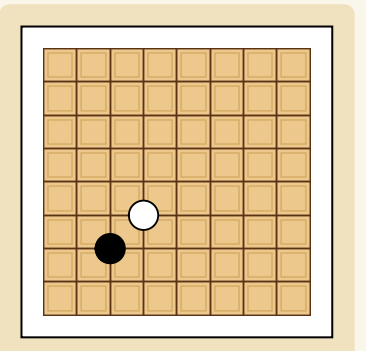
The file: a `path` (relative to the document that names it) or its `bytes`. With several files and `fit: "tile"`, each tile uses one, chosen by its place in the mesh: always the same one in each place, in every diagram. With `"stretch"`, the first one is used.



```
#goban(size: 5, hoshi: false,
  grid: (image: (fit: "tile", file: (
    path("img/pgfgo-cell.png"),
    path("img/pgfgo-tile.png")))))
```

`grid.image.fit` `"stretch"` or `"tile"` (initially `"stretch"`)

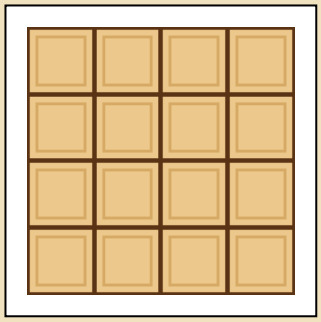
With `"stretch"`, the image is stretched from A1 to the opposite corner of the full board: the outer lines must be on the edges of the image. With `"tile"`, it is drawn once in each cell (between four intersections), stretched to the size of the cell: with the image of one square the whole board is built.



```
#goban(hoshi: false, grid: (image: (
    file: path("img/pgfgo-cell.png"),
    fit: "tile")),
  stones(black: "C3", white: "D4"))
```

`grid.image.tiles` `auto`, a number or `(columns, rows)` initially `auto`

With `fit: "tile"`, the mesh in which the image is repeated, over the area that goes from A1 to the opposite corner of the full board; each image is stretched to the size of its cell. A number holds for both directions. With `auto`, one image per cell of the board.



```
#goban(hoshi: false, grid: (image: (
  file: path("img/pgfgo-cell.png"),
  fit: "tile", tiles: 4)))
```

`grid.image.rotate` an angle, `"random"` or `(min, max)` initially `0deg`

Rotates the image, like `background.fill.image.rotate`: in quarter turns, and with `"random"` or a range, each tile with its own.

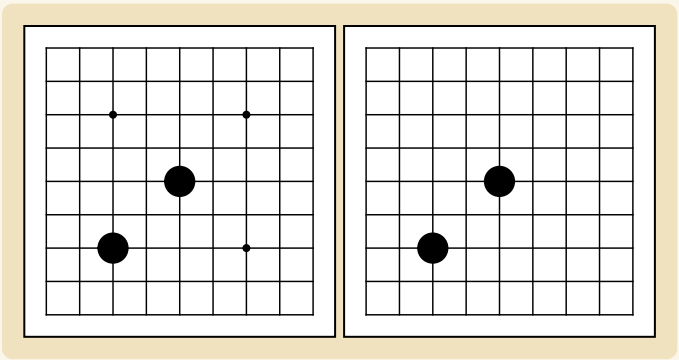
2 Star points

The star points (*hoshi*) are drawn on the grid, with its color. By default they are the usual ones: on each side, the fourth line from the border (the third on boards of fewer than 13 lines); the center if both sides are odd; and, from 15 lines on, the middle of the sides. A 19×19 board has nine, and a 13×13 or 9×9 one, five. Boards of fewer than 7 lines have none. Stones cover them.

`hoshi` `true`, `false` or a dictionary

The star points.

```
#goban(stones(black: "C3, E5"))
#goban(hoshi: false, stones(black: "C3, E5"))
```



`hoshi.radius` a length initially `1.5pt`

The radius of the points.

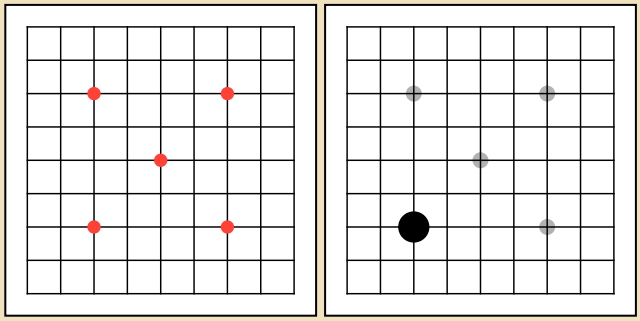
`hoshi.color` `auto` or a color initially `auto`

The color of the points. With `auto`, that of the grid.

`hoshi.opacity` `auto` or a number from 0 to 1 initially `auto`

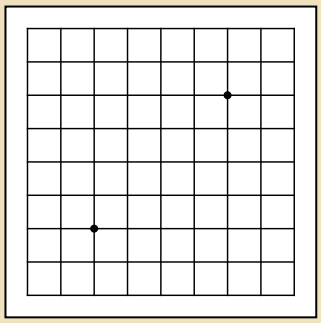
The opacity of the points (also that of their image). With `auto`, that of the grid.

```
#goban(hoshi: (radius: 2.5pt, color: red))
#goban(hoshi: (radius: 3pt, opacity: 0.3),
  stones(black: "C3"))
```



`hoshi.at` `auto` or coordinates — initially `auto`

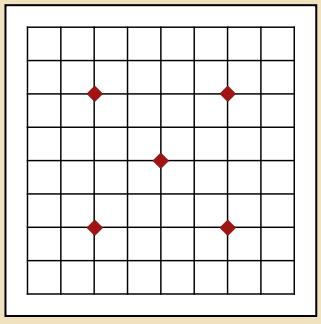
The points, with the syntax of `stones` (p. 33). With `auto`, the usual ones.



```
#goban(hoshi: (at: "C3, G7"))
```

`hoshi.image` a file or a dictionary

An image on each point, instead of the circle. What is not a dictionary is the file; `none` removes it.



```
#goban(hoshi: (radius: 3pt,
  image: path("img/pgfgo-hoshi.png")))
```

`hoshi.image.file` a file — initially `none`

The file of the image.

`hoshi.image.width` `auto` or a length — initially `auto`

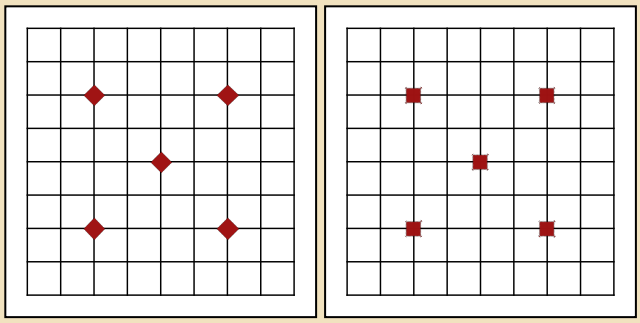
The width of the image; the height keeps the proportion. With `auto`, the diameter of the points.

`hoshi.image.rotate` an angle, `"random"` or `(min, max)` — initially `0deg`

Rotates the image, counterclockwise. With `"random"` or a range, each point with its angle, always the same one (p. 58).

```
#let hoshi = path("img/pgfgo-hoshi.png")

#goban(hoshi: (image: (file: hoshi,
  width: 8pt)))
#goban(hoshi: (image: (file: hoshi,
  width: 8pt, rotate: 45deg)))
```

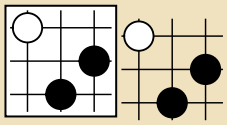


3 The background

The background of the board is the rectangle under the grid, with its fill and its border. By default, the fill is white and the border, black.

`background` `true`, `false` or a dictionary

The background: the fill and the border. With `false`, no background.



```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b)
#goban(b, background: false)
```

`background.fill` `true`, `false` or a dictionary

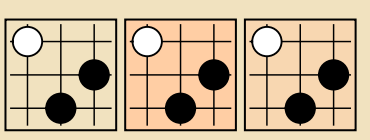
The fill. With `false`, the background is transparent and only the border remains.

`background.fill.color` a color — initially `white`

The color of the fill.

`background.fill.opacity` a number from 0 to 1 — initially `1`

The opacity of the fill, also that of its image.

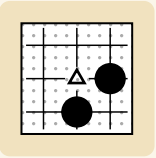


```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, background: (fill: false))
#goban(b, background: (fill: (
  color: orange.lighten(60%))))
#goban(b, background: (fill: (
  color: orange.lighten(60%), opacity: 0.5)))
```

`background.fill.pattern` `none` or a `tiling` — initially `none`

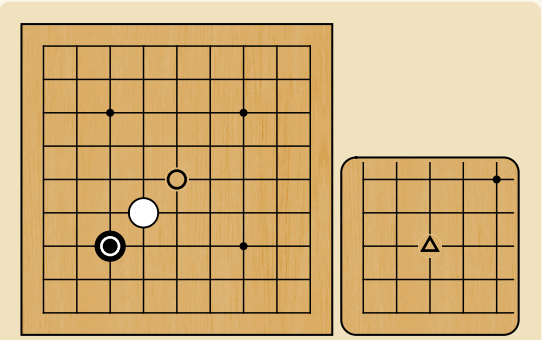
A pattern over the color and the image of the background: a Typst `tiling`. The marks on empty points and the `dim` mark, which cover the grid with the background, repeat the pattern.



```
#goban(size: 5, partial: "B2:D4",
  background: (fill: (pattern: tiling(
    size: (4pt, 4pt),
    circle(radius: 0.6pt, fill: gray)))),
  stones(black: "C2, D3"),
  marks(triangle: "C3"))
```

`background.fill.image` a file or a dictionary

A background image (for example, the wood of a board), over the fill color and with its same opacity. The image is placed according to the full board: with `partial`, the part that corresponds to it is seen, cut by the border and its rounded corners. The marks on empty points, which cover the grid with the background, show the part of the image underneath, and so does the `dim` mark. What is not a dictionary is the file; `none` removes it.

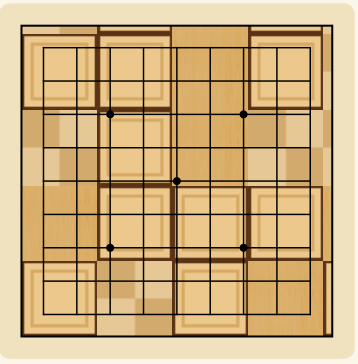


```
#let wood = path("img/pgfgo-board.jpg")

#goban(background: (fill: (image: wood)),
  stones(black: "C3", white: "D4"),
  marks(circle: "E5, C3"))
#goban(partial: "A1:E5", background: (
    fill: (image: wood),
    rounded-corners: 6pt),
  marks(triangle: "C3"))
```

`background.fill.image.file` a file or an array initially `none`

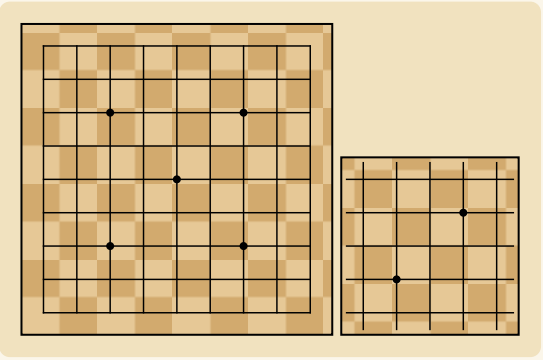
The file of the image. With several files and `fit: "tile"`, each tile uses one, chosen by its place: always the same one in each place, also with `partial`. The tiles have the size of the first image (or that of their cell, with `tiles`). With `"stretch"` and `"cover"`, the first one is used.



```
#goban(background: (fill: (image: (
  file: (path("img/pgfgo-tile.png"),
    path("img/pgfgo-cell.png"),
    path("img/pgfgo-board.jpg")),
  fit: "tile", width: 1cm))))
```

`background.fill.image.fit` `"cover"`, `"stretch"` or `"tile"` initially `"cover"`

How the image fills the background of the full board. `"stretch"` stretches it to fill the whole background, without keeping the proportion. `"cover"` enlarges or shrinks it, keeping the proportion, until it covers the background, centered: what is left over is not seen. `"tile"` repeats it as tiles, from a corner, a side or the center of the full board (`anchor`).

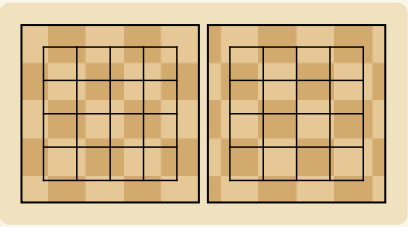


```
#let tile = (file: path("img/pgfgo-tile.png"),
  fit: "tile", width: 1cm)

#goban(background: (fill: (image: tile)))
#goban(partial: "D4:H8",
  background: (fill: (image: tile)))
```

`background.fill.image.anchor` an alignment — initially `bottom + left`

With `fit: "tile"`, where the tiles start from, on the background of the full board. With a corner (`bottom + left`, `top + right`, …), a whole tile lies in that corner. With a side (`top`, `bottom`, `left` or `right`), the first tile lies against that side, centered along it. With `center`, a tile lies at the center of the board.



```
#let tile = (file: path("img/pgfgo-tile.png"),
  fit: "tile", width: 1cm)

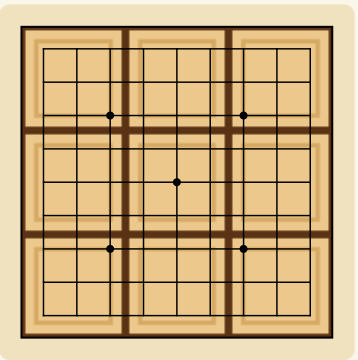
#goban(size: 5, background: (fill: (
  image: tile + (anchor: top + right))))
#goban(size: 5, background: (fill: (
  image: tile + (anchor: top))))
```

`background.fill.image.width` `auto` or a length — initially `auto`

With `fit: "tile"`, the width of each tile; the height keeps the proportion. With `auto`, the natural width of the image.

`background.fill.image.tiles` `auto`, a number or `(columns, rows)` — initially `auto`

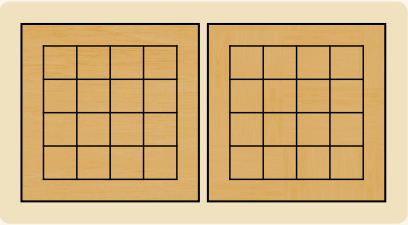
With `fit: "tile"`, instead of tiles of width `width` from `anchor`, divides the background of the full board into that mesh and puts the image in each cell, stretched to its size. A number holds for both directions.



```
#goban(background: (fill: (image: (
  file: path("img/pgfgo-cell.png"),
  fit: "tile", tiles: (3, 3)))))
```

`background.fill.image.rotate` an angle, `"random"` or `(min, max)` — initially `0deg`

Rotates the image in quarter turns: the angle is rounded to 0, 90, 180 or 270 degrees, so that the rotated image still fills its place without leaving gaps. With 90 or 270, the width and the height are swapped. With `"random"` or a range, each tile has its own rotation, always the same one.



```
#let wood = path("img/pgfgo-board.jpg")

#goban(size: 5, background: (fill: (image: (
  file: wood, rotate: 90deg))))
#goban(size: 5, background: (fill: (image: (
  file: wood, fit: "tile", width: 1cm,
  rotate: "random"))))
```

`background.line` `true`, `false` or a dictionary

The border. With `false`, no border.

`background.line.color` a color — initially `black`

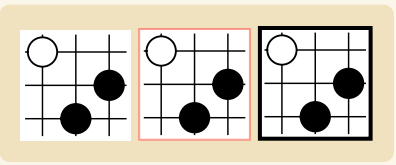
The color of the border.

`background.line.opacity` a number from 0 to 1 — initially `1`

The opacity of the border.

`background.line.width` a length — initially `0.75pt`

The width of the border.



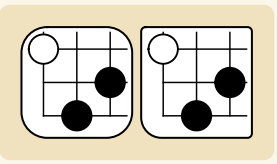
```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, background: (line: false))
#goban(b, background: (line: (
  color: red, opacity: 0.5)))
#goban(b, background: (line: (
  width: 1.5pt)))
```

`background.rounded-corners` a length or `(outer: ..., inner: ...)`

initially `(outer: 0pt, inner: 0pt)`

The radius of the corners of the background: `outer`, that of the corners of the board, and `inner`, that of the ones seen on a partial board. A single length goes to both.

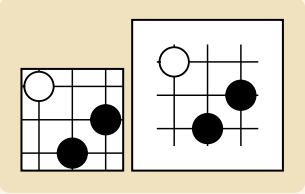


```
#gobanist-set(size: 5, partial: "A1:C3")
#let b = board(stones(black: "B1, C2", white: "A3"))

#goban(b, background: (
  rounded-corners: 10pt))
#goban(b, background: (
  rounded-corners: (outer: 8pt, inner: 2pt)))
```

`background.sep` a length — initially `8.25pt`

The distance between the grid and the border.

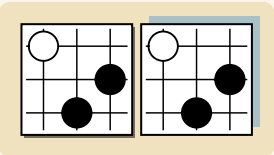


```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, background: (sep: 0.625em))
#goban(b, background: (sep: 1.5em))
```

`background.shadow` `true`, `false` or a dictionary

A shadow under the board.



```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, background: (shadow: true))
#goban(b, background: (shadow: (
  on: true, color: blue, opacity: 0.3,
  angle: 45deg, offset: 4pt)))
```

`background.shadow.angle` an angle — initially `-45deg`

Where the shadow falls: with `0deg`, to the right; with `-45deg`, down and to the right.

`background.shadow.offset` a length — initially `1.5pt`

The distance between the board and its shadow.

`background.shadow.opacity` a number from 0 to 1 — initially `0.5`

The opacity of the shadow.

`background.shadow.color` a color — initially `black`

The color of the shadow.

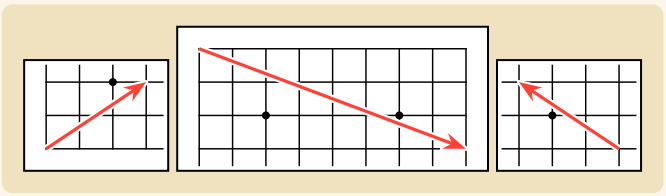
4 Partial board

Often the whole board is not needed, only part of it. `partial` defines it with two opposite corners of a rectangle, in any order: `"A1:D3"` is the same as `"D3:A1"`, and as `"A3:D1"`.

`partial` `none`, `"A1:D3"` or `auto` — initially `none`

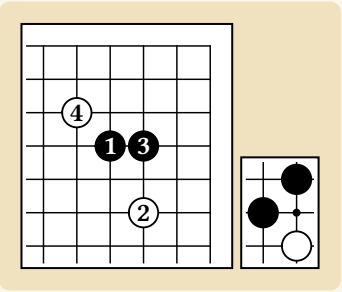
Shows only part of the board: the rectangle with those two intersections as opposite corners. What falls outside is not drawn. With `auto`, the board chooses the part: the one used by the stones, moves, marks, arrows, lines and territories that are drawn, with `partial-margin` more lines. With `none`, the whole board.

```
#goban(partial: "A1:D3",
  arrows(("A1", "D3"), color: red))
#goban(partial: "A9:J6",
  arrows(("A9", "J6"), color: red))
#goban(partial: "E2:B4",
  arrows(("E2", "B4"), color: red))
```



`partial-margin` a number — initially `1`

The lines that `partial: auto` adds around what is drawn. A side two lines or less from the border reaches the border. With nothing drawn, the whole board is seen.



```
#goban(size: 19, partial: auto,
  moves("Q16, R14, R16, P17"))
#goban(size: 19, partial: auto,
  partial-margin: 0,
  stones(black: "J10, K11", white: "K9"))
```

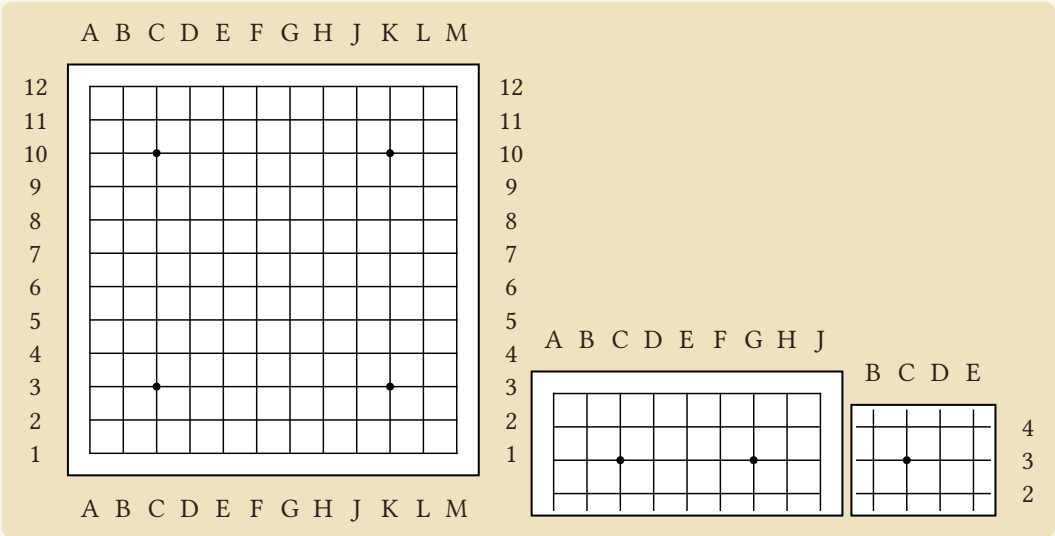
5 The border coordinates

The labels are the coordinates written on the sides of the board.

`label.at` an alignment or an array — initially `()`

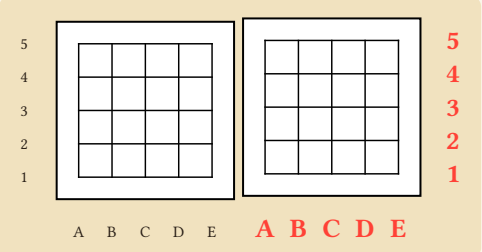
On which sides the labels go: `top`, `bottom`, `left` and `right`, or an array of them. The columns go at the top and the bottom; the rows, on the sides. With `()`, no labels.

```
#goban(size: 12,
  label: (at: (top, bottom, left, right)))
#goban(partial: "A6:J9", label: (at: top))
#goban(partial: "B2:E4",
  label: (at: (top, right)))
```



`label.text` a dictionary — initially `(size: 10pt)`

The `text` options of the labels: the font, the size, the color.

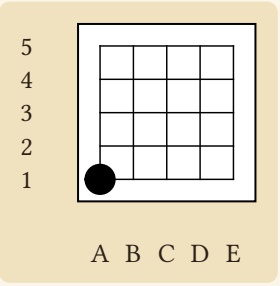



```
#gobanist-set(size: 5)

#goban(label: (at: (bottom, left),
  text: (size: 6pt)))
#goban(label: (at: (bottom, right),
  text: (weight: "bold", fill: red)))
```

`label.sep` a length — initially `0.35cm`

The distance between the labels and the border of the board.




```
#goban(size: 5, label: (at: (bottom, left),
  sep: 0.6cm), stones(black: "A1"))
```

`label.columns` `"A"` or a `numbering` pattern — initially `"A"`

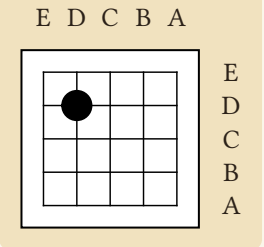
How the coordinates of the columns are written (at the top and the bottom): with `"A"`, the letters of Go (without the I, except with `use-i`); with another Typst `numbering` pattern, such as `"1"` or `"一"`, numbers.

`label.rows` `"A"` or a `numbering` pattern — initially `"1"`

The same for the rows (on the sides). The kanji (`"一"`) of the rows are aligned on the side of the board, so that those with two characters do not run into it.

`label.columns-direction` `ltr` or `rtl` — initially `ltr`

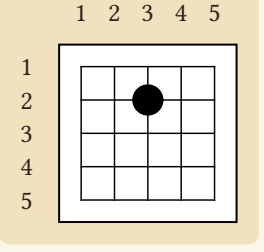
In which direction the columns are counted: from the left or from the right.




```
#goban(size: 5, label: (at: (top, right),
    columns-direction: rtl, rows: "A"),
  stones(black: "B4"))
```

`label.rows-direction` `btt` or `ttb` — initially `btt`

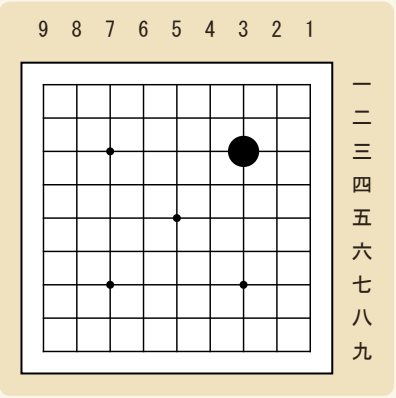
In which direction the rows are counted: from the bottom or from the top. These four options change only what is written on the border: the coordinates of the elements (`"C3"`) stay the same. On a partial board, they are counted from the border of the full board.




```
#goban(size: 5, label: (at: (top, left),
    columns: "1", rows-direction: ttb),
  stones(black: "C4"))
```

`label.style` `"western"`, `"japanese"` or `"chinese"`

Sets the four previous options according to the custom of each place. `"western"`: columns with letters from the left and rows with numbers from the bottom, as at the start. `"japanese"`: columns with numbers from the right and rows with kanji from the top, as in Japanese books: the top right corner is 1-一. `"chinese"`: columns with numbers from the left and rows with kanji from the top. The kanji need a font that has them.




```
#goban(label: (at: (top, right),
    style: "japanese",
    text: (font: "MS Gothic", size: 8pt)),
  stones(black: "G7"))
```

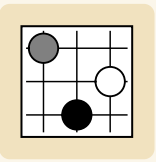
6 Stones, moves and prisoners

Stones are placed with `stones`; moves, with `moves`, and prisoners, with `prisoners`. Their style is that of their player (p. 104), and each element can change it with its options.

1 Stones

```
stones(coordinates, black: ..., white: ..., neutral: ..., ..options)
```

Places stones on the intersections, written with the coordinate syntax (p. 33). The player is the key: `stones(black: "C2, D3", white: "B4")`. Without a key, the stones are neutral: `stones("C3")`. The options change their style (those of the `stone` branch, below) and put a mark on them (`mark`).



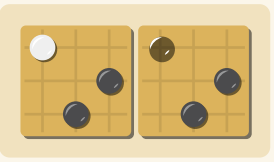
```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2", white: "D3"),
  stones("B4"))
```

Stones, marks and labels that fall outside the visible part of a partial board are not drawn. A coordinate that does not exist on the board (`"E9"` on a 5×5 board) is an error. A stone on a point where there already is one covers it.

The style of the stones of the three players is changed with `gobanist-set(stone: ...)`, which is `gobanist-set(players: (all: (stone: ...)))`; that of one, with `gobanist-set(players: (black: (stone: ...)))`. The options of the `stone` branch are:

`stone.fill` `true`, `false` or a dictionary

The fill of the stone. With `false`, no fill: the rest remains (the border, the shadow, the shine).

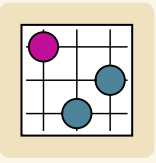


```
#gobanist-set(profile: "pgfgosmooth", size: 5,
  partial: "B2:D4")

#goban(stones(black: "C2, D3", white: "B4"))
#goban(stones(black: "C2, D3"),
  stones(white: "B4", fill: false))
```

`stone.fill.color` a color or `none`

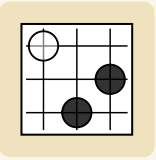
The color of the fill: black for Black, white for White and gray for neutral stones. With `none`, no color: the pattern remains, if there is one.



```
#goban(size: 5, partial: "B2:D4",
  stones("C2, D3", fill: (color: aqua.darken(40%))),
  stones("B4", fill: (color: fuchsia.darken(20%))))
```

`stone.fill.opacity` a number from 0 to 1 — initially `1`

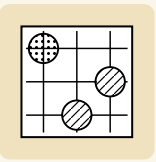
The opacity of the fill.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", fill: (opacity: 0.8)),
  stones(white: "B4", fill: (opacity: 0.5)))
```

`stone.fill.pattern` `none` or a `tiling` — initially `none`

A pattern over the color of the fill, with its same opacity: a Typst `tiling`. With `fill.color: none`, the stone has only the pattern. The `t-circle` and `t-square` marks of the player use the same fill. The pattern does not change size with `scale`.

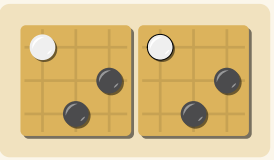


```
#let stripes = tiling(size: (3pt, 3pt),
  line(start: (0pt, 3pt), end: (3pt, 0pt),
    stroke: 0.4pt))

#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3",
    fill: (color: white, pattern: stripes)),
  stones(white: "B4", fill: (color: none,
    pattern: tiling(size: (2pt, 2pt),
      circle(radius: 0.5pt, fill: black)))))
```

`stone.line` `true`, `false` or a dictionary

The border of the stone.



```
#gobanist-set(profile: "pgfgosmooth", size: 5,
  partial: "B2:D4")

#goban(stones(black: "C2, D3", white: "B4"))
#goban(stones(black: "C2, D3"),
  stones(white: "B4", line: true))
```

`stone.line.color` a color — initially `black`

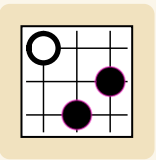
The color of the border.

`stone.line.opacity` a number from 0 to 1 — initially `1`

The opacity of the border.

`stone.line.width` a length — initially `0.7pt`

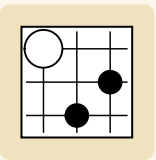
The width of the border.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", line: (
    color: fuchsia, opacity: 0.5)),
  stones(white: "B4", line: (width: 2pt)))
```

`stone.radius` a length — initially `5.5pt`

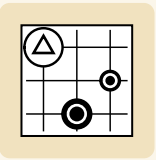
The radius of the stone.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", radius: 0.15cm),
  stones(white: "B4", radius: 0.25cm))
```

`stone.scale` a number — initially `1`

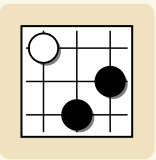
Scales the whole stone: the radius, the border and the marks.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2", mark: "circle"),
  stones(black: "D3", scale: 0.7,
    mark: "circle"),
  stones(white: "B4", scale: 1.3,
    mark: "triangle"))
```

`stone.shadow` `true`, `false` or a dictionary

The shadow of the stone, drawn underneath. It has the silhouette of the stone: the circle, or the shape of the image if the stone is an image (`image`). With `shadow.image`, the shadow is an image of its own. In both cases it is shifted `offset` towards `angle` and enlarged `scale` times.



```
#goban(size: 5, partial: "B2:D4",
  stone: (shadow: (on: true, angle: -45deg,
    offset: 1.5pt)),
  stones(black: "C2, D3", white: "B4"))
```

`stone.shadow.angle` an angle — initially `0deg`

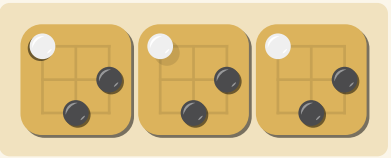
Where the shadow falls: with `0deg`, to the right; with `-45deg`, down and to the right. It does not rotate the shape of the shadow.

`stone.shadow.offset` a length — initially `1pt`

How far the shadow is shifted from the center of the stone, towards `angle`.

`stone.shadow.opacity` a number from 0 to 1 — initially `0.5`

The opacity of the shadow (also that of the shadow with an image).



```
#gobanist-set(profile: "pgfgosmooth", size: 3)
#let b = board(stones(black: "B1, C2", white: "A3"))

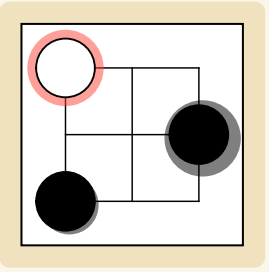
#goban(b, players: (white: (stone: (
  shadow: (angle: 225deg)))))
#goban(b, players: (white: (stone: (
  shadow: (offset: 0.125cm, opacity: 0.1)))))
#goban(b, players: (white: (stone: (
  shadow: false))))
```

`stone.shadow.color` a color — initially `black`

The color of the shadow. A shadow with an image has its own colors, except with `image.silhouette`.

`stone.shadow.scale` a number — initially `1`

Enlarges (or shrinks) the shadow relative to the stone. A shadow larger than the stone makes it look taller. It also enlarges the shadow with an image.

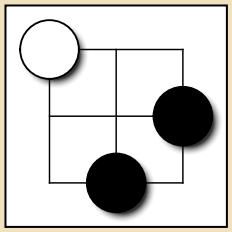


```
#goban(size: 3, scale: 2,
  stones(black: "A1", shadow: (on: true,
    angle: -45deg)),
  stones(black: "C2", shadow: (on: true,
    angle: -45deg, scale: 1.3)),
  stones(white: "A3", shadow: (on: true,
    offset: 0pt, scale: 1.3, color: red)))
```

`stone.shadow.image` a file or a dictionary

A shadow of your own: an image, centered on the stone and drawn as it is instead of the silhouette, for example a soft shadow. It is shifted and enlarged like the silhouette (`angle`, `offset`, `scale`) and uses

`opacity`, but not `color`. It does not rotate with the image of the stone, so it always falls to the same side. What is not a dictionary is the file; `none` removes it.



```
#goban(size: 3, scale: 2,
  stone: (shadow: (on: true, angle: -45deg,
    offset: 1.5pt, opacity: 0.8,
    image: (file: path("img/pgfgo-shadow.png"),
      width: 13pt))),
  stones(black: "B1, C2", white: "A3"))
```

`stone.shadow.image.file` a file — initially `none`

The file of the image.

`stone.shadow.image.width` `auto` or a length — initially `auto`

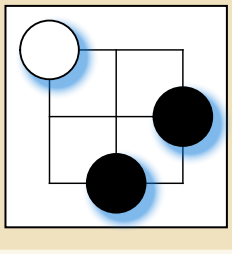
The width of the image; the height keeps the proportion. With `auto`, the diameter of the stone.

`stone.shadow.image.rotate` an angle — initially `0deg`

Rotates the image of the shadow, the same on every stone.

`stone.shadow.image.silhouette` `true` or `false` — initially `false`

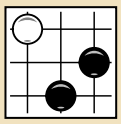
Uses only the shape of the image (its transparency), painted with `shadow.color`, instead of the image as it is. Only with PNG and JPEG.



```
#goban(size: 3, scale: 2,
  stone: (shadow: (on: true, angle: -45deg,
    offset: 2pt, color: blue,
    image: (file: path("img/pgfgo-shadow.png"),
      width: 16pt, silhouette: true))),
  stones(black: "B1, C2", white: "A3"))
```

`stone.shine` `true`, `false` or a dictionary

The shine of the stone, drawn on top of it, be it a circle or an image.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", shine: true),
  stones(white: "B4", shine: (on: true,
    color: luma(50%))))
```

`stone.shine.angle` an angle — initially `0deg`

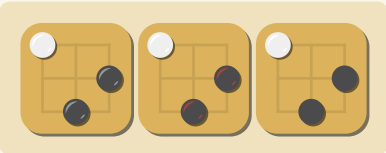
Rotates the drawn shine. With `0deg`, the light comes from the top left.

`stone.shine.opacity` a number from 0 to 1 — initially `1`

The opacity of the shine (also that of the shine with an image).

`stone.shine.color` a color — initially `luma(75%)`

The color of the drawn shine.

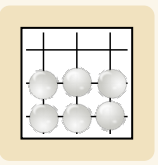


```
#gobanist-set(profile: "pgfgosmooth", size: 3)
#let b = board(stones(black: "B1, C2", white: "A3"))

#goban(b, players: (black: (stone: (
  shine: (angle: 45deg)))))
#goban(b, players: (black: (stone: (
  shine: (opacity: 0.5, color: red)))))
#goban(b, players: (black: (stone: (
  shine: false))))
```

`stone.shine.image` a file or a dictionary

A shine of your own: an image, centered on the stone, instead of the drawn shine. If the stone is also an image, the shine is drawn on top and does not rotate with it: the stones can have grain rotated at random and the light always on the same side. What is not a dictionary is the file; `none` removes it.



```
#goban(size: 5, partial: "B2:D4",
  stones(white: "B2:D3",
    image: (file: path("img/pgfgo-yunzi-white-1.png"),
      rotate: "random"),
    shine: (on: true,
      image: path("img/pgfgo-shine.png"))))
```

`stone.shine.image.file` a file — initially `none`

The file of the image.

`stone.shine.image.width` `auto` or a length — initially `auto`

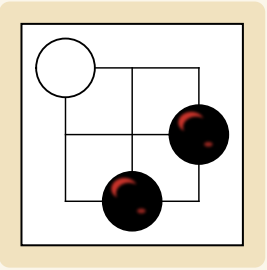
The width of the image; the height keeps the proportion. With `auto`, the diameter of the stone.

`stone.shine.image.rotate` an angle — initially `0deg`

Rotates the image of the shine, the same on every stone.

`stone.shine.image.silhouette` `true` or `false` — initially `false`

Uses only the shape of the image (its transparency), painted with `shine.color`, instead of the image as it is. Only with PNG and JPEG.



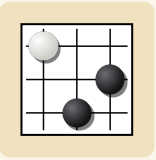
```
#goban(size: 3, scale: 2,
  players: (black: (stone: (shine: (on: true,
    color: red, image: (
      file: path("img/pgfgo-shine.png"),
      silhouette: true))))),
  stones(black: "B1, C2", white: "A3"))
```

• Stones with images

`stone.image` a file or a dictionary

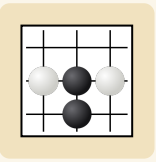
Draws the stone with an image instead of the circle: neither the fill nor the border are drawn, and the image need not be round. If the shine is on, it is drawn on top of the image. The shadow is drawn with the shape of the image; for that, the image must have transparency (a PNG with an alpha channel): the shadow of a JPEG is a rectangle. A prisoner with an image looks translucent, like any prisoner. What is not a dictionary is the file; `none` removes it.

A file is a `path`, relative to the document that names it, or its `bytes`: PNG, JPEG, GIF, WebP, SVG or PDF. Opacity and silhouettes (the shadow with the shape of the image) are possible only with PNG and JPEG: they are made by the kernel (p. 31). The images of the `shell`, `yunzi` and `baduktv` profiles are in the `images` folder of the package.



```
#goban(size: 5, partial: "B2:D4",
  stone: (shadow: (on: true, angle: -45deg,
    offset: 1.5pt)),
  stones(black: "C2, D3", image:
    path("img/pgfgo-yunzi-black.png")),
  stones(white: "B4", image:
    path("img/pgfgo-yunzi-white-1.png")))
```

For all the stones of a player, the image goes in their options:

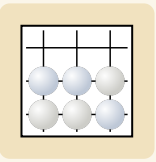


```
#gobanist-set(players: (
  black: (stone: (image:
    path("img/pgfgo-yunzi-black.png"))),
  white: (stone: (image:
    path("img/pgfgo-yunzi-white-1.png")))))

#goban(size: 5, partial: "B2:D4",
  moves("C3, D3, C2, B3", numbers: false))
```

`stone.image.file` a file or an array — initially `none`

The file of the image. With several files, each stone uses one, chosen by its place: a stone on a point always shows the same image, in every diagram.



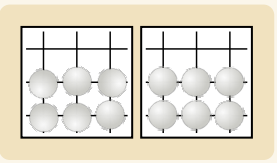
```
#goban(size: 5, partial: "B2:D4",
  stones(white: "B2:D3", image: (
    path("img/pgfgo-yunzi-white-1.png"),
    path("img/pgfgo-yunzi-white-2.png"))))
```

`stone.image.width` `auto` or a length — initially `auto`

The width of the image; the height keeps the proportion. With `auto`, the diameter of the stone: 2 × `radius` × `scale`.

`stone.image.rotate` an angle, `"random"` or `(min, max)` — initially `0deg`

Rotates the image, counterclockwise. With `"random"`, each stone is rotated by a different angle, chosen by its place: a stone on a point always looks the same, in every diagram. With a range `(min, max)`, the angle is chosen the same way, between those two (`"random"` is `(0deg, 360deg)`). The shadow rotates with the image, but it still falls to the same side.

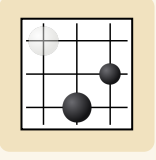


```
#let shell = path("img/pgfgo-yunzi-white-1.png")

#goban(size: 5, partial: "B2:D4",
  stones(white: "B2:D3", image: (file: shell,
    rotate: "random")))
#goban(size: 5, partial: "B2:D4",
  stones(white: "B2:D3", image: (file: shell,
    rotate: (-20deg, 20deg))))
```

`stone.image.opacity` a number from 0 to 1 — initially `1`

The opacity of the image. It is multiplied by that of the fill: so a prisoner with an image is translucent.



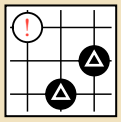
```
#let slate = path("img/pgfgo-yunzi-black.png")

#goban(size: 5, partial: "B2:D4",
  stones(black: "C2", image: slate),
  stones(black: "D3", image: (file: slate,
    width: 8pt)),
  stones(white: "B4", image: (opacity: 0.5,
    file: path("img/pgfgo-yunzi-white-1.png"))))
```

The other options of the stones

`mark` a shape or a dictionary — initially `none`

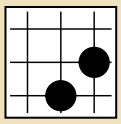
A mark on the stones, with the shape (`"triangle"`) or, with options, a dictionary with the shape in `shape` and the options of the marks (p. 64): `mark: (shape: "circle", line: (color: red))`. It is an option of `stones`, `prisoners` and `moves`, not of the `stone` branch.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", mark: "triangle"),
  stones(white: "B4", mark: (shape: "custom",
    content: [!], label: (text: (fill: red)))))
```

`forget` `true` or `false` — initially `false`

`resume` does not bring these stones (p. 80). `goban-view` draws them only with `forgotten`. It is an option of `stones`, `prisoners` and `moves` (where it can also be `"number"`, p. 60), of `marks`, `arrows`, `lines` and `territory`.



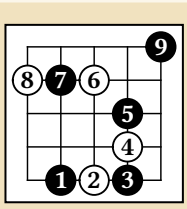
```
#gobanist-set(size: 5, partial: "B2:D4")

#let a = board(stones(black: "C2, D3"),
  stones(white: "B4", forget: true))
#goban(resume: a)
```

2 Moves

```
moves(moves, black: ..., white: ..., ..options)
```

Plays the intersections in the order in which they are written, alternating Black and White, and numbers them. Without a player, they follow the turn: `moves("C3, D4")`; with one, they start with it: `moves(white: "D2, B3")`. It takes a single sequence. An intersection can be repeated (for example, to take back a ko with `captures`). It accepts the syntax of `stones` (p. 33) and plays each element in its order (p. 37): a path (`A1 -- A3 -- C3`) goes from its first point to its last, and a stretch, from the first end to the second. What has no fixed path (rectangles, `r`, whole columns and rows, `*`) is played all the same. The `;` is an error: `moves` only adds moves.

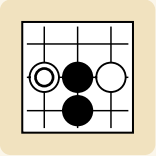



```
#goban(size: 5,
  moves("B1 -| D3, C:A 4, E5"))
```

The options are those of the style of the stone (`fill`, `line`, `radius`, `shine`, `image`, ...), `mark`, `forget` and those of this section: `numbers`, `label` and `from`.

Within the same `goban`, the numbering and the turn carry on from one `moves` to the next. Each `goban` starts at move 1 with Black; to continue a game in another diagram, use `from` and the player, or `resume` (p. 80).

The options of the stones serve, for example, to mark the last move:

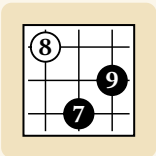


```
#goban(size: 5, partial: "B2:D4",
  moves("C3, D3, C2", numbers: false),
  moves("B3", numbers: false, mark: "circle"))
```

The style of the moves of the three players is changed with `gobanist-set(move: ...)`, which is `gobanist-set(players: (all: (move: ...)))`.

`from` a number

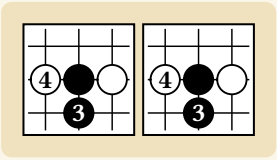
The number of the first move of this `moves`. The following ones carry on from there.



```
#goban(size: 5, partial: "B2:D4",
  moves("C2, B4", from: 7),
  moves("D3"))
```

`move.numbers` `true` or `false` initially `true`

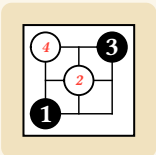
Whether the moves carry their number. Without numbers, the moves still count: the next one carries on the numbering and the turn. In `moves(numbers: false)`, it holds for those moves; with `gobanist-set(move: (numbers: false))` or `goban(move: (numbers: false))`, for all those that follow.



```
#goban(size: 5, partial: "B2:D4",
  moves("C3, D3", numbers: false),
  moves("C2, B3"))
#goban(size: 5, partial: "B2:D4",
  move: (numbers: false),
  moves("C3, D3"),
  moves("C2, B3", numbers: true))
```

`move.label` a dictionary

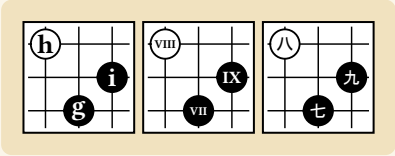
The numbers of the moves.



```
#goban(size: 3,
  players: (white: (move: (label: (
    text: (fill: red, style: "italic"),
    size: 6pt)))),
  moves("A1, B2, C3, A3"))
```

`move.label.format` a `numbering` pattern initially `"1"`

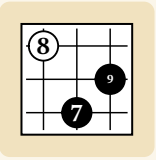
The format of the numbers: a Typst `numbering` pattern, such as `"1"`, `"a"`, `"A"`, `"i"`, `"I"` or `"一"` (the kanji need a font that has them).



```
#goban(size: 5, partial: "B2:D4",
  move: (label: (format: "a")),
  moves("C2, B4", from: 7), moves("D3"))
#goban(size: 5, partial: "B2:D4",
  move: (label: (format: "I")),
  moves("C2, B4", from: 7), moves("D3"))
#goban(size: 5, partial: "B2:D4",
  move: (label: (format: "一",
    text: (font: "MS Gothic"))),
  moves("C2, B4", from: 7), moves("D3"))
```

`move.label.size` `auto` or a length initially `auto`

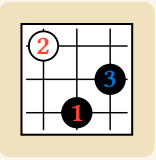
The size of the numbers. With `auto`, it is adjusted to the number of digits.



```
#goban(size: 5, partial: "B2:D4",
  moves("C2, B4", from: 7),
  moves("D3", label: (size: 5pt)))
```

`move.label.text` a dictionary — initially `(weight: "bold", fill: ...)`

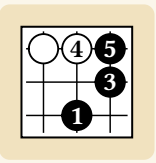
The `text` options of the numbers: the font, the weight, the color. The color is the opposite of that of the stone: white on Black and black on White.



```
#goban(size: 5, partial: "B2:D4",
  moves("C2, B4", label: (text: (fill: red))),
  moves("D3", label: (text: (fill: blue))))
```

`forget` `true`, `false` or `"number"` — initially `false`

In `moves`: with `true`, `resume` does not bring these moves, neither the stone nor the number, and the numbering carries on from the last move it brings. With `"number"`, the move comes without its number even if `numbers` is asked for (p. 80); with `goban-view`, the number is seen only with `forgotten`.



```
#gobanist-set(size: 5, partial: "B2:D4")

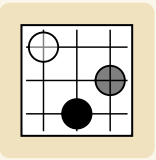
#let a = board(moves("C2"),
  moves("B4", forget: "number"),
  moves("D3"))
#goban(resume: (board: a, numbers: true),
  moves("C:D4"))
```

3 Prisoners

Prisoners are captured stones that are still on the board: by default, translucent.

```
prisoners(coordinates, black: ..., white: ..., neutral: ..., ..options)
```

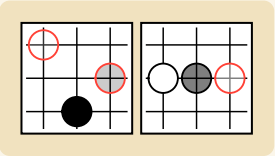
Like `stones`, with the style of the prisoners: that of the stone of the player, with the `prisoner` branch on top.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2"),
  prisoners(black: "D3", white: "B4"))
```

`prisoner` stone options — initially `(fill: (opacity: 0.5))`

The style of the prisoners: options of the `stone` branch, which are put on top of those of the stone of the player. With `gobanist-set(prisoner: ...)`, for the three players; with `gobanist-set(players: (white: (prisoner: ...)))`, for one. It holds for those of `prisoners` and for the stones captured with `captures: "fade"` (p. 72). The other stones of the player do not change.



```
#gobanist-set(size: 5, partial: "B2:D4")

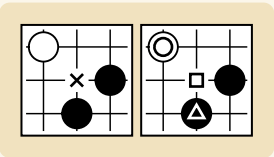
#goban(prisoner: (fill: (opacity: 0.2),
    line: (color: red)),
  stones(black: "C2"),
  prisoners(black: "D3", white: "B4"))
#goban(players: (white: (prisoner: (
    line: (color: red)))),
  stones(white: "B3"),
  prisoners(black: "C3", white: "D3"))
```

7 Marks, arrows and lines

Marks go on an intersection, with or without a stone: shapes (circle, triangle, cross, ...), numbers in sequence or any content.

```
marks(coordinates, shape: ..., ..options)
```

Places marks on the intersections, written with the coordinate syntax (p. 33). The shape is the key: `marks(triangle: "C3", circle: "E5")`. Without a shape, a cross: `marks("C3")`. The same `marks` can hold several shapes, and its options hold for all of them.



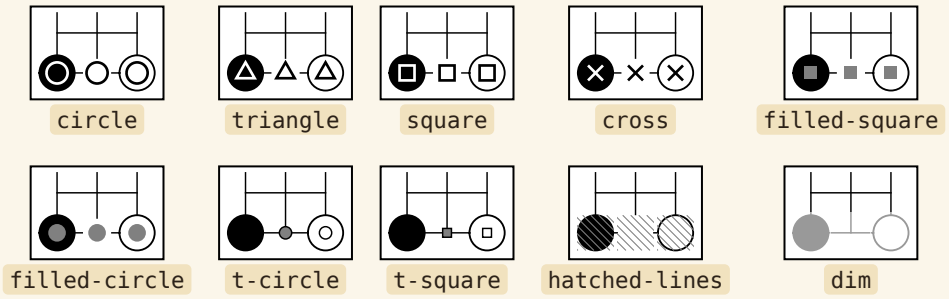
```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, marks("C3"))
#goban(b, marks(triangle: "C2",
  circle: "B4", square: "C3"))
```

Without `player`, a mark on a stone belongs to the player of the stone, with their style, so that it can be seen; on an empty point, it is neutral. On an empty point, the mark covers the grid with the background of the board (except `t-circle`, `t-square` and `dim`).

1 The shapes

The shapes of the package are `circle`, `triangle`, `square`, `cross`, `filled-square`, `filled-circle`, `t-circle`, `t-square`, `hatched-lines`, `dim`, `sequence` and `custom`. Those of the `shapes` option (p. 68) are used the same way.



`circle`

A circle.

`triangle`

A triangle.

`square`

A square.

`cross`

A cross. It is the shape of `marks` without a shape.

`filled-square`

A filled square, with the fill of the mark (`fill`).

`filled-circle`

A filled circle, with the fill of the mark.

`t-circle`

A small circle, with the fill of the stones of the player and the border of the grid. It is the mark of Black's territory.

`t-square`

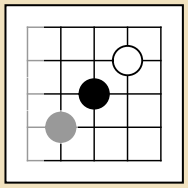
Like `t-circle`, but square. It is the mark of White's territory and of the neutral points.

`hatched-lines`

A hatching of lines, the color of the fill of the mark.

`dim`

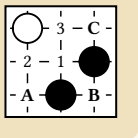
Dims the point: it covers it with the background of the board, translucent, over the stone and the grid. It is what a game draws for the `DD` property.



```
#goban(size: 5,
  stones(black: "B2, C3", white: "D4"),
  marks(dim: "A1:5, B2"))
```

`sequence`

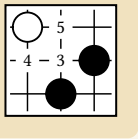
Numbers the marks, in the order of the coordinates (p. 37), from `from`. The format, the size and the color of the numbers are those of `label`, like the content of `custom`.




```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(sequence: "C3, B3, C4"),
  marks(sequence: "B2, D2, D4",
    label: (format: "A",
      text: (weight: "bold"))))
```

`from` a number initially `1`

In `marks`, the number of the first mark of `sequence`.



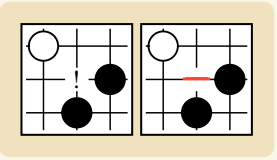

```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(sequence: "C3, B3, C4", from: 3))
```

`custom`

A mark with any content, that of `content`: a text, a symbol ("!", "?") or a drawing.

`content` content

In `marks`, what `custom` draws. It is required with `custom`. Its size is adjusted to the number of characters, except with `label.size`.



```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

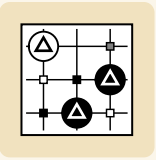
#goban(b, marks(custom: "C3",
  content: [!]))
#goban(b, marks(custom: "C3",
  content: box(stroke: red, radius: 2pt,
    inset: 1pt, text(fill: red)[…])))
```

2 The options of the marks

The style of the marks is that of their player: `gobanist-set(mark: ...)` is `gobanist-set(players: (all: (mark: ...)))`, and the options of each `marks` go on top.

`player` `auto`, `"black"`, `"white"` or `"neutral"` — initially `auto`

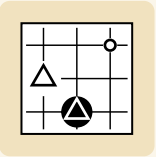
The player of the marks, whose style they take. With `auto`, that of the stone on the point, or neutral if there is none.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(t-square: "B2, C3", player: "black"),
  marks(t-square: "D2, B3", player: "white"),
  marks(t-square: "D4", player: "neutral"),
  marks(triangle: "C2, D3, B4"))
```

`mark.scale` a number — initially `1`

The size of the mark relative to the usual one: the shapes, the images and the text.



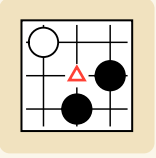
```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2"),
  marks(triangle: "C2, B3", scale: 1.5),
  marks(circle: "D4", scale: 0.6))
```

`mark.line` a dictionary

The stroke of the mark.

`mark.line.color` a color

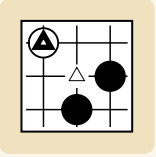
The color of the stroke: white on Black's marks and black on the others.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(triangle: "C3",
    line: (color: red)))
```

`mark.line.width` a length — initially `1pt`

The width of the stroke.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(triangle: "C3", line: (width: 0.3pt)),
  marks(triangle: "B4", line: (width: 2pt)))
```

`mark.line.opacity` a number from 0 to 1 — initially `1`

The opacity of the stroke.

`mark.fill` `true`, `false` or a dictionary

The fill of the filled marks (`filled-square`, `filled-circle` and `hatched-lines`), and the `fill` of the shapes of your own (p. 68). With `false`, the filled marks are not seen.

`mark.fill.color` a color or `none` initially `luma(50%)`

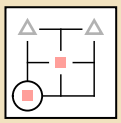
The color of the fill.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(filled-circle: "C3",
    fill: (color: red)))
```

`mark.fill.opacity` a number from 0 to 1 initially `1`

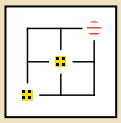
The opacity of the fill.



```
#goban(size: 3,
  players: (white: (mark: (
    fill: (color: red, opacity: 0.5),
    line: (opacity: 0.3)))),
  stones(white: "A1"),
  marks(filled-square: "A1, B2",
    player: "white"),
  marks(triangle: "C3, A3", player: "white"))
```

`mark.fill.pattern` `none` or a `tiling` initially `none`

A pattern over the color of the fill of `filled-square` and `filled-circle`, like `stone.fill.pattern` (p. 54). `hatched-lines` and the shapes of your own use only the color.



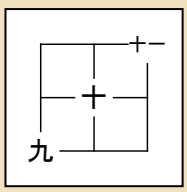
```
#goban(size: 3,
  marks(filled-square: "A1, B2", fill: (
    color: yellow, pattern: tiling(
      size: (2pt, 2pt), square(size: 1pt,
        fill: black)))),
  marks(filled-circle: "C3", fill: (
    color: none, pattern: tiling(
      size: (2pt, 2pt), line(length: 2pt,
        stroke: red)))))
```

`mark.label` a dictionary

The text of the marks that have one: `sequence` and `custom`.

`mark.label.format` a `numbering` pattern initially `"1"`

The format of the numbers of `sequence`: a `numbering` pattern, such as `"1"`, `"a"`, `"A"`, `"i"`, `"I"` or `"一"`. The kanji need a font that has them, in `label.text`.



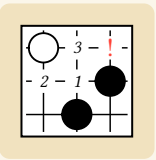

```
#goban(size: 3, scale: 1.6,
  marks(sequence: "A1, B2, C3", from: 9,
    label: (format: "一",
      text: (font: "MS Gothic"))))
```

`mark.label.size` `auto` or a length initially `auto`

The size of the text. With `auto`, it is adjusted to the number of characters.

`mark.label.text` a dictionary

The `text` options of the text: the font, the style, the color. The color is that of `line.color`.




```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(sequence: "C3, B3, C4",
    label: (text: (style: "italic"))),
  marks(custom: "D4", content: [!],
    label: (text: (fill: red))))
```

`forget` `true` or `false` initially `false`

In `marks`, `arrows` and `lines`: `resume` does not bring these marks, arrows or lines, even if `marks` is asked for (p. 80).

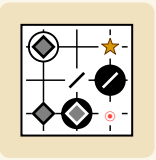
3 Shapes of your own

`shapes` a dictionary of shapes initially `(:)`

Mark shapes of your own, by name: `shapes: (diamond: ..., star: ...)`. They are used like those of the package: in `marks`, in the `mark` option of the stones, in `territory` and in `last-mark`. A shape with the name of another replaces it. None can be named like an option of `marks` (`line`, `fill`, `label`, `scale`, `player`, `from`, `content` or `forget`). A shape is:

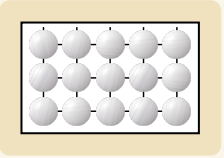
- a function: it takes a dictionary with `radius` (the radius of the stone of the player, with `scale`), `stroke` (the stroke of the mark), `fill` (its fill, `none` if it is off) and `color` (the color of its lines), and what it returns is drawn with its origin on the point;
- content, centered on the point;
- an image, `(image: file)` or `(image: (file: file, width: ...))`, centered on the point, without the colors of the mark. `width` is its width (with `auto`, 1.2 stone radii), which changes with `scale`. With several files, each point uses one, always the same one.

On an empty point, a shape of your own covers the grid like the others.



```
#gobanist-set(shapes: (
  diamond: m => polygon(fill: m.fill,
    stroke: m.stroke,
    (0pt, -0.7 * m.radius),
    (0.7 * m.radius, 0pt),
    (0pt, 0.7 * m.radius),
    (-0.7 * m.radius, 0pt)),
  bar: m => line(
    start: (-0.5 * m.radius, 0.5 * m.radius),
    end: (0.5 * m.radius, -0.5 * m.radius),
    stroke: m.stroke),
  star: (image: path("img/pgfgo-star.pdf")),
  eye: text(fill: red, size: 8pt)[◉],
))

#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(diamond: "C2, B4, B2", bar: "D3, C3",
    star: "D4", eye: "D2"))
```



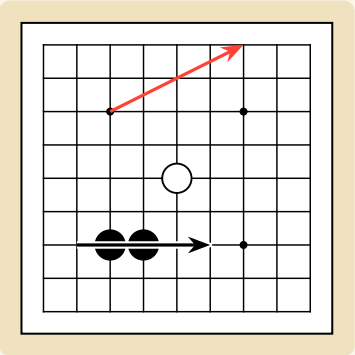
```
#goban(size: 5, partial: "A1:E3",
  shapes: (shell: (image: (width: 1em,
    file: range(1, 6).map(n => path(
      "img/pgfgo-shell-white-" + str(n)
        + ".png"))))),
  marks(shell: "A:E 1:3"))
```

4 Arrows and lines

Arrows and lines join two intersections. They are drawn on top of the stones, with a halo the color of the background of the board that separates them from the stones and the grid. A game draws the arrows (`AR`) and the lines (`LN`) of the file.

`arrows(..pairs, ..options)`

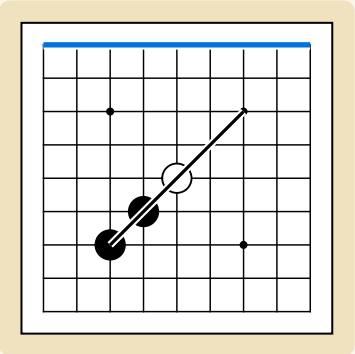
Draws arrows, each one from one intersection to another: `arrows(("B3", "F3"), ("C7", "G9"))`. The options hold for all of them.



```
#goban(stones(black: "C3, D3", white: "E5"),
  arrows(("B3", "F3")),
  arrows(("C7", "G9"), color: red,
    halo: false))
```

`lines(..pairs, ..options)`

Draws lines, each one from one intersection to another. It has the options of `arrows`, except `tip` and `tips`.



```
#goban(stones(black: "C3, D4", white: "E5"),
  lines(("C3", "G7")),
  lines(("A9", "J9"), color: blue, width: 2pt))
```

The style of every arrow is set with `gobanist-set(arrow: ...)`, and that of every line, with `gobanist-set(line: ...)`.

`arrow.color` a color — initially `black`

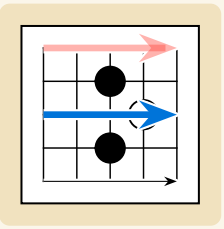
The color of the arrow.

`arrow.width` a length — initially `1.2pt`

The width of the arrow. The tip grows with it.

`arrow.opacity` a number from 0 to 1 — initially `1`

The opacity of the arrow.



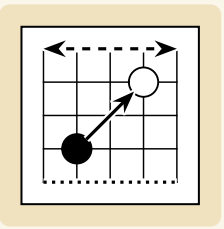
```
#goban(size: 5,
  stones(black: "C2, C4", white: "D3"),
  arrows(("A1", "E1"), width: 0.4pt),
  arrows(("A3", "E3"), color: blue,
    width: 2.5pt),
  arrows(("A5", "E5"), color: red,
    opacity: 0.4, width: 2.5pt))
```

`arrow.dash` `"solid"`, `"dashed"` or `"dotted"` — initially `"solid"`

The stroke: solid, dashed or dotted. The halo and the tip are always solid.

`arrow.shorten` a length — initially `0pt`

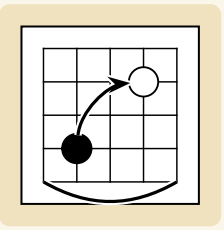
Shortens both ends.



```
#goban(size: 5,
  stones(black: "B2", white: "D4"),
  arrows(("B2", "D4"), shorten: 5pt),
  arrows(("A5", "E5"), dash: "dashed",
    tips: "both"),
  lines(("A1", "E1"), dash: "dotted"))
```

`arrow.bend` an angle — initially `0deg`

Draws an arc instead of a straight line: the arrow leaves turned that angle to the left and arrives turned the same. With a negative angle, it bends to the right.



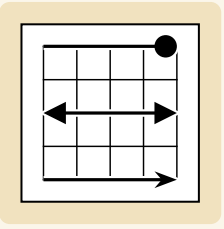
```
#goban(size: 5,
  stones(black: "B2", white: "D4"),
  arrows(("B2", "D4"), bend: 40deg,
    shorten: 5pt),
  lines(("A1", "E1"), bend: -30deg))
```

`arrow.tip` `"stealth"`, `"triangle"` or `"circle"` — initially `"stealth"`

The tip of the arrow.

`arrow.tips` `"end"` or `"both"` — initially `"end"`

Where the tip goes: at the end, or at both ends.



```
#goban(size: 5,
  arrows(("A1", "E1")),
  arrows(("A3", "E3"), tip: "triangle",
    tips: "both"),
  arrows(("A5", "E5"), tip: "circle"))
```

`arrow.halo` `true`, `false` or a dictionary

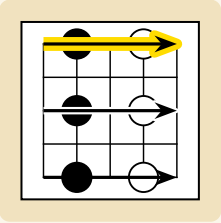
The halo around the arrow.

`arrow.halo.color` `auto` or a color — initially `auto`

The color of the halo. With `auto`, that of the background of the board.

`arrow.halo.width` a length — initially `0.8pt`

The width of the halo on each side of the arrow.



```
#goban(size: 5,
  stones(black: "B1, B3, B5",
    white: "D1, D3, D5"),
  arrows(("A1", "E1"), halo: false),
  arrows(("A3", "E3")),
  arrows(("A5", "E5"), halo: (color: yellow,
    width: 2pt)))
```

Lines have the same options, in the `line` branch, except `tip` and `tips`:

`line.color` a color — initially `black`

Like `arrow.color`.

`line.width` a length — initially `1.2pt`

Like `arrow.width`.

`line.opacity` a number from 0 to 1 — initially `1`

Like `arrow.opacity`.

`line.dash` `"solid"`, `"dashed"` or `"dotted"` — initially `"solid"`

Like `arrow.dash`.

`line.shorten` a length — initially `0pt`

Like `arrow.shorten`.

`line.bend` an angle — initially `0deg`

Like `arrow.bend`.

`line.halo` `true`, `false` or a dictionary

Like `arrow.halo`, with `color` and `width`.

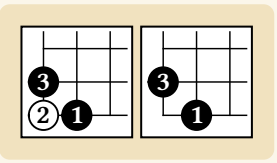
8 The game: captures, notes and territory

A `goban` can follow the rules of the game: remove the captured stones, note the moves that fall on points already played and count the territory. This is done by the kernel (p. 31), which plays the whole board when drawing it.

1 Captures

`captures` `none`, `"remove"` or `"fade"` — initially `none`

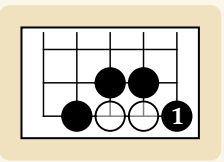
The captures. When a move leaves an opposing group without liberties, that group is removed from the board; if it leaves its own group without liberties (a suicide), that group is removed. The stones of `stones` count too; neutral stones and prisoners do not. A move outside a partial board also captures. A move on an occupied point is an error, because it is not possible in a game (except in `sgf-diagrams` and `sgf-problem`, p. 90). With `none`, there are no captures: the moves are drawn on top of one another.




```
#gobanist-set(size: 5, partial: "A1:C3")

#goban(moves("B1, A1, A2"))
// 3 captures 2.
#goban(captures: "remove",
  moves("B1, A1, A2"))
```

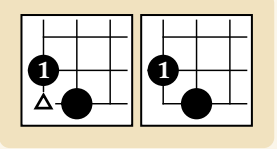
With `"remove"`, the captured stones disappear. With `"fade"`, they stay on the board, drawn as prisoners (with the style of `prisoner`, p. 62).




```
#goban(size: 5, partial: "A1:E3",
  captures: "fade",
  stones(white: "C1, D1",
    black: "B1, C2, D2"),
  moves("E1"))
```

`captured-marks` `"keep"` or `"remove"` — initially `"keep"`

What happens to the marks on a stone when it is captured. With `"keep"`, they stay on the empty point. With `"remove"`, they go away with the stone, also with `captures: "fade"`; sequences keep their numbering. The marks on empty points do not change.

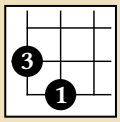



```
#gobanist-set(size: 5, partial: "A1:C3",
  captures: "remove")

#goban(
  stones(white: "A1", black: "B1"),
  marks(triangle: "A1"), moves("A2"))
#goban(captured-marks: "remove",
  stones(white: "A1", black: "B1"),
  marks(triangle: "A1"), moves("A2"))
```

`goban-prisoners(player)`

How many prisoners the player (`"black"` or `"white"`) has in the last `goban`: the opponent's stones captured by their moves.



Black: 1. White: 0.

```
#goban(size: 5, partial: "A1:C3",
  captures: "remove",
  moves("B1, A1, A2"))

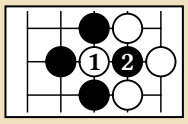
Black: #goban-prisoners("black").
White: #goban-prisoners("white").
```

2 Moves on points already played

In numbered diagrams, when a move falls on a point where there was already a move (for example, when taking back a ko), books do not draw it: the point keeps showing the first stone, and below the diagram one notes, for example, “3 at 1”.

`notes` `true`, `false` or a dictionary

A numbered move on a point where there was already a numbered move of the same `goban` is not drawn: the point keeps showing the first one, even if it was captured, and “N at M” is noted. Only the numbered moves seen on the board count. A pass of a game that would be numbered is noted “N: pass”, in order with the other notes; the passes with which the game ends are not noted.



3 at 1

```
#goban(size: 5, partial: "A2:E4",
  notes: true,
  stones(black: "B3, C2, C4",
    white: "D2, D4, E3"),
  moves(white: "C3"), moves("D3"),
  moves(white: "C3", from: 3))

#goban-notes()
```

`notes.at` `auto` or a text — initially `auto`

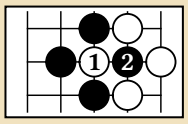
The text between the two numbers. With `auto`, that of the language (`at` in `texts`, p. 31).

`notes.separator` a text or content — initially `", "`

The text between the notes.

```
goban-notes(at: auto, separator: auto)
```

The notes of the last `goban`, separated by `separator`. Without notes, nothing. With `auto`, `at` and `separator` are those of the `notes` branch.



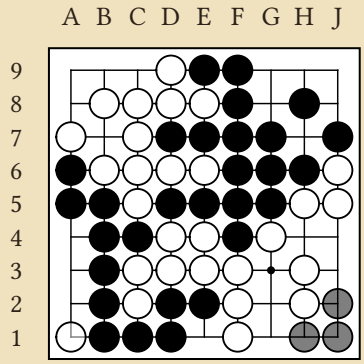
3 on 1; 4 on 2

```
#goban(size: 5, partial: "A2:E4",
  notes: true, captures: "remove",
  stones(black: "B3, C2, C4",
    white: "D2, D4, E3"),
  moves(white: "C3"), moves("D3"),
  moves(white: "C3", from: 3), moves("D3"))

#goban-notes(at: "on", separator: "; ")
```

3 Territory

At the end of the game, the territory can be marked by hand with `marks` (p. 64) or with `territory`, which also counts it. In the examples, `end` is this position, with its prisoners:



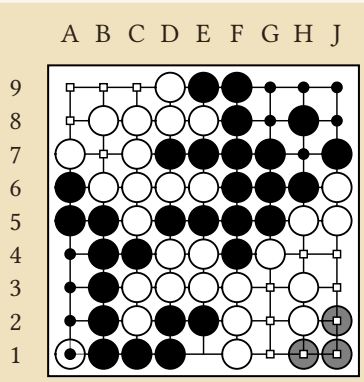
```
#let end = board(
  stones(black: "A6 |- B5 |- D1 |- E2, C4,
      E9 -| F4, D5 -| G7 -- D7, H6, H8, J7",
    white: "E8 -| C5, B:E6, A7, B8, D9,
      J6 |- H5, H2:3, F1 |- C3, C2, D:E4, G4"),
  prisoners(black: "H1 -| J2", white: "A1"))

#goban(label: (at: (top, left)), resume: end)
```

`territory(polygon, black: ..., white: ..., neutral: ..., ..options)`

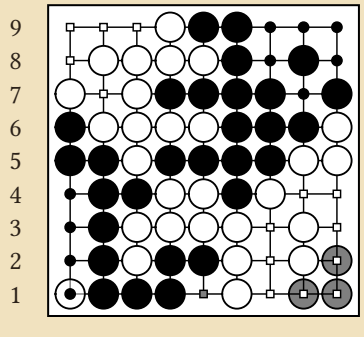
Marks as territory the points of the polygon with those vertices (in order, with the coordinate syntax), its border included. It does not mark the points that have a living stone: prisoners count as territory. A single vertex marks one point, and two, a stretch. The player is the key: `territory(black: "A1, A5, B5, B1")`; without a player, the polygon is neutral.

The territory is drawn at the end of the `goban`. Each point keeps the last `territory` that marked it, with the player and the options of that `territory` (so `territory(neutral: "A1")` after another `territory` makes A1 neutral), and it is neither drawn nor counted if at the end it has a living stone.



```
#goban(label: (at: (top, left)),
  resume: end,
  territory(black: "A1, A5, B5, B1"),
  territory(black: "F9, F7, G7, G6, J6, J9"),
  territory(white: "F1, F4, G4, G5, J5, J1"),
  territory(white: "A9, A6, C6, C8, D8, D9"))
```

With `auto` instead of a polygon, it finds the territory by itself: each region of points without living stones belongs to the color that surrounds it. If both colors touch it (dame, seki) or neither does, it is neutral. Prisoners count as dead stones. Without a player, `territory(auto)` marks the territory of both; with `territory(black: auto)`, only that of this player.



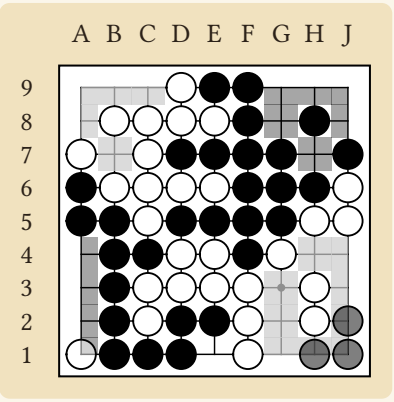
```
#goban(label: (at: (top, left)),
  resume: end,
  territory(auto, dame: true))
```

`auto` only looks at which color touches each region; it cannot read the position. Therefore:

- It is for finished games. In an open position, almost the whole board touches both colors and is neutral; with stones of a single color, the whole board is theirs. If a wall has a gap, the territory inside joins the dame and also becomes neutral.
- It does not know which stones are dead: they have to be placed with `prisoners`. A dead stone placed with `stones` counts as alive, and the region it touches becomes neutral.
- The eyes of a group in seki count as territory (under Japanese rules they are not counted). They can be made neutral afterwards, with a `territory(neutral: ...)` for each one.
- Neutral stones occupy their point, but belong to no one: a region that only touches neutral stones is neutral.
- It finds the regions with the stones on the board where the `territory` is, so it is best placed at the end of the `goban`.
- With `partial: auto`, the board grows to show all the territory (and the dame, with `dame`).

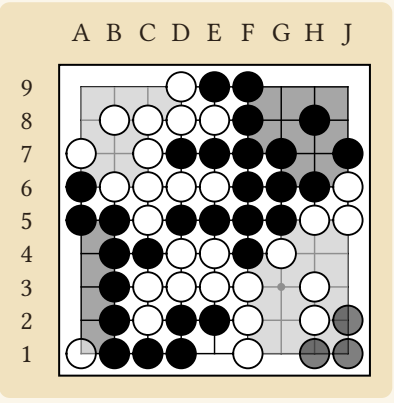
`territory.draw` `"marks"`, `"fill"` or `"region"` initially `"marks"`

How the territory is drawn. With `"marks"`, each point carries the territory mark of the player. With `"fill"`, each point is filled under the stones.




```
#goban(label: (at: (top, left)),
  resume: end,
  territory(auto, draw: "fill"))
```

With `"region"`, the territory of each color is a single polygon: every cell of the grid with a corner in the territory is painted (dead stones count as territory). Its sides run along the lines of the board, its vertices are always on intersections, and it reaches the border of the board. Two areas that touch at a single corner stay separate. All the `territory`s of a color in the same `goban` (also the `TB` and `TW` territory of a game) make a single region, drawn with the style of the last one. Neutral points are filled one by one, as with `"fill"`. The outline and its vertices can be drawn on top, with `line` and `vertices`, and `goban-region` gives its properties.




```
#goban(label: (at: (top, left)),
  resume: end,
  territory(auto, draw: "region"))
```

It is an option of `territory`, and with `gobanist-set(territory: (draw: ...))`, of all of them.

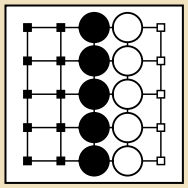
`territory.dame` `true` or `false` initially `false`

With `auto`, also marks the neutral points.

The other options of `territory` (`mark`, `fill`, `line` and `vertices`) change the style of the territory of the player only for that `territory`. The style of each player is in their `territory` branch:

`players.black.territory.mark` a shape or a dictionary — initially `"t-circle"`

The mark of each point, with `draw: "marks"`: `"t-circle"` for Black and `"t-square"` for White and the neutral points. With options, a dictionary with the shape in `shape`, like the `mark` of the stones.



```
#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    mark: "t-square"),
  territory(white: "E1, E5"))
```

`players.black.territory.fill` `true`, `false` or a dictionary

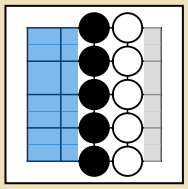
The fill, with `draw: "fill"` and `draw: "region"`. With `false`, no fill: with `"region"`, only the outline or the vertices remain.

`players.black.territory.fill.color` a color or `none` — initially `black`

The color of the fill: black for Black, `luma(80%)` for White and `luma(50%)` for the neutral points. With `none`, no color.

`players.black.territory.fill.opacity` a number from 0 to 1 — initially `0.35`

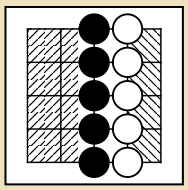
The opacity of the fill: `0.35` for Black and the neutral points, `0.7` for White.



```
#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    draw: "fill",
    fill: (color: blue, opacity: 0.5)),
  territory(white: "E1, E5", draw: "fill"))
```

`players.black.territory.fill.pattern` `none` or a `tiling` — initially `none`

A pattern over the color of the fill, like `stone.fill.pattern` (p. 54). With `fill.color: none`, only the pattern remains.

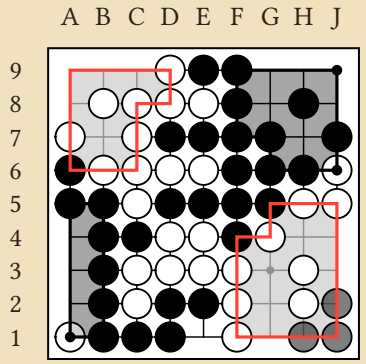


```
#let stripes(a, b) = tiling(size: (3pt, 3pt),
  line(start: (0pt, a), end: (3pt, b),
    stroke: 0.4pt))

#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    draw: "fill", fill: (color: none,
      pattern: stripes(3pt, 0pt))),
  territory(white: "E1:5", draw: "region",
    fill: (color: none,
      pattern: stripes(0pt, 3pt))))
```

`players.black.territory.line` `true`, `false` or a dictionary

The outline of the region, with `draw: "region"`.



```
#goban(label: (at: (top, left)),
  resume: end,
  territory(black: auto, draw: "region",
    line: true, vertices: true),
  territory(white: auto, draw: "region",
    line: (on: true, color: red)))
```

`players.black.territory.line.color` a color — initially `black`

The color of the outline: black for Black and `luma(40%)` for White.

`players.black.territory.line.width` a length — initially `1.2pt`

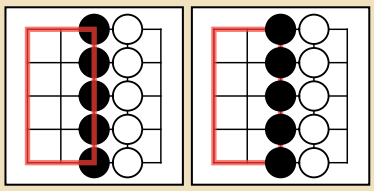
The width of the outline.

`players.black.territory.line.opacity` a number from 0 to 1 — initially `1`

The opacity of the outline and of the vertices.

`players.black.territory.line.above` `true` or `false` — initially `true`

With `true`, the outline and the vertices are drawn above the stones and the marks; with `false`, below the stones, together with the fill.



```
#let outline = (on: true, color: red,
  width: 2pt, opacity: 0.7)

#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    draw: "region", fill: false,
    line: outline))
#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    draw: "region", fill: false,
    line: outline + (above: false)))
```

`players.black.territory.vertices` `true`, `false` or a dictionary

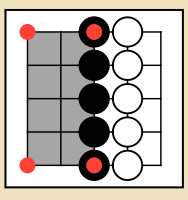
Dots on the vertices of the outline, with `draw: "region"`, on the same layer as the outline.

`players.black.territory.vertices.radius` a length — initially `2pt`

The radius of the dots.

`players.black.territory.vertices.color` `auto` or a color — initially `auto`

The color of the dots. With `auto`, that of the outline.

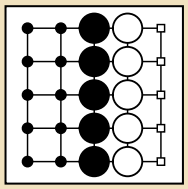


```
#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    draw: "region", vertices: (on: true,
      radius: 3pt, color: red)))
```

4 Score

```
goban-score(player, komi: auto)
```

The score of the player (`"black"` or `"white"`) in the last `goban`, with Japanese rules: the points of their territory, plus the dead stones of the opponent in it, plus the stones they captured (with `captures`), plus the komi for White. A point marked twice counts once. With `"neutral"`, the number of neutral points marked. With `auto`, the komi is that of `score.komi`.



Black: 10. White: 11.5.

```
#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(auto))

Black: #goban-score("black").
White: #goban-score("white", komi: 6.5).
```

`score.komi` a number initially `0`

The komi that `goban-score` adds to White: `gobanist-set(score: (komi: 6.5))`.

```
goban-region(player, property, n: none)
```

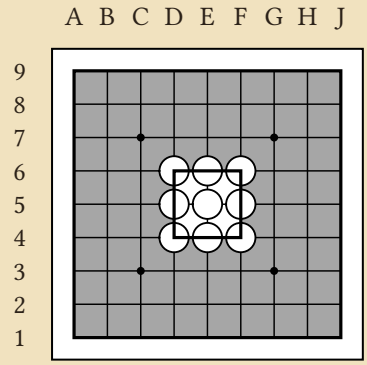
A property of the region (`draw: "region"`) of the player (`"black"` or `"white"`) in the last `goban`, as text. The properties are those of the whole outline, even if the board is partial. Without a region, the numbers are 0. The properties are:

- `"polygons"`: the number of outlines, outer ones and holes.
- `"outlines"`: the number of outer outlines, one for each separate area.
- `"holes"`: the number of holes.
- `"vertices"`: the number of vertices of all the outlines.
- `"area"`: the number of painted cells.
- `"perimeter"`: the length of all the outlines, in cells.
- `"polygon"`, with `n`: the vertices of outline `n` (from 1), as coordinates separated by commas (`"A1, D1, D4, A4"`). Empty if there are not that many outlines.

Each outline starts at its lowest vertex and, among those, at the leftmost one. Outer outlines run counter-clockwise and holes, clockwise. The outlines are ordered by that first vertex, from bottom to top.

```
#goban(label: (at: (top, left)),
  stones(white: "D4:F6"),
  territory(black: "B2, B8, H8, H2",
    draw: "region", line: true))

Outlines: #goban-region("black", "polygons")
(#goban-region("black", "holes") hole).
Cells: #goban-region("black", "area").
The hole:
#goban-region("black", "polygon", n: 2).
```



Outlines: 2 (1 hole). Cells: 60. The hole: D4, D6, F6, F4.

`goban-prisoners`, `goban-notes`, `goban-score` and `goban-region` speak of the last board drawn: that of a `goban` or the last diagram of `sgf-diagrams`, `sgf-problem` or `goban-layout`. Views (`goban-view`) do not count. They return content: what they say is known when drawing.

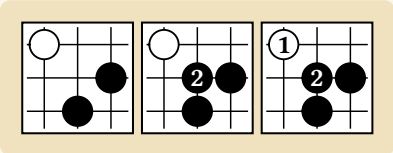
9 Boards as values and views

1 Going on from a board

A board can be a value: `board` (p. 40) takes the same as `goban` and draws nothing. It is saved with `let`, and it is used in two ways:

- first in a `goban` or in another `board`: `goban(base, moves("C3"))` is the same board, with everything it has (its options, its marks, its numbers), plus what is added;
- with `resume`: another board starts with that one, as it was at the end, with the captures resolved; it brings only what is asked for (by default, the stones), and the numbering and the turn carry on from there.

`goban-view` shows a part in the middle of the text (p. 81).




```
#gobanist-set(size: 5, partial: "B2:D4")

#let board1 = board(stones(black: "C2, D3"),
  moves(white: "B4"))

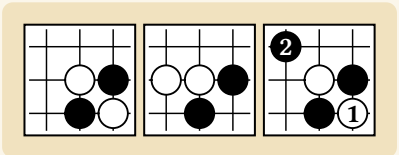
#goban(resume: board1)
// Carries on with 2 (Black).
#goban(resume: board1, moves("C3"))
#goban(resume: (board: board1,
  numbers: true), moves("C3"))
```

`resume` a board or `(board: ..., numbers: ..., marks: ..., territory: ...)`

Starts the board with the board `board`, as it was at the end: with the captures resolved (if that board had `captures`, this one does too, unless it gives its own). The numbering and the turn carry on from there. A board alone, `resume: base`, brings only the stones, without numbers, marks or territory; in a dictionary, `numbers`, `marks` and `territory` ask for the rest.

The size is that of the `resume` board, unless this one gives its own (`size`, or a game); one set with `gobanist-set` does not count. The other options of that board (`partial`, the coordinates, the style) do not come: they are given on each board.

A `board` with `resume` has what it brought, as it was drawn, plus its own, and the `resume` board does not change: that is how variations are built.




```
#gobanist-set(size: 5, partial: "B2:D4")

#let base = board(stones(black: "C2, D3",
  white: "C3"))
#let variation1 = board(resume: base,
  moves(white: "D2"))
#let variation2 = board(resume: base,
  moves(white: "B3"))

#goban(resume: variation1)
#goban(resume: variation2)
#goban(resume: (board: variation1,
  numbers: true), moves("B4"))
```

`resume.numbers` `true` or `false` initially `false`

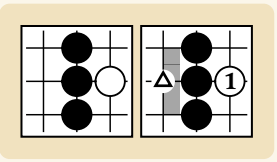
Also brings the numbers of the moves.

`resume.marks` `true` or `false` initially `false`

Also brings the marks, the arrows and the lines.

`resume.territory` `true` or `false` initially `false`

Also brings the territory.



```
#gobanist-set(size: 5, partial: "B2:D4")

#let saved = board(stones(black: "C2:4"),
  moves(white: "D3"), marks(triangle: "B3"),
  territory(black: "B2, B4", draw: "fill"))

#goban(resume: saved)
#goban(resume: (board: saved,
  numbers: true, marks: true,
  territory: true))
```

So that `resume` does not bring a stone, a move, a mark, an arrow, a line or a territory, it is drawn with the option `forget: true` (`stones(white: "C3", forget: true)`, `marks(circle: "A1", forget: true)`, ...); with `forget: "number"`, a move comes without its number even if `numbers` is asked for (p. 60). What is forgotten is still on the board: `goban-view` shows it with `forgotten`.

Since a board is a value, it can be saved in a module and imported in another document: `#import "boards.typ": ko`. That way the same board is used in an article and in its slides (p. 25).

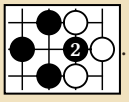
2 Views in the text

`goban-view(board, part, ..options)`

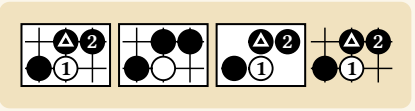
Draws the intersection (`"C3"`) or the rectangle (`"B2:E4"`, written as in `partial`) of a board (a `board(...)` or a game), as it was at the end, with the captures resolved, in the middle of the text. By default, everything the board drew in that part is seen (stones, numbers of the moves, marks, arrows, lines and territory), with its style, at the scale of `view.goban` and centered at the height of the lowercase letters. The options are those of the `view` branch.

```
#let board1 = board(size: 5,
  captures: "remove",
  stones(black: "B3, C2, C4",
    white: "D2, D4, E3"),
  moves(white: "C3, D3"))

The black stone at D3 #goban-view(board1, "D3")
captures the white one at C3
#goban-view(board1, "C3"). This is the ko:
#goban-view(board1, "B2:E4").
```

The black stone at D3  captures the white one at C3  . This is the ko:  .

The view is resolved with the global options of the place where it is, like a `goban`; on top go those of the board and, after them, those of `view.goban`. Since the board is a value, the view can go before or after the diagram that draws it. A view does not change what `goban-prisoners`, `goban-score`, `goban-notes` or `goban-region` say.



```
#let board2 = board(size: 5,
  stones(black: "B2, C3"),
  moves(white: "C2, D3"),
  marks(triangle: "C3"))

#goban-view(board2, "B2:D3")
#goban-view(board2, "B2:D3",
  marks: false, numbers: false)
#goban-view(board2, "B2:D3",
  goban: (grid: false))
#goban-view(board2, "B2:D3",
  goban: (background: false))
```

`view` a dictionary

The options of the views: in `goban-view`, for that view, and with `gobanist-set(view: ...)`, for all of them.

`view.marks` `true` or `false` initially `true`

With `false`, without the marks.

`view.arrows` `true` or `false` initially `true`

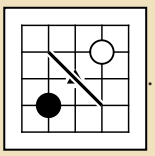
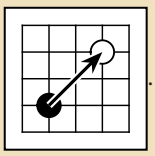
With `false`, without the arrows.

`view.lines` `true` or `false` initially `true`

With `false`, without the lines.

```
#let arrowed = board(size: 5,
  stones(black: "B2", white: "D4"),
  arrows(("B2", "D4")), lines(("B4", "D2")),
  marks(triangle: "C3"))

Without arrows:
#goban-view(arrowed, "A1:E5", arrows: false).
Without lines or marks:
#goban-view(arrowed, "A1:E5", lines: false,
  marks: false).
```

Without arrows:  . Without lines or marks:  .

`view.numbers` `true` or `false` initially `true`

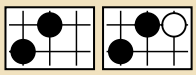
With `false`, without the numbers of the moves.

`view.territory` `true` or `false` initially `true`

With `false`, without the territory.

`view.forgotten` `true` or `false` initially `false`

Also draws what has `forget`.



```
#let board3 = board(size: 5,
  stones(black: "B2, C3"),
  stones(white: "D3", forget: true))

#goban-view(board3, "B2:D3")
#goban-view(board3, "B2:D3",
  forgotten: true)
```

`view.raise` `auto` or a length — initially `auto`

How high above the baseline of the text the center of the view lies. With `auto`, half the height of the lowercase letters.

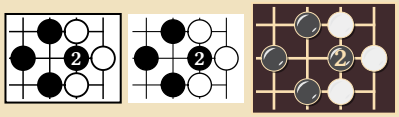
```
#let b = board(size: 5, stones(black: "C3"))

Text #goban-view(b, "C3", raise: 0pt)
text #goban-view(b, "C3")
text #goban-view(b, "C3", raise: 0.8em).
```

Text  text  text  .

`view.goban` board options — initially `(scale: 0.8)`

The options of the board of the view, on top of its own: the style (`profile`, `players`, `stone`, `mark`, `grid`, `background`, `label`, ...) and the scale. They cannot be those that change the game or the visible part: `size`, `use-i`, `partial`, `captures`, `captured-marks`, `territory` or `sgf`.



```
#let ko = board(size: 5, captures: "remove",
  stones(black: "B3, C2, C4",
    white: "D2, D4, E3"),
  moves(white: "C3, D3"))

#goban-view(ko, "B2:E4")
#goban-view(ko, "B2:E4",
  goban: (profile: "simple"))
#goban-view(ko, "B2:E4",
  goban: (profile: "wain", scale: 1))
```

10 Games in SGF

SGF is the format used by almost every Go program and server to save games. gobanist reads them with the kernel (p. 31): a game on a board, its data and comments, the whole game in diagrams and problems with their solutions.

In the examples, `game` is `game.sgf`, a short 9×9 game:

```
(;GM[1]FF[4]SZ[9]PB[Black]PW[White]C[A short game.]
;B[ee];W[ef];B[df];W[ce];B[ff];W[gd];B[eg]TE[1]C[Black captures.];W[dd];B[cf];W[fd])
```

and `variations` is `variations.sgf`, a 7×7 game with two branchings, marks and `MN[10]` on the fourth move of the main line:

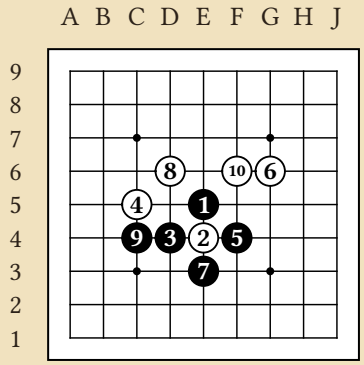
```
(;GM[1]FF[4]SZ[7]PB[Ann]PW[Bob]RE[B+R]KM[6.5]DT[2026-10-13]
AB[dd]
;B[cc]TR[dd]
(;W[ee]SQ[cc]CR[ee]MA[aa]LB[gg:A][ag:B]
 ;B[ec]
 (;W[ce]MN[10]
  ;B[cf])
 (;W[fe]))
(;W[ce]
 ;B[ec]TR[ec]))
```

1 Reading a game

```
sgf-read(source, game: 1, charset: auto)
```

Reads an SGF game. `source` is a file (`path("game.sgf")`, relative to the document that names it), its content (`bytes`) or the SGF text (a string). It returns the game as a value, which is saved with `let` and used everywhere: on a board, in `sgf-info`, `sgf-comment`, `sgf-diagrams`, `sgf-problem`, `goban-layout` and `goban-view`.

On a board, the game is an element, like `stones` or `moves`: it draws the main line, and the board takes the size of its `SZ` (another `size` is an error).




```
#let game = sgf-read(path("game.sgf"))

#goban(label: (at: (top, left)), game)
```

`game` a number — initially `1`

In `sgf-read`: an SGF file can hold several games (a collection); `game` chooses which one is read. One that does not exist is an error.

```
#let third = sgf-read(path("collection.sgf"), game: 3)
```

`charset` `auto` or a name — initially `auto`

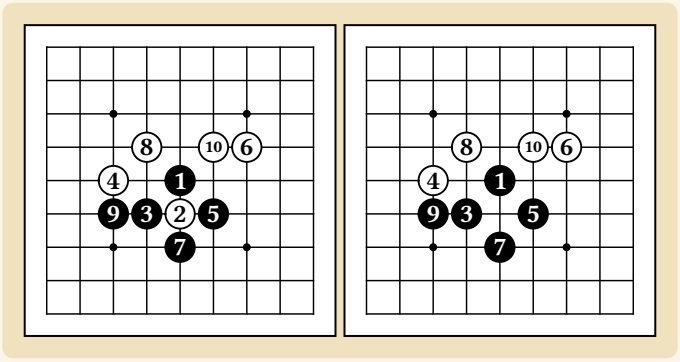
In `sgf-read`: the encoding of the file, if it is not UTF-8. With `auto`, the one the file gives with `CA` or, if it gives none, Latin-1 (the SGF default). Latin-1 (and windows-1252), GBK (GB2312, GB18030), Big5, Shift_JIS, EUC-JP and EUC-KR are read; the names ignore case and punctuation (`"shift-jis"` and `"EUC_KR"` also work). A file that is valid UTF-8 is always read as UTF-8, even if `CA` says otherwise; one in UTF-16 (with the byte order mark at the start) is read too. Another encoding is an error: the file has to be converted to UTF-8.

Many files from Asian servers do not state their encoding: `sgf-read(path("qipu.sgf"), charset: "GBK")`. To write their comments, a font with those characters is needed.

2 A game on a board

A game on a board draws the initial stones (`AB` and `AW`), as with `stones`, and the moves of the main line, as with `moves`. Passes count for the numbering, but are not drawn (with `notes`, they are noted: “N: pass”, p. 73). The stones added (`AB`, `AW`) or removed (`AE`) by the nodes after the first move are applied together with the previous move; removing stones needs `captures`, and without `captures` it is an error. At the end, the numbering and the turn are ready to go on with the game with `moves`. The game also reads `PL` (who plays next) and `MN` (the number of a move).

```
// Without captures, 2 is not removed.
#goban(game)
#goban(captures: "remove", game)
```

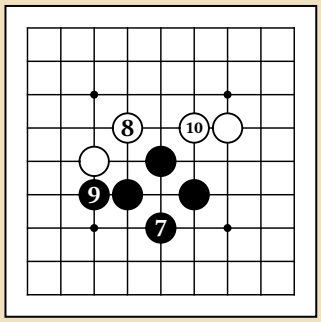


`sgf(game, ..options)`

The same game with other options, those of the `sgf` branch, which are added to those it already had: `sgf(game, to: 7)`. Also, `forget` (p. 80). With `gobanist-set(sgf: ...)`, the options hold for every game.

`sgf.from` a number — initially `1`

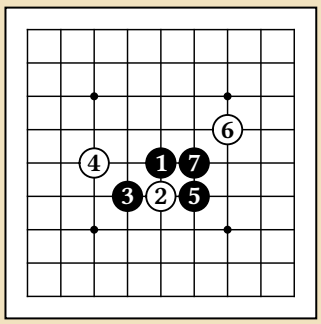
The first numbered move. The previous ones are drawn without a number.




```
#goban(captures: "remove",
  sgf(game, from: 7))
```

`sgf.to` `auto` or a number — initially `auto`

The last move played; the following ones are not drawn. With `auto`, up to the last one.



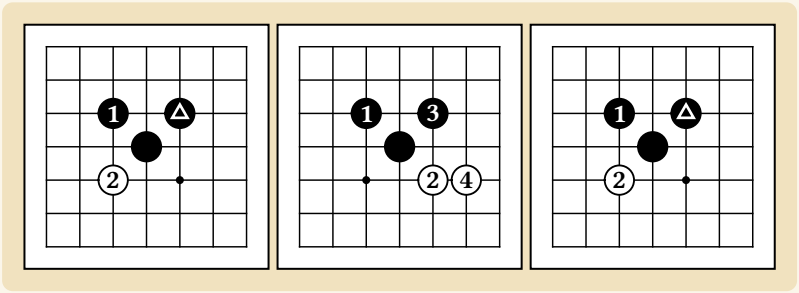

```
#goban(captures: "remove",
  sgf(game, to: 6),
  // Carries on with 7 (Black), elsewhere.
  moves("F5"))
```

`from` and `to` count the moves of the file. The number shown can be another: `MN` changes it (the following moves carry on from there), and so does `restart`.

`sgf.variation` `none`, numbers or moves — initially `none`

The line that is drawn. With `none`, the main one. With numbers, at each branching the indicated variation is taken, in order (1 is the main one): `(2, 1)` takes the second at the first branching and the first at the second. With a text, the moves of the line from the first one, separated by commas or spaces: `"R16, P17, Q15"`; at each branching, the variation that plays the next one is taken, and a pass is written `pass`. In both cases, when the list runs out, the first variation is followed. A variation or a move that does not exist is an error.

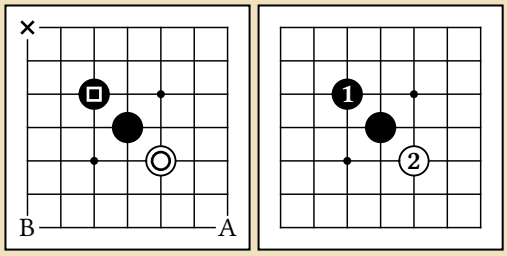
```
#goban(sgf(variations, variation: (2,)))
#goban(sgf(variations, variation: (1, 2)))
#goban(sgf(variations, variation: "C5, C3"))
```



`sgf.marks` `true` or `false` — initially `true`

Draws the marks of the file (`TR`, `SQ`, `CR`, `MA`, `SL` and the labels of `LB`), the arrows (`AR`) and the lines (`LN`) of the last node shown, and dims the points of `DD` (which stay dimmed until the next `DD`). Each mark takes the player of the stone on that point, and replaces the number of the move; labels do not cover

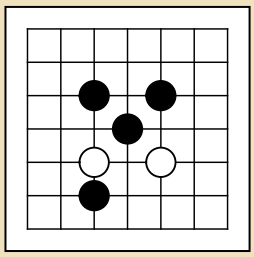
a number, so they are not drawn on a numbered stone. The territory of the file (`TB` and `TW`) is drawn like that of `territory` (p. 73) and counts for `goban-score`.




```
#goban(sgf(variations, to: 2))
#goban(sgf(variations, to: 2, marks: false))
```

`sgf.numbers` `auto`, `true` or `false` — initially `auto`

With `false`, the moves are drawn without a number. With `auto`, what the file asks for with `PM`: with `PM[0]`, they are not numbered.



```
#goban(sgf(variations, numbers: false))
```

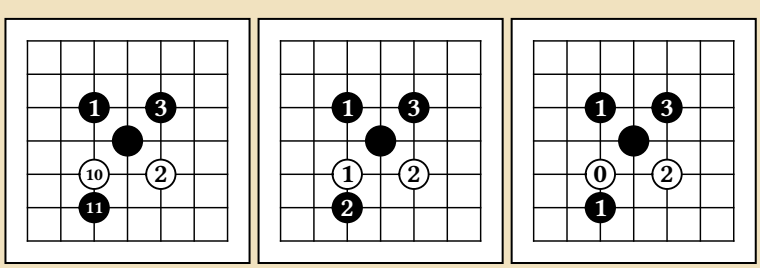
`sgf.step` a number or an array — initially `100`

The size of the stretches in which `restart` restarts the numbering. With an array, each number is the size of a stretch, and the last one is repeated: with `(50, 100)`, the first stretch has 50 moves and the others 100.

`sgf.restart` `auto`, `true`, `false` or a dictionary — initially `auto`

From the second stretch of `step` moves on, each one numbers from 1 (or from `restart.from`), as in books: with `step: 100, restart: true`, move 101 is shown as 1. The first stretch keeps the numbers of the game. With `false`, the moves carry the numbers of the game (with `MN`). With `auto`, what the file asks for with `PM`: with `PM[2]`, the numbering restarts every 100 moves, regardless of `step`.

```
#goban(variations)
#goban(sgf(variations, step: 3,
  restart: true))
#goban(sgf(variations, step: 3,
  restart: (on: true, from: 0)))
```

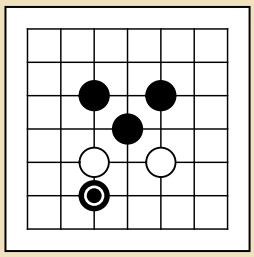


`sgf.restart.from` a number — initially `1`

The number of the first move of each stretch, from the second on. With `step: 99` and `restart: (on: true, from: 0)`, move 100 is shown as 0, move 101 as 1, and so on.

`sgf.last-mark` `none`, a shape or a dictionary — initially `none`

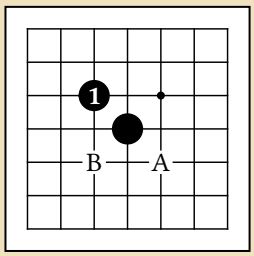
Marks the last move shown, with a shape (`"circle"`) or with the options of a mark (`(shape: "triangle", line: (color: red))`). A pass is not marked, nor is a point the file already marks or a move with a number (the number already tells which is the last one): in practice, it serves with `numbers: false`.



```
#goban(sgf(variations, numbers: false,
  last-mark: "circle"))
```

`sgf.letters` `none`, `"children"`, `"siblings"` or `auto` — initially `none`

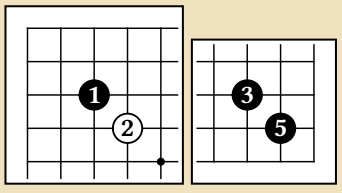
Marks with letters the variations of the last move shown: with `"children"`, the moves that can follow; with `"siblings"`, the alternatives to that move (the letter replaces its number). With `auto`, what the file asks for with `ST`. They are marked only if there is more than one.




```
#goban(sgf(variations, to: 1, marks: false,
  letters: "children"))
```

`sgf.view` `false`, `true` or a number — initially `false`

If the file has a view (`VW`), shows only that part of the board, like `partial`. The view can change during the game: a `VW` holds from the move of its node (or from the next one, if the node has no move) until the next `VW`, and `VW[]` shows the whole board again. With `true`, the view that covers those of moves `from` to `to`; with a number, that of that move (with 0, that of the start). A `partial` of the board wins over it.




```
// views: VW[aa:ee] at the start, VW[] at
// move 3 and VW[ff:ii] at move 5.
#goban(sgf(views, to: 2, view: true))
#goban(sgf(views, view: 5))
```

3 Data and comments

```
sgf-info(game, property, move: 0, variation: none)
```

A property of the root of the game, as text: `"PB"` and `"PW"` (the players), `"RE"` (the result), `"KM"` (the komi), `"DT"` (the date), etc. With `move`, that of that move: that of its node and of the nodes without a move that follow it (with 0, that of the root and of the nodes before the first move), in the line of `variation` (like `sgf.variation`). If it is not there, `""`. If it has several values, they are separated by commas.

```
#sgf-info(variations, "PB") (Black) against
#sgf-info(variations, "PW") (White),
komi #sgf-info(variations, "KM").
Result: #sgf-info(variations, "RE").
```

Ann (Black) against Bob (White), komi 6.5. Result: B+R.

The value is that of the file, with the escaped characters already resolved and in the encoding of `CA`: `"RE"` stays as `B+R`, not in words (`sgf-diagrams` writes it in words). Any property works. Those of the game data, according to the SGF format, are:

Property	What it is	Example
`PB`, `PW`	The name of Black and that of White.	`Lee Sedol`
`BR`, `WR`	The rank of Black and that of White.	`9p`
`BT`, `WT`	The team of Black and that of White.	
`RE`	The result: `B+` or `W+` and the points, `R` (resignation), `T` (time) or `F` (forfeit); `0` or `Draw`, a draw; `Void`, no result; `?`, unknown.	`W+3.5`
`KM`	The komi.	`6.5`
`HA`	The handicap: how many stones.	`2`
`DT`	The date, `YYYY-MM-DD` (or several, separated by commas).	`2016-03-09`
`EV`	The tournament or event.	`Meijin`
`RO`	The round.	`3 (final)`
`PC`	The place.	`Tokyo`
`GN`	The name of the game.	
`GC`	A comment about the game.	
`ON`	The opening (fuseki).	
`RU`	The rules.	`Japanese`
`TM`	The time of each player, in seconds.	`7200`
`OT`	The overtime (byo-yomi).	`5x30 byo-yomi`
`SO`	The source: a book, a magazine.	
`AN`	Who commented on the game.	
`US`	Who wrote the file.	
`CP`	The copyright.	
`AP`	The program that wrote the file.	`CGoban:3`
`SZ`	The size of the board (`19`, or `19:13`).	`19`
`CA`	The encoding of the file.	`UTF-8`

```
sgf-comment(game, move, variation: none, annotations: true, chat: true)
```

The comment of a move (0: the one before the first move), in the line of `variation`: the name of the node (`N`), in bold; the annotations of the move (`TE`, `BM`, `DO`, `IT`) and of the position (`GB`, `GW`, `DM`, `UC`), with the texts of the language (p. 31); and the comment (`C`). Values (`V`) and hotspots (`HO`) are not written. `sgf-diagrams` and `sgf-problem` write the comments the same way.

```
Move 7: #sgf-comment(game, 7)

#sgf-comment(problem, 1, variation: (2,))
```

Move 7: Tesuji. Black captures.

Bad move. Wrong: White lives.

`annotations` `true` or `false` initially `true`

In `sgf-comment`: with `false`, only the comment, without the name or the annotations.

```
With: #sgf-comment(game, 7) \
Without: #sgf-comment(game, 7, annotations: false)
```

With: Tesuji. Black captures.
Without: Black captures.

`chat` `true` or `false` initially `true`

In `sgf-comment`: with `false`, removes the chat of the servers from the comment: on OGS, everything after the line "`-- chat --`"; on KGS, the lines "`name [rank]: ...`" and, if there is any, also the lines "`name: ...`" of the same comment. If nothing is left, the move has no comment.

4 Diagrams as in books

```
sgf-diagrams(game, ..options)
```

Draws the game as in books: a diagram every `step` moves (1–50, 51–100, ...). Each diagram starts from the real position, with the previous moves without numbers and their captures; the stones captured by the moves of the diagram are not removed (they are removed in the next one). Below each one go its caption and its "N at M" notes (p. 73), and below the last one, the result. A move on an occupied point removes the stone that was there, without prisoners, to draw games that are not legal. It goes outside `goban`.

The options are those of the `diagrams` branch; with `gobanist-set(diagrams: ...)`, they hold for the whole document. Those of `sgf(game, ...)` hold on each board, except those set by each diagram (`from`, `to`, `variation`, `numbers`, `marks`, `last-mark`, `step`, `restart` and `view`), which are an error.

```
#sgf-diagrams(game, step: 5)
```

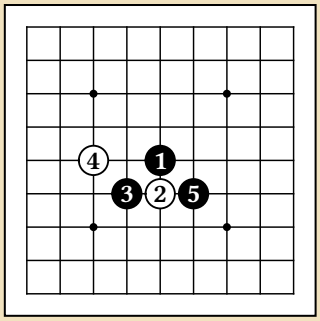


Diagram 1 (1–5)

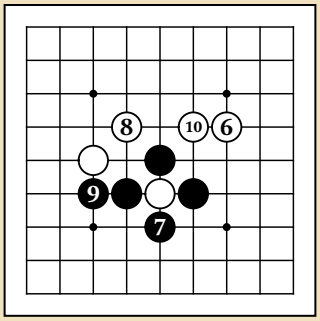


Diagram 2 (6–10)

`diagrams.step` a number or an array initially `50`

The number of moves of each diagram. With an array, the first diagram has as many moves as the first number says, the second as the second says, and so on; the last one is repeated: with `(5, 10, 3)`, the diagrams have 5, 10, 3, 3, 3, ... moves.

`diagrams.from` a number initially `1`

The first move. The diagrams are the same as without `from`, with the same number: only those with moves from `from` on are drawn, and the first one starts at `from`. With `step: 50` and `from: 120`, the first one is "Diagram 3 (120–150)".

`diagrams.to` `auto` or a number initially `auto`

The last move. With `auto`, the last one of the game.

`diagrams.numbers` `true` or `false` — initially `true`

With `false`, the moves of each diagram are drawn without a number (the "N at M" notes still use the numbers of the game).

`diagrams.last-mark` `none`, a shape or a dictionary — initially `none`

Like `sgf.last-mark`, in each diagram.

```
#sgf-diagrams(game, step: 5, to: 10,
  numbers: false, last-mark: "circle")
```

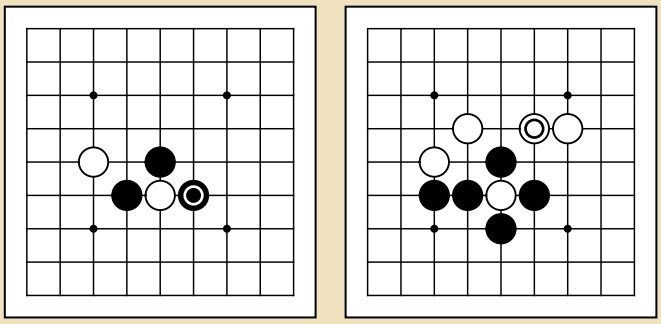


Diagram 1 (1–5) Diagram 2 (6–10)

`diagrams.restart` `auto`, `true`, `false` or a dictionary — initially `auto`

Like `sgf.restart`, with one diagram per stretch: from the second diagram on, each one numbers from 1 (or from `restart.from`), as some books do. The first diagram keeps the numbers of the game. With `auto`, what `PM[2]` asks for: every 100 moves, regardless of the diagrams.

```
#sgf-diagrams(game, step: (2, 3), to: 8,
  restart: (on: true, from: 0))
```

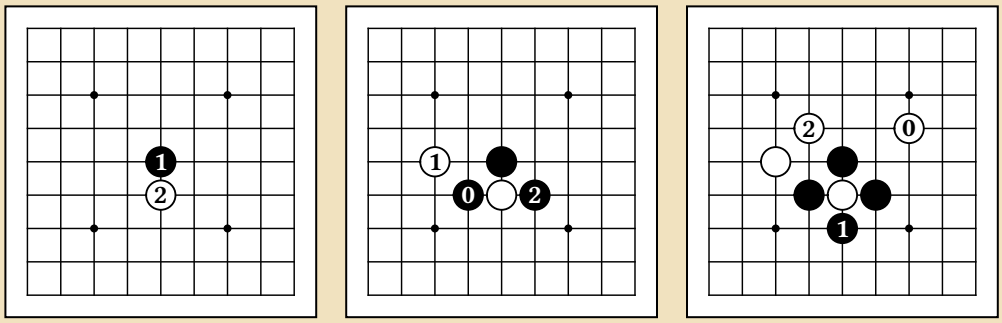


Diagram 1 (1–2) Diagram 2 (3–5) Diagram 3 (6–8)

`diagrams.restart.from` a number — initially `1`

The number of the first move of each diagram, from the second on.

`diagrams.marks` `true` or `false` — initially `true`

Like `sgf.marks`: each diagram shows the marks, arrows and lines of its last move.

`diagrams.notes` `true` or `false` — initially `true`

Writes below each diagram its notes: "N at M" and the passes, "N: pass".

`diagrams.title` `auto` or a text — initially `auto`

The word of the caption: "Diagram 2 (51–100)". With `auto`, that of the language (`diagram` in `texts`).

`diagrams.caption` a dictionary

The caption, the notes and the result.

`diagrams.caption.text` a dictionary initially `(size: 0.9em)`

The `text` options of the caption, the notes and the result.

```
#sgf-diagrams(game, from: 6, title: "Figure",
  caption: (text: (style: "italic")),
  goban: (label: (at: (top, left))))
```

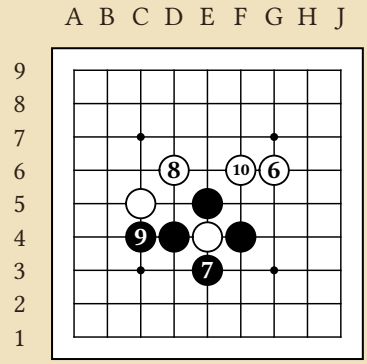


Figure 1 (6–10)

`diagrams.caption.min-width` a length initially `8em`

Outside the grid, a caption wider than its board is broken into the lines it needs at the width of the board or, if the board is narrower, at this measure; then it is narrowed as far as those lines still fit, so that they are even. It is never narrower than its longest word. In the grid, the caption is broken at the width of the cell.

`diagrams.goban` board options initially `(:)`

The options of each board, on top of the global ones. The boards start with `captures: "remove"` and `notes: true`, and have the size of the game.

`diagrams.layout` layout options initially `(:)`

How the diagrams are laid out: the options of `layout` (p. 101), only for these diagrams: `layout: (columns: 3)`.

`diagrams.variation` `none`, numbers or moves initially `none`

The line that is drawn, like `sgf.variation`.

`diagrams.variations` `true`, `false` or a dictionary

Also draws the variations of the game, each one in its diagram, after the diagram where it is born: “Diagram 1a”, “Diagram 1b”, … Each variation goes whole, with its moves numbered from 1; the caption says which move it replaces (`variation-title`), and the previous moves are seen without numbers. Its comments and notes are written like those of the other diagrams. The variations are those that depart from the line of `variation` (with `variation`, the main one too).

```
#sgf-diagrams(variations, step: 2, to: 3,
  marks: false,
  variations: (on: true, select: "all"),
  goban: (scale: 0.7))
```

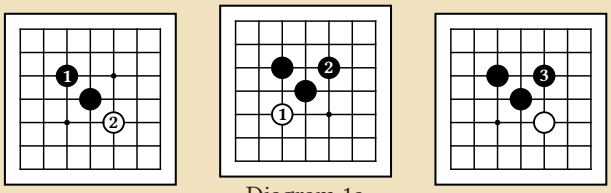


Diagram 1 (1–2) Diagram 1a (variation at 2) Diagram 2 (3–3)

`diagrams.variations.select` `"commented"` or `"all"` — initially `"commented"`

Which ones are drawn: those with some comment (`C`) on their own moves, or all of them.

`diagrams.variations.depth` a number — initially `1`

Down to which depth: with 1, those that depart from the line drawn; with 2, also the variations of those, and so on. Those of a variation go after it, and their caption says which of its moves they replace.

`diagrams.variations.from` a number — initially `1`

Only the variations born at that move or later.

`diagrams.variations.paths` an array of paths — initially `()`

Variations that are always drawn, even if `variations` is off, at any depth: each path is like `sgf.variation`, with numbers (`(2,)`, `(2, 3, 1)`) or with moves (`"R16 P17 Q15"`). Trailing 1s do not change the line: `(2, 3, 1)` is `(2, 3)`. A path the game does not have, or that does not depart from the line, is an error.

```
#sgf-diagrams(josekis, variations: (paths: ((2,), "Q16 R14 O16")))
```

`diagrams.variation-title` `auto` or a function — initially `auto`

What goes in parentheses in the caption of a variation: “Diagram 3a (variation at 26)”. A function takes the number of the move it replaces: `variation-title: n => [instead of #n]`. With `auto`, the text of the language (`variation-at` in `texts`).

`diagrams.figures` `true` or `false` — initially `true`

If the file marks figures (`FG`), each figure is a diagram, with its name in the caption, instead of one every `step` moves.

`diagrams.view` `true` or `false` — initially `true`

If the file has a view (`VW`), each diagram shows only the one that covers the views of its moves, like `sgf.view`; a variation, that of its line. A `partial` in `goban` wins over it.

`diagrams.partial` `none` or `auto` — initially `none`

With `auto`, where there is no view in the file, every diagram shows the same part of the board: the one used by the lines drawn (that of `variation` and those of `variations`), like `partial: auto`.

```
#sgf-diagrams(josekis, partial: auto, variations: true)
```

`diagrams.comments` `true`, `false` or a dictionary

Writes the comments of the moves of each diagram, like `sgf-comment`, each one with the number of the move.

```
#sgf-diagrams(game, step: 5,
  comments: (on: true, root: true),
  layout: (columns: 2))
```

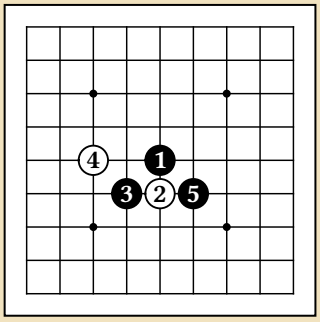


Diagram 1 (1–5)

A short game.

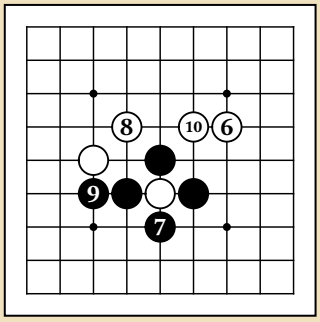


Diagram 2 (6–10)

7: Tesuji. Black captures.

`diagrams.comments.text` a dictionary — initially `(:)`

The `text` options of the comments, on top of those of `caption.text`.

`diagrams.comments.align` `left`, `center` or `right` — initially `left`

The alignment of the comments in the width of the diagram.

`diagrams.comments.justify` `true` or `false` — initially `false`

Justifies the comments. In a narrow diagram, large spaces are left between the words.

`diagrams.comments.root` `true` or `false` — initially `false`

Also writes the comment of the root (that of the nodes before the first move), without a number, before those of the diagram that starts at move 1.

`diagrams.comments.number` a function — initially `n => [#strong(str(n)): ]`

What goes in front of each comment: a function of the number of the move, the one seen on the board (with `restart`, the restarted one). For example, `n => [Move #n. ]`.

`diagrams.comments.separator` content — initially `parbreak()`

What goes between the comments. With the initial value, each one goes in its own paragraph; with `h(1em)`, they run on.

`diagrams.comments.position` `bottom`, `left`, `right` or `"after"` — initially `bottom`

Where the comments go: below the caption; beside the diagram, as in books of commented games; or, with `"after"`, all together after the diagrams, across the full width, each group with the caption of its diagram as a heading. Beside the diagram, they start at the height of the top border of the board; the caption, the notes and the result stay below the board.

```
#sgf-diagrams(game, step: 5,
  comments: (on: true, position: right))
```

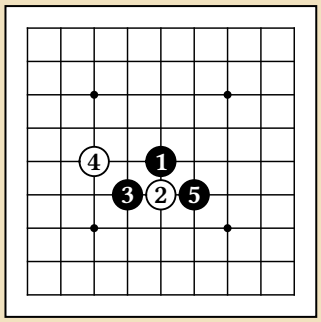


Diagram 1 (1–5)

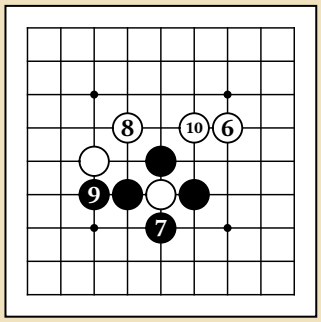


Diagram 2 (6–10)

7: Tesuji. Black captures.

`diagrams.comments.width` `auto` or a length — initially `auto`

The width of the comments beside the diagram. With `auto`, what is left: outside the grid, the rest of the line; in the grid, the rest of the cell, and with `fit`, the board takes half of the cell and the comments the other half.

`diagrams.comments.sep` a length — initially `1em`

The space between the diagram and the comments beside it.

`diagrams.comments.height` `none` or a length — initially `none`

The maximum height of the comments of a diagram, below or beside it. What does not fit (it is broken between words) goes after the diagrams, as with `"after"`, with the caption of the diagram as a heading.

```
#sgf-diagrams(lee, step: 30, layout: (columns: 2),
  comments: (on: true, height: 6em))
```

`diagrams.annotations` `true` or `false` — initially `true`

With the comments, also writes the name and the annotations of each move.

`diagrams.chat` `true` or `false` — initially `true`

With the comments, like `chat` of `sgf-comment`: with `false`, without the chat of KGS and OGS. A move that had only chat left does not appear.

`diagrams.result` `true`, `false` or a dictionary

Writes below the last diagram, if it reaches the end of the game, the result (`RE`) and, if the file marks the territory (`TB`, `TW`), the score of each player: “Result: Black wins by 0.5 points (Black 6, White 5.5)”. The score is counted as with `goban-score`, with the komi of the file (`KM`).

`diagrams.result.title` `auto` or a text — initially `auto`

The word before the result. With `auto`, that of the language (`result` in `texts`).

`diagrams.result.format` `"text"` or `"raw"` — initially `"text"`

How the result is written: with `"text"`, in words, in the language of the text (“Black wins by resignation”); with `"raw"`, as it is in the file (“B+R”). With `"text"`, `0` or `Draw` (a draw), `Void` (no result), `?` (unknown) and `B+` or `W+` followed by nothing, by the difference, by `R` or `Resign`, by `T` or `Time`, or by `F` or `Forfeit` are understood. Another result is written as it is.

`diagrams.result.black` `auto` or a text — initially `auto`

The name of Black in the score and in the result. With `auto`, that of the language.

`diagrams.result.white` `auto` or a text — initially `auto`

The name of White in the score and in the result.

```
#sgf-diagrams(variations,
  result: (title: "Final", format: "raw"))
#sgf-diagrams(variations,
  result: (black: "Ann", white: "Bob"))
```

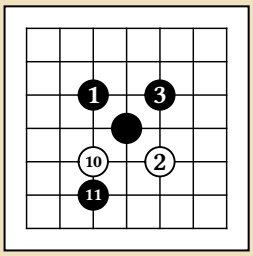


Diagram 1 (1–5)
Final: B+R

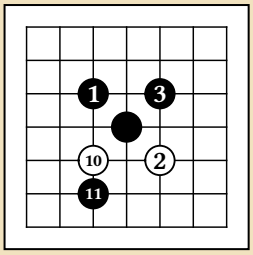


Diagram 1 (1–5)
Result: Ann wins by resignation

5 Problems

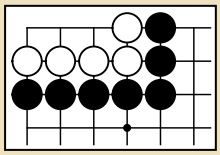
`sgf-problem(..problems, ..options)`

Draws a problem (tsumego). First, the statement: the initial position with its marks, who plays (`PL`, or the color of the first move) and the comment of the root. Then, a diagram for each line of the tree of the file (each variation of the solution, right or wrong), numbered from 1, with its “N at M” notes and the comments and annotations of its moves, which usually say whether the variation is right. Every diagram shows the same part of the board: the view of the file (`VW`) or, without a view, the one the problem uses. It goes outside `goban`.

With several games (`sgf-problem(p1, p2, p3)`), all the statements go first, each one with its number (that of the game in its file), and then the solutions of those that have them. That is how a problem sheet is made.

The options are those of the `problems` branch; with `gobanist-set(problems: ...)`, they hold for the whole document. Those of `sgf(problem, ...)` hold on each board, except those set by the diagram (`from`, `to`, `variation`, `numbers`, `last-mark`, `step`, `restart` and `view`). In the examples, `problem` is `problem.sgf`, a life-and-death problem: the right move and two wrong ones, with their comments.

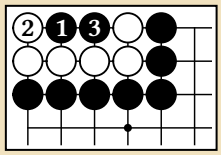
```
#sgf-problem(problem, title: "Problem 1")
```



Problem 1:

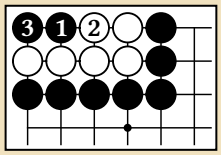
Black to play

Black to kill.



Variation 1

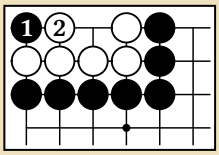
1: Tesuji. Correct: the vital point.



Variation 2

1: Tesuji. Correct: the vital point.

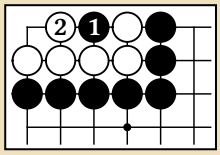
3: White has only one eye.



Variation 3

1: Bad move.

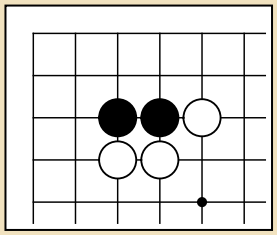
Wrong: White lives.



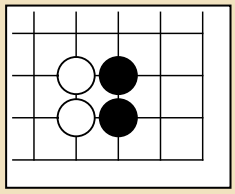
Variation 4

1: Wrong.

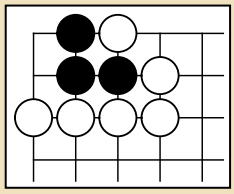
```
// problems: the three games of problems.sgf.
#sgf-problem(..problems, comments: false,
  layout: (columns: 4, fit: true))
```



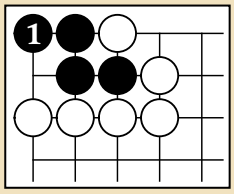
Problem 1



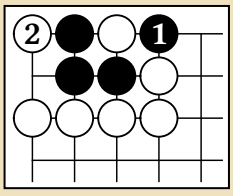
Problem 2: White to play



Problem 3: Black to play



Problem 3: Variation 1



Problem 3: Variation 2

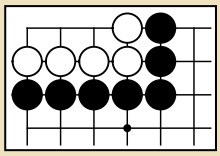
`problems.statement` `true`, `false` or `"alone"` initially `true`

Draws the statement. With `"alone"`, the statement goes alone in its row, centered, and the variations below.

`problems.solutions` `true` or `false` initially `true`

Draws the variations. With `false`, only the statement remains; the solutions go elsewhere, with `statement: false`.

```
#sgf-problem(problem, solutions: false,
  title: "Problem 2")
```



Problem 2:

Black to play

Black to kill.

`problems.title` `auto` or a text initially `auto`

The start of the caption of the statement: “Problem 1: Black to play”. With several games, it is followed by the number of each one; with `auto`, nothing, or with several games, the word of the language (`problem` in `texts`): “Problem 7: Black to play”. The solutions also say which problem they belong to: “Problem 7: Variation 1”.

`problems.to-play` `auto`, `"black"`, `"white"` or `none` initially `auto`

Who plays, for the caption of the statement. With `auto`, what `PL` says or, if it is not there, the color of the first move; if neither is there, it is not said. Many collections do not say it in the file: `to-play: "black"`. With `none`, the caption does not say it.

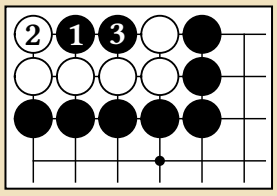
`problems.variation-title` `auto` or a function initially `auto`

The caption of each solution: a function of its number, `n => [Solution #n]`. With `auto`, “Variation 2”, with the word of the language (`variation` in `texts`).

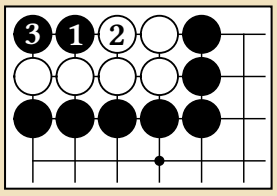
`problems.verdict` `true` or `false` initially `false`

The caption of each solution says whether it is right or wrong (“Variation 2 (wrong)”), with the texts `correct` and `wrong` of the language. It is decided with the annotations of the line: the last `GB` or `GW` (the position is good for Black or for White), according to who plays; if there is none, the first move with `TE` or `BM` (the odd moves belong to the player to play: a `BM` of theirs is a mistake, one of the opponent’s is not); if not, the word `RIGHT` in a comment, as in the files of goproblems.com. If there is none of that, it says nothing.

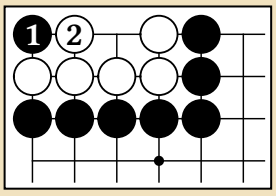
```
#sgf-problem(problem, verdict: true,
  statement: false, comments: false,
  layout: (columns: 4, fit: true))
```



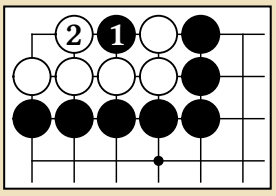
Variation 1 (correct)



Variation 2 (correct)



Variation 3 (wrong)



Variation 4

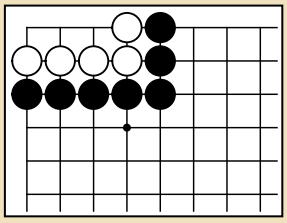
`problems.view` `true` or `false` initially `true`

If the file has a view (`VW`) at the start, every diagram shows that part.

`problems.partial` `auto` or `none` initially `auto`

With `auto`, if the view of the file is not used, every diagram shows the part of the board where there are stones, moves, marks, arrows or lines in some line of the problem, like `partial: auto`. The lines added around are changed with `goban: (partial-margin: ...)`. With `none`, the whole board is seen.

```
#sgf-problem(problem, solutions: false,
  goban: (partial-margin: 3))
```



Black to play

Black to kill.

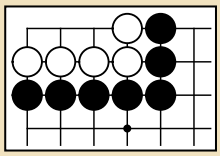
`problems.notes` `true` or `false` initially `true`

Writes the "N at M" notes of each variation.

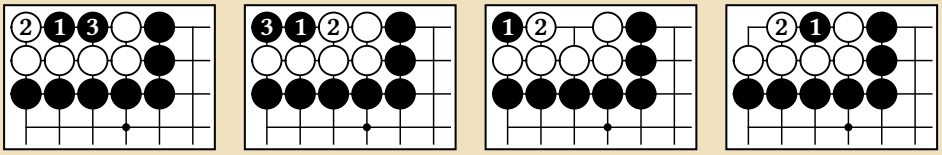
`problems.caption` a dictionary initially `(text: (size: 0.9em), min-width: 8em)`

The captions, the notes and the comments, like `diagrams.caption`.

```
#sgf-problem(problem, statement: "alone",
  comments: false, notes: false,
  caption: (text: (style: "italic")))
```



Black to play



Variation 1 *Variation 2* *Variation 3* *Variation 4*

`problems.comments` `true`, `false` or a dictionary

The comment of the root below the statement and those of each variation below its diagram, like `diagrams.comments` (without `root`). Here they are on at the start. `text`, `align` and `justify` also hold for the comment of the statement.

`problems.annotations` `true` or `false` initially `true`

With the comments, also writes the name and the annotations of each move (`TE`, `BM`, ...).

`problems.chat` `true` or `false` initially `true`

With the comments, like `chat` of `sgf-comment`.

`problems.goban` board options initially `(:)`

The options of each board, like `diagrams.goban`.

`problems.layout` layout options initially `(:)`

How the statement and the variations are laid out: the options of `layout` (p. 101), only for these diagrams.

11 Layout of the diagrams

The diagrams of `sgf-diagrams` and `sgf-problem` are laid out according to the `layout` branch, and `goban-layout` lays out single boards and those diagrams together. Without a grid (`columns: auto`), the diagrams go in the paragraph, side by side, and move to the next line when they do not fit. With a number of columns, they go in a grid of columns of the same width: each row is a separate line (the page can break between rows), the boards of a row are aligned at the top and an incomplete row leaves its last columns empty. With `fit`, the boards are also scaled to fill the width of their column.

```
goban-layout(..boards, ..options)
```

Lays out, in the order in which they come, boards made with `board(...)`, games (`sgf-read`, `sgf`) and the diagrams of `sgf-diagrams(...)` and `sgf-problem(...)`. The options are those of the `layout` branch; the `layout` options of the `sgf-diagrams` and `sgf-problem` inside are not used. With `fit`, each single board fills its column, and the diagrams of an `sgf-diagrams` or an `sgf-problem` use the scale of the first one. A `goban(...)` is already drawn, so inside it is an error: `board(...)` goes there.

```
#goban-layout(columns: 4, fit: true,
  board(size: 5, stones(black: "C3")),
  board(size: 5, stones(white: "C3")),
  sgf-diagrams(game, step: 5))
```

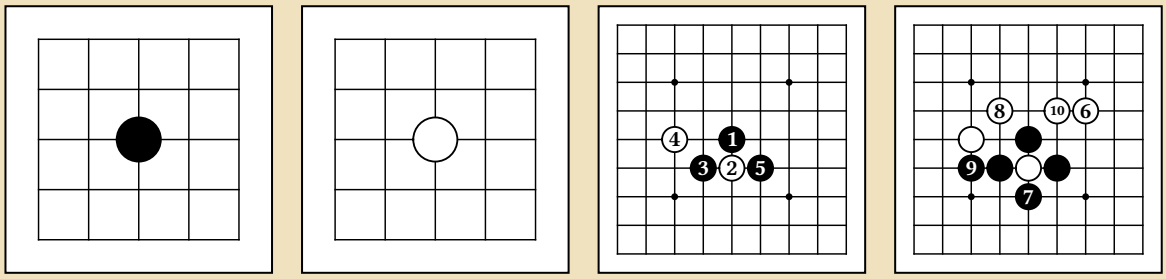


Diagram 1 (1–5) Diagram 2 (6–10)

```
#sgf-diagrams(game, step: 2, to: 6,
  layout: (columns: 3, fit: true))
```

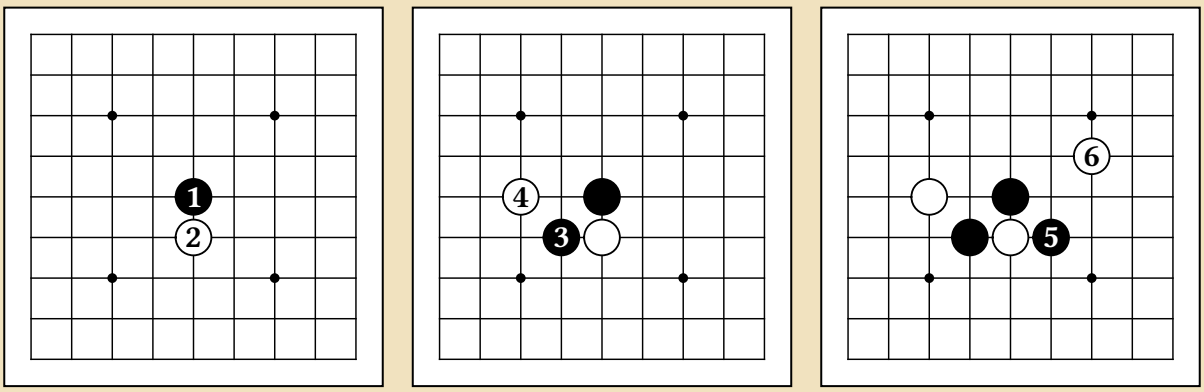


Diagram 1 (1–2) Diagram 2 (3–4) Diagram 3 (5–6)

`layout` a dictionary

The options of the layout: with `gobanist-set(layout: ...)`, for the whole document; in `goban-layout`, for that layout; with `diagrams.layout` and `problems.layout`, for those of `sgf-diagrams` and `sgf-problem`, on top of those of `layout`.

`layout.columns` `auto` or a number — initially `auto`

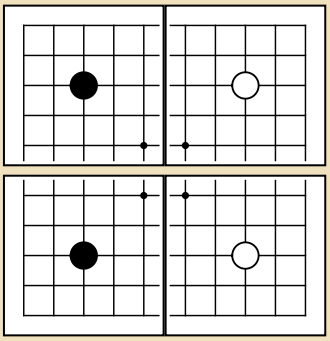
The number of columns of the grid. With `auto`, the diagrams go in the paragraph, without a grid.

`layout.width` a length, a fraction of the width or `auto` — initially `100%`

The width of the grid. If it is less than that of the line, the grid is centered. Each column measures what is left after subtracting the `column-sep`s from `width`, shared among the columns; a narrower board lies in its column according to `align`, so between two boards there is more space than `column-sep`.

With `auto`, the grid adjusts to the diagrams: the columns have the width of the widest diagram (with its caption, which is broken as without a grid, p. 90) and between two boards of the same width there is exactly `column-sep`. The grid measures that times the number of columns (or of diagrams, if there are fewer) and is centered in the line. `fit` does not work with `width: auto`: it is an error.

```
#goban-layout(columns: 2, width: auto,
  column-sep: 0em, row-sep: 0em,
  board(partial: "A9:E5", scale: 0.9,
    stones(black: "C7")),
  board(partial: "E9:J5", scale: 0.9,
    stones(white: "G7")),
  board(partial: "A1:E5", scale: 0.9,
    stones(black: "C3")),
  board(partial: "E1:J5", scale: 0.9,
    stones(white: "G3")))
```



`layout.column-sep` a length — initially `1em`

The space between the columns.

`layout.row-sep` a length — initially `1em`

The space between the rows.

`layout.fit` `true` or `false` — initially `false`

Scales each board so that it fills the width of its column; it replaces the `scale` of the board. The lines do not change width. In `sgf-diagrams` and `sgf-problem`, every board uses the scale that fills the column with the first one, so the stones have the same size in all of them; if with that scale a board does not fit in the column (a problem wider than the first one, for example), only that board is shrunk to fill it. Only with `columns`, and not with `width: auto`. Without an available width (a layout measured with `measure` without `width`), the grid takes the width of the diagrams, without `fit`.

`layout.align` `left`, `center` or `right` initially `center`

Where each diagram goes within its column, with its caption and its notes. The comments always go according to `comments.align`.

```
#sgf-diagrams(game, step: 2, to: 6,
  layout: (columns: 2, width: 60%,
    column-sep: 3em, row-sep: 2em,
    align: left))
```

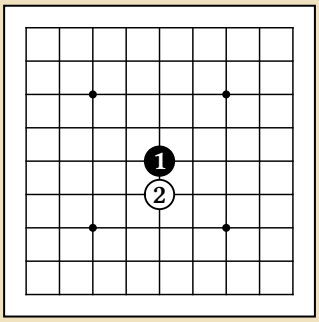


Diagram 1 (1–2)

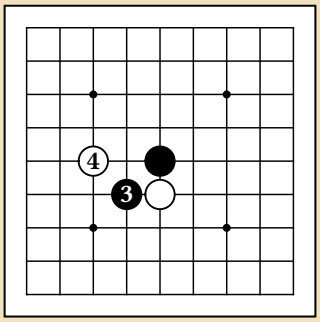


Diagram 2 (3–4)

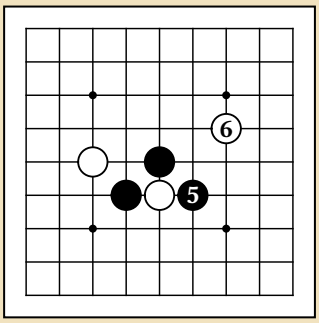


Diagram 3 (5–6)

`layout.per-page` `none` or a number initially `none`

Every so many diagrams, a new page starts: with `per-page: 1`, one diagram per page. In the grid, a half-filled row is drawn before the break; the number should be a multiple of `columns`. With `none`, there are no breaks. In `sgf-problem` with `statement: "alone"`, the statement counts as a diagram. With several page columns (Typst's `columns`, or `columns` in `page`), the break moves to the next column.

```
#sgf-diagrams(lee, step: 50, layout: (columns: 2, fit: true, per-page: 4))
```

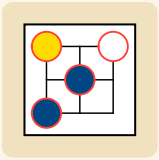
12 Players and profiles

Each stone, move and mark belongs to one of three players: the first one (`black`), the second one (`white`) or the neutral one (`neutral`). Their look is not in each element but in the player: the branch of each one gathers the options of their stones, their prisoners, their moves, their marks and their territory. Here are the players and, after them, the profiles.

1 Players

`players` a dictionary

The options of the players: `neutral`, `black` and `white`, and `all`, which goes to the three.



```
#gobanist-set(players: (
  all: (stone: (line: (color: red))),
  neutral: (stone: (fill: (color: yellow))),
  black: (stone: (fill: (
    color: blue.darken(40%))))))

#goban(size: 3,
  stones(black: "A1, B2", white: "C3"),
  stones("A3"))
```

`players.all` player options

The options of the three players.

`players.neutral` player options

The options of the neutral player: the stones and the marks without a player.

`players.black` player options

The options of the first player.

`players.white` player options

The options of the second player.

Each player has the same tree of options. Each branch is documented with the object it applies to, without the player in the path: for the white player, `stone.radius` is `players.white.stone.radius`.

`players.white.stone` stone options

The style of their stones (p. 54). `stone` at the first level is `players.all.stone`.

`players.white.prisoner` stone options

The style of their prisoners, on top of that of their stones (p. 62). `prisoner` at the first level is `players.all.prisoner`.

`players.white.move` move options

The style of their moves (p. 60). `move` at the first level is `players.all.move`.

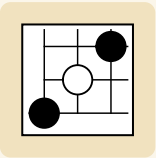
`players.white.mark` mark options

The style of their marks (p. 64). The shape is not part of the style: it goes in each `marks`. `mark` at the first level is `players.all.mark`.

`players.white.territory` territory options

The style of their territory (p. 73).

Copying a player into another is copying their branch: the tree is a value.



```
#gobanist-set(players: (neutral: (stone: (
  fill: (color: white)))))

// Neutral, but with the look of White.
#goban(partial: "A1:C3",
  stones("B2"), stones(black: "A1, C3"))
```

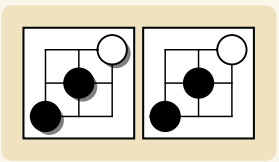
2 Profiles

A profile is a complete style, for the board and the players, saved as a dictionary of options: the same as those of `gobanist-set`. The profiles of the package are named with a string (p. 106); one of your own is a dictionary.

`profile` the name of a profile or a dictionary

Applies a profile: `profile: "wain"` uses one of the package, and `profile: (background: ..., players: ...)`, one of your own. It goes in `gobanist-set`, in the options of a board and in `view.goban`.

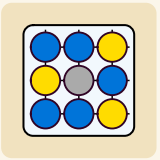
Before applying it, the board and the players go back to the initial style (that of the `default` profile): that way a profile does not inherit what it does not define, and a shadow or an image set before is lost. What is not style stays as it was: `size`, `use-i`, `partial`, `partial-margin`, `scale`, `scale-line-width`, `label.at`, `label.columns`, `label.rows` and their directions, `hoshi.at`, `move.numbers`, the arrows, the lines, the game and the branches of the games, the diagrams and the views. What goes after the profile is added to it.



```
#gobanist-set(stone: (shadow: (on: true,
  angle: -45deg, offset: 2pt)))

#goban(size: 3,
  stones(black: "A1, B2", white: "C3"))
#goban(size: 3, profile: "default",
  stones(black: "A1, B2", white: "C3"))
```

A `profile` inside a profile is the profile it starts from: `pgfgosmooth` is `(profile: pgfgo, ...)`.



```
#let example = (
  background: (fill: (color: blue.lighten(95%))),
  players: (
    all: (stone: (line: (
      color: fuchsia.darken(80%)))),
    neutral: (stone: (fill: (color: gray))),
    white: (stone: (fill: (color: yellow))),
    black: (stone: (fill: (color: blue))),
  ),
)
#let example2 = (profile: example,
  background: (rounded-corners: 4pt))

#gobanist-set(profile: example2)

#goban(partial: "A1:C3",
  moves("A1:3", numbers: false),
  stones(black: "B1, B3, C2", white: "C1, C3"),
  stones("B2"))
```

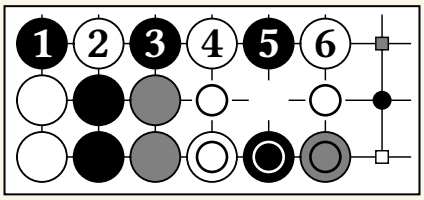
A profile of your own can be saved in a module and imported in any document (p. 22).

13 Catalog of profiles

The profiles that come with gobanist, each one with a sample and with its definition, as it is in `src/profiles.typ`. They are used with `profile: "name"` (p. 105). In the definitions, `xmix(a, p, b)` mixes `p` of `a` with the rest of `b`, like xcolor's `a!p!b`; `pgf-gray` is `luma(50%)`; and `picture("board.jpg")` is the image `images/pgfgo-board.jpg` of the package.

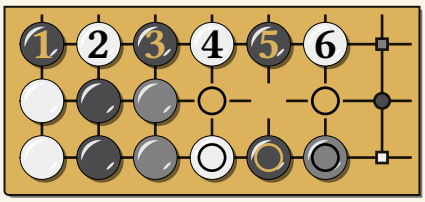
1 default

The initial style of the package: the one the board and the players have before a profile is used, and the one they go back to before any other is applied. Its definition is the empty dictionary: it is the tree of initial values of `src/options.typ`.



2 pgfgo

A light wooden board, with a shadow and barely rounded corners, and stones with shadow and shine. It uses the colors of `wain` and the shadow and shine of `raised`, defined here.




```
#let pgfgo = (
  background: (
    fill: (color: pgfgo-color, opacity: 1),
    line: (color: pgfgo-dark, opacity: 1),
    shadow: true,
    rounded-corners: (inner: 0pt, outer: 2pt),
  ),
  grid: (line: (color: pgfgo-dark, width: 1pt)),
  players: (
    all: (
      stone: raised + (line: (on: true, color: black, width: 0.3pt, opacity: 1), radius:
0.17cm),
      mark: (line: (color: black, width: 1pt), label: (text: (fill: black))),
    ),
    white: (stone: (fill: (color: wain-white), shine: (color: white)), move: (label:
(text: (fill: black)))),
    black: (stone: (fill: (color: wain-black)), move: (label: (text: (fill: pgfgo-
color))), mark: (line: (color: pgfgo-color))),
```

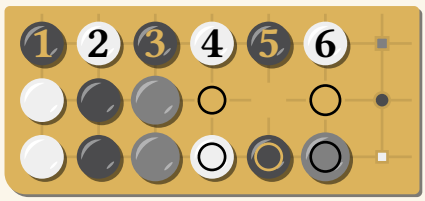
```
  ),
)
```

```
#let pgf-gray = luma(50%)
#let wain-goban = rgb("#402A2D")
#let wain-grid = rgb("#FAE0B2")
#let wain-black = rgb("#4B4B4D")
#let wain-shade = rgb("#E6E7E8")
#let wain-white = rgb("#EFEFEF")
#let pgfgo-color = rgb(220, 179, 92)
#let pgfgo-dark = xmix(pgfgo-color, 10%, black)
#let pgfgosmooth-grid = rgb("#C7A253")
```

```
#let raised = (
  shadow: (on: true, opacity: 0.5, offset: 1pt, angle: -45deg),
  shine: (on: true, angle: 45deg, color: wain-shade),
)
```

3 pgfgosmooth

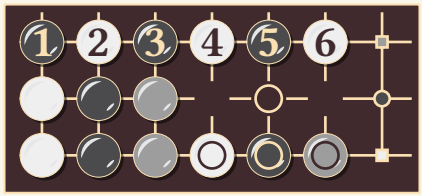
Like `pgfgo`, without the border of the board or that of the stones, with more rounded corners.




```
#let pgfgosmooth = (
  profile: pgfgo,
  background: (line: false, rounded-corners: (inner: 2pt, outer: 8pt)),
  grid: (line: (color: pgfgosmooth-grid)),
  players: (
    neutral: (stone: (line: false, radius: 0.19cm, shine: (opacity: 0.5))),
    black: (stone: (line: false, shine: (opacity: 0.5))),
    white: (stone: (line: false)),
  ),
)
```

4 wain

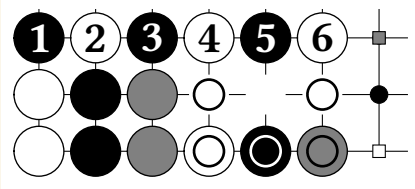
A dark board, with light lines, and stones with shadow and shine. Its colors and `raised` are in `pgfgo`.



```
#let wain = (
  background: (fill: (color: wain-goban, opacity: 1), line: (color: wain-grid, opacity:
1)),
  grid: (line: (color: wain-grid, width: 1pt)),
  label: (text: (fill: wain-goban)),
  players: (
    all: (
      stone: raised + (line: (on: true, color: wain-grid, width: 0.2pt, opacity: 1),
radius: 0.17cm),
      move: (label: (text: (fill: black))),
      mark: (line: (color: wain-goban, width: 1pt), label: (text: (fill: wain-goban))),
    ),
    neutral: (stone: (fill: (color: xmix(wain-black, 50%, wain-white)))),
    white: (stone: (fill: (color: wain-white)), move: (label: (text: (fill: wain-
goban)))),
    black: (
      stone: (fill: (color: wain-black)),
      move: (label: (text: (fill: wain-grid))),
      mark: (line: (color: wain-grid), label: (text: (fill: wain-grid))),
    ),
  ),
)
```

5 simple

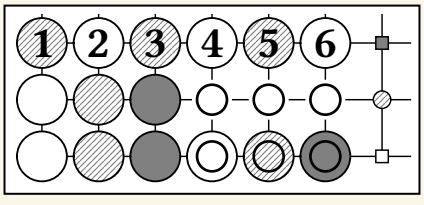
Like the initial style, without the border of the board and with thinner lines.




```
#let simple = (
  background: (line: false),
  grid: (line: (width: 0.4pt)),
  players: (all: (stone: (line: (width: 0.5pt)))),
)
```

6 draft

Black and white, for printing or photocopying: where there is a fill, stripes. The black stones, the filled marks and the territory use `fill.pattern`; the territory, with stripes in one direction for each player, is seen with `draw: "fill"` or `draw: "region"`. `lines` draws the stripes: a `tiling` of lines at 45 or −45 degrees.

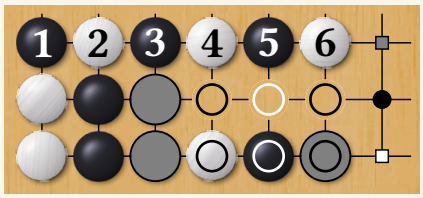


```
#let draft = (
  players: (
    all: (mark: (fill: (color: white, pattern: lines(45deg, 1.2pt, 0.3pt, black)))),
    black: (
      stone: (fill: (color: white, pattern: lines(45deg, 1.5pt, 0.4pt, pgf-gray))),
      move: (label: (text: (fill: black))),
      mark: (line: (color: black), label: (text: (fill: black))),
      territory: (fill: (color: none, pattern: lines(45deg, 2pt, 0.4pt, pgf-gray))),
    ),
    white: (territory: (fill: (color: none, pattern: lines(-45deg, 2pt, 0.4pt, pgf-
gray)))),
  ),
)
```

```
#let lines(angle, distance, width, color) = {
  let s = distance * calc.sqrt(2)
  tiling(size: (s, s), for k in (-1, 0, 1) {
    let (y1, y2) = if angle > 0deg { (s, 0pt) } else { (0pt, s) }
    place(line(start: (k * s, y1), end: (k * s + s, y2), stroke: (paint: color,
thickness: width)))
  })
}
```

7 shell

A wooden board with slate and shell stones, drawn with the images in the `images` folder of the package. Each white stone uses one of five images, with a small rotation. The shadow is a soft image (`soft-shadow`).

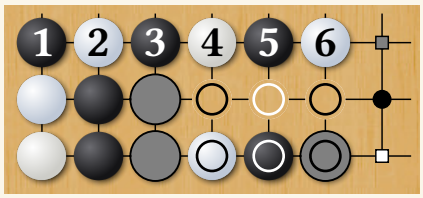



```
#let shell = (
  background: (line: false, fill: (image: picture("board.jpg"))),
  players: (
    all: soft-shadow,
    black: (stone: (image: picture("shell-black.png"))),
    white: (stone: (image: (
      file: range(1, 6).map(n => picture("shell-white-" + str(n) + ".png")),
      rotate: (-15deg, 15deg),
    ))),
  ),
)
```

```
#let picture(name) = path("../images/pgfgo-" + name)
#let soft-shadow = (stone: (shadow: (
  on: true, angle: -45deg, offset: 0.75pt, scale: 1.2, opacity: 0.6, image:
picture("shadow.png"),
)))
```

8 yunzi

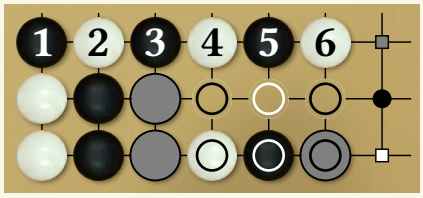
Like `shell`, with matte stones, without grain, like the Chinese *yunzi*: two images for White, without rotation, because the light of each image comes from one side.




```
#let yunzi = (
  profile: shell,
  players: (
    black: (stone: (image: picture("yunzi-black.png"))),
    white: (stone: (image: (file: (picture("yunzi-white-1.png"), picture("yunzi-
white-2.png")), rotate: 0deg))),
  ),
)
```

9 baduktv

A light board, of plain wood, with plain stones, one image for each color, without rotation; the soft shadow is that of `shell`.




```
#let baduktv = (
  background: (line: false, fill: (image: picture("baduktv-goban.png"))),
  players: (
    all: soft-shadow,
    black: (stone: (image: picture("baduktv-black.png"))),
    white: (stone: (image: picture("baduktv-white.png"))),
  ),
)
```

14 News

- 0.1.0: first version. It is a port of pgf-go 0.2026.10.23, with the same ideas and almost all of its options, written with Typst values (p. 5). The complete list of changes is in the file `CHANGELOG.md`.

Index of functions and options

annotations ... 89
arrow.bend ... 70
arrow.color ... 69
arrow.dash ... 70
arrow.halo ... 70
arrow.halo.color ... 70
arrow.halo.width ... 70
arrow.opacity ... 69
arrow.shorten ... 70
arrow.tip ... 70
arrow.tips ... 70
arrow.width ... 69
arrows ... 69
background ... 46
background.fill ... 46
background.fill.color ... 46
background.fill.image ... 47
background.fill.image.anchor ... 48
background.fill.image.file ... 47
background.fill.image.fit ... 47
background.fill.image.rotate ... 48
background.fill.image.tiles ... 48
background.fill.image.width ... 48
background.fill.opacity ... 46
background.fill.pattern ... 46
background.line ... 49
background.line.color ... 49
background.line.opacity ... 49
background.line.width ... 49
background.rounded-corners ... 49
background.sep ... 49
background.shadow ... 49
background.shadow.angle ... 50
background.shadow.color ... 50
background.shadow.offset ... 50
background.shadow.opacity ... 50
board ... 40
captured-marks ... 72
captures ... 72
charset ... 85
chat ... 90
circle ... 64
content ... 65
cross ... 64
custom ... 65
diagrams.annotations ... 95
diagrams.caption ... 92
diagrams.caption.min-width ... 92
diagrams.caption.text ... 92
diagrams.chat ... 95
diagrams.comments ... 94
diagrams.comments.align ... 94
diagrams.comments.height ... 95
diagrams.comments.justify ... 94
diagrams.comments.number ... 94
diagrams.comments.position ... 94
diagrams.comments.root ... 94
diagrams.comments.sep ... 95
diagrams.comments.separator ... 94
diagrams.comments.text ... 94
diagrams.comments.width ... 95
diagrams.figures ... 93
diagrams.from ... 90
diagrams.goban ... 92
diagrams.last-mark ... 91
diagrams.layout ... 92
diagrams.marks ... 91
diagrams.notes ... 91
diagrams.numbers ... 91
diagrams.partial ... 93
diagrams.restart ... 91
diagrams.restart.from ... 91
diagrams.result ... 95
diagrams.result.black ... 96
diagrams.result.format ... 96
diagrams.result.title ... 96
diagrams.result.white ... 96
diagrams.step ... 90
diagrams.title ... 91
diagrams.to ... 90
diagrams.variation ... 92
diagrams.variation-title ... 93
diagrams.variations ... 92
diagrams.variations.depth ... 93
diagrams.variations.from ... 93
diagrams.variations.paths ... 93
diagrams.variations.select ... 93
diagrams.view ... 93
dim ... 65
filled-circle ... 65
filled-square ... 65
forget ... 60, 62, 68
from ... 61, 65
game ... 85
goban ... 40
goban-layout ... 101
goban-notes ... 73
goban-prisoners ... 72
goban-region ... 78
goban-score ... 78
goban-view ... 81
gobanist-set ... 30
grid ... 42
grid.image ... 43

grid.image.file ... 43
grid.image.fit ... 43
grid.image.rotate ... 44
grid.image.tiles ... 44
grid.line ... 42
grid.line.color ... 42
grid.line.opacity ... 42
grid.line.width ... 42
grid.sep ... 42
hatched-lines ... 65
hoshi ... 44
hoshi.at ... 45
hoshi.color ... 44
hoshi.image ... 45
hoshi.image.file ... 45
hoshi.image.rotate ... 45
hoshi.image.width ... 45
hoshi.opacity ... 44
hoshi.radius ... 44
label.at ... 51
label.columns ... 52
label.columns-direction ... 52
label.rows ... 52
label.rows-direction ... 52
label.sep ... 51
label.style ... 52
label.text ... 51
layout ... 102
layout.align ... 103
layout.column-sep ... 102
layout.columns ... 102
layout.fit ... 102
layout.per-page ... 103
layout.row-sep ... 102
layout.width ... 102
line.bend ... 71
line.color ... 71
line.dash ... 71
line.halo ... 71
line.opacity ... 71
line.shorten ... 71
line.width ... 71
lines ... 69
mark ... 31, 60
mark.fill ... 67
mark.fill.color ... 67
mark.fill.opacity ... 67
mark.fill.pattern ... 67
mark.label ... 67
mark.label.format ... 67
mark.label.size ... 67
mark.label.text ... 68
mark.line ... 66
mark.line.color ... 66
mark.line.opacity ... 66
mark.line.width ... 66
mark.scale ... 66
marks ... 64
move ... 31
move.label ... 61
move.label.format ... 61
move.label.size ... 61
move.label.text ... 62
move.numbers ... 61
moves ... 60
notes ... 73
notes.at ... 73
notes.separator ... 73
partial ... 50
partial-margin ... 50
player ... 66
players ... 31, 104
players.all ... 104
players.black ... 104
players.black.territory.fill ... 76
players.black.territory.fill.color ... 76
players.black.territory.fill.opacity ... 76
players.black.territory.fill.pattern ... 76
players.black.territory.line ... 76
players.black.territory.line.above ... 77
players.black.territory.line.color ... 77
players.black.territory.line.opacity ... 77
players.black.territory.line.width ... 77
players.black.territory.mark ... 76
players.black.territory.vertices ... 77
players.black.territory.vertices.color . 77
players.black.territory.vertices.radius 77
players.neutral ... 104
players.white ... 104
players.white.mark ... 104
players.white.move ... 104
players.white.prisoner ... 104
players.white.stone ... 104
players.white.territory ... 105
prisoner ... 31, 62
prisoners ... 62
problems.annotations ... 99
problems.caption ... 99
problems.chat ... 99
problems.comments ... 99
problems.goban ... 99
problems.layout ... 100
problems.notes ... 99
problems.partial ... 99
problems.solutions ... 98
problems.statement ... 97
problems.title ... 98
problems.to-play ... 98
problems.variation-title ... 98
problems.verdict ... 98
problems.view ... 98
profile ... 105
resume ... 80
resume.marks ... 81
resume.numbers ... 80
resume.territory ... 81
scale ... 41
scale-line-width ... 41
score.komi ... 78
sequence ... 65

sgf ... 85
sgf-comment ... 89
sgf-diagrams ... 90
sgf-info ... 88
sgf-problem ... 96
sgf-read ... 84
sgf.from ... 85
sgf.last-mark ... 88
sgf.letters ... 88
sgf.marks ... 86
sgf.numbers ... 87
sgf.restart ... 87
sgf.restart.from ... 87
sgf.step ... 87
sgf.to ... 86
sgf.variation ... 86
sgf.view ... 88
shapes ... 68
size ... 41
square ... 64
stone ... 31
stone.fill ... 54
stone.fill.color ... 54
stone.fill.opacity ... 54
stone.fill.pattern ... 55
stone.image ... 58
stone.image.file ... 59
stone.image.opacity ... 59
stone.image.rotate ... 59
stone.image.width ... 59
stone.line ... 55
stone.line.color ... 55
stone.line.opacity ... 55
stone.line.width ... 55
stone.radius ... 55
stone.scale ... 55
stone.shadow ... 56
stone.shadow.angle ... 56
stone.shadow.color ... 56
stone.shadow.image ... 56
stone.shadow.image.file ... 57
stone.shadow.image.rotate ... 57
stone.shadow.image.silhouette ... 57
stone.shadow.image.width ... 57
stone.shadow.offset ... 56
stone.shadow.opacity ... 56
stone.shadow.scale ... 56
stone.shine ... 57
stone.shine.angle ... 57
stone.shine.color ... 57
stone.shine.image ... 58
stone.shine.image.file ... 58
stone.shine.image.rotate ... 58
stone.shine.image.silhouette ... 58
stone.shine.image.width ... 58
stone.shine.opacity ... 57
stones ... 54
t-circle ... 65
t-square ... 65
territory ... 74
territory.dame ... 75
territory.draw ... 75
texts ... 32
triangle ... 64
use-i ... 41
view ... 82
view.arrows ... 82
view.forgotten ... 82
view.goban ... 83
view.lines ... 82
view.marks ... 82
view.numbers ... 82
view.raise ... 83
view.territory ... 82